



Automated build and test system with MPLAB[®]X IDE and MPLAB XC Compilers (24014DEV4)

Stefan Luethin, MCHP-Germany



Objectives

The **goals** of this class

- CICD stands for ContinuousIntegration **and** ContinuousDeployment but what does it mean and what problem does it solve?
- But why this class, doesn't the MPLAB®X CICD wizard do all for me?
- Well, that is the problem: if you just heard about CICD, Jenkins, Docker than CICD-wizard is two steps too far
 - This class **will try** to *bridge that gap*, explain the background of CICD, step-by-step introduce all parts and how they interact, so the user can get the most out of MPLAB®X CICD wizard
 - But **will NOT** teach all features of git, Jenkins or Docker BUT its goals are:
 - be an **introduction** into CICD-methodology, not covering all possible angles and features
 - It'll lay the base to understand the MPLAB®X CICD-wizard outputs and how to use them
 - and finally show the added value enabled with MPLAB®X features and capabilities
- We will
 - first look at MLABX®X CICD-wizard and its outputs -> identifies knowledge holes
 - review major pillars of a CICD system, how they interact and how to setup all
 - look at the added value of MPLAB®X ecosystems' test and analysis capabilities
 - finally, go back to the MLABX®X CICD-wizard -> hopefully now understand

Agenda

- Introduction to CICD
- MPLAB®X ecosystem testing features and analysis capabilities
- [Demos \(part1\)](#): used FW and expected result
- Three pillars of a CICD system:
 - SourceControlManagement=SCM/git
 - Automation/Jenkins: Jenkins-Master -> -Agent -> Pipeline -> [Demo \(bash-in-Pipe\)](#)
 - RuntimeEnv/PC-Docker
- [Demos \(part2\)](#): simple-pipeline, Jenkins-Trigger, errors+olutions
- MPLAB®X CICD wizard
- [Demos \(part3\)](#): HW-in-the-loop testing, DockerContainer
- Summary and Outlook

Agenda

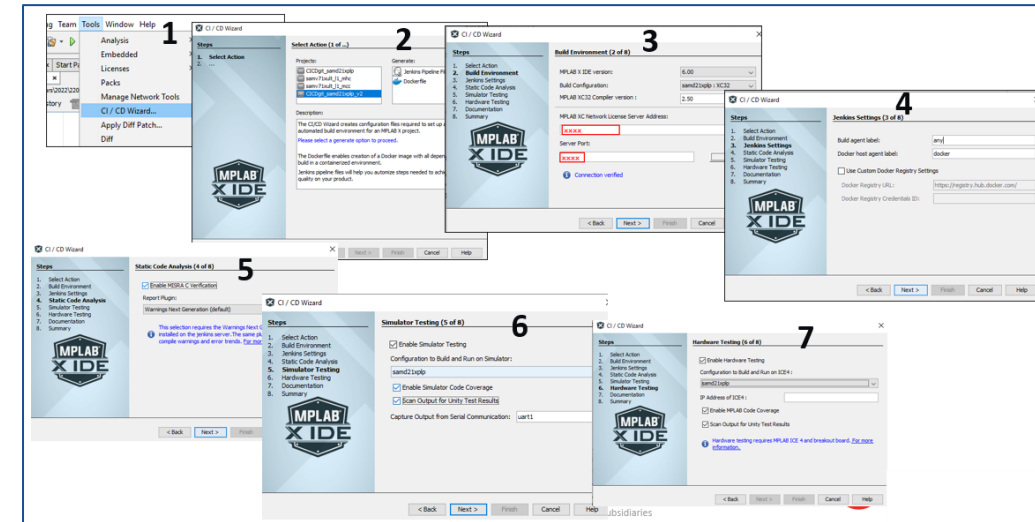
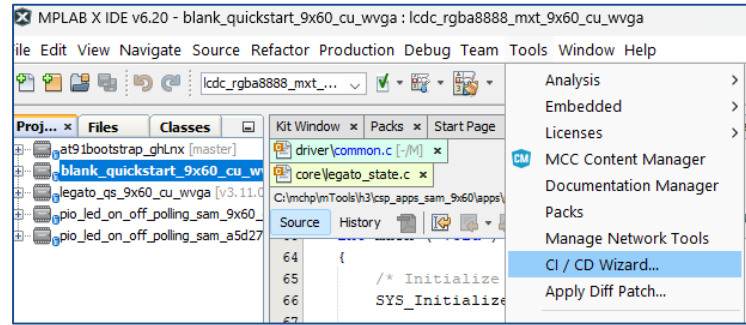
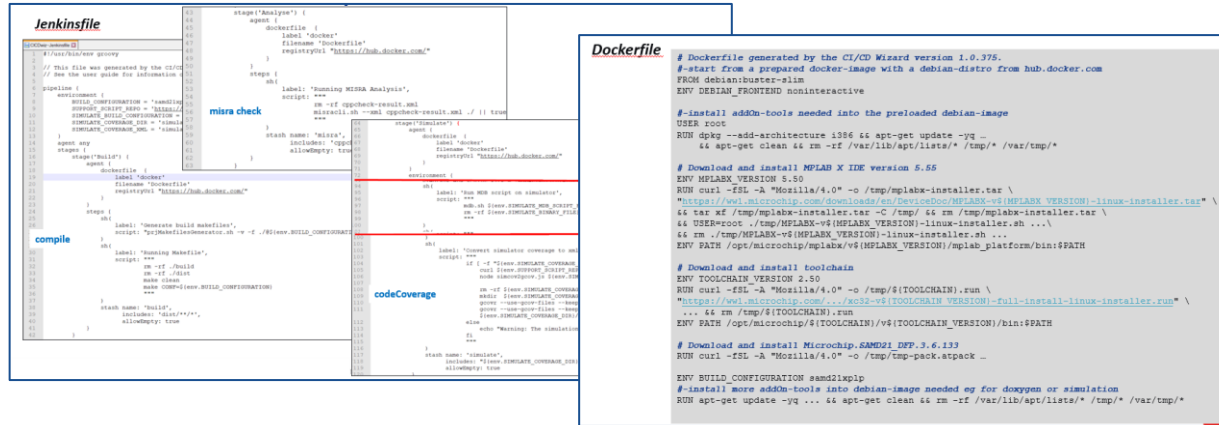
- **Introduction to CICD**
- MPLAB®X ecosystem testing features and analysis capabilities
- Demos (part1): used FW and expected result
- Three pillars of a CICD system:
 - SourceControlManagement=SCM/git
 - Automation/Jenkins: Jenkins-Master -> -Agent -> Pipeline -> Demo (bash-in-Pipe)
 - RuntimeEnv/PC-Docker
- Demos (part2): simple-pipeline, Jenkins-Trigger, errors+solutions
- MPLAB®X CICD wizard
- Demos (part3): HW-in-the-loop testing, DockerContainer
- Summary and Outlook

Introduction to CICD

- **CICD in a nutshell - the What, Why and How**
 - **What** does CICD mean and what is it?
 - CICD = ContinuousIntegration and ContinuousDeployment
 - CICD is not **one** tool BUT a **methodology** with 3 major pillars (below)
 - **Why** CICD? Which problem does it solve?
 - divide-and-conquer, **small-steps** and **continuously** verify after each change **automatically**
 - **How** does CICD work? The 3 major pillars are
 - Source-Control-Management/SCM like **git** (**pillar1**)
 - Automation like **Jenkins** (**pillar2**)
 - Runtime-Environment like **Docker** (**pillar3**)
- Next, let's have a brief look at the CICD-wizard and its outputs, to see the knowledge-holes

Introduction to CICD

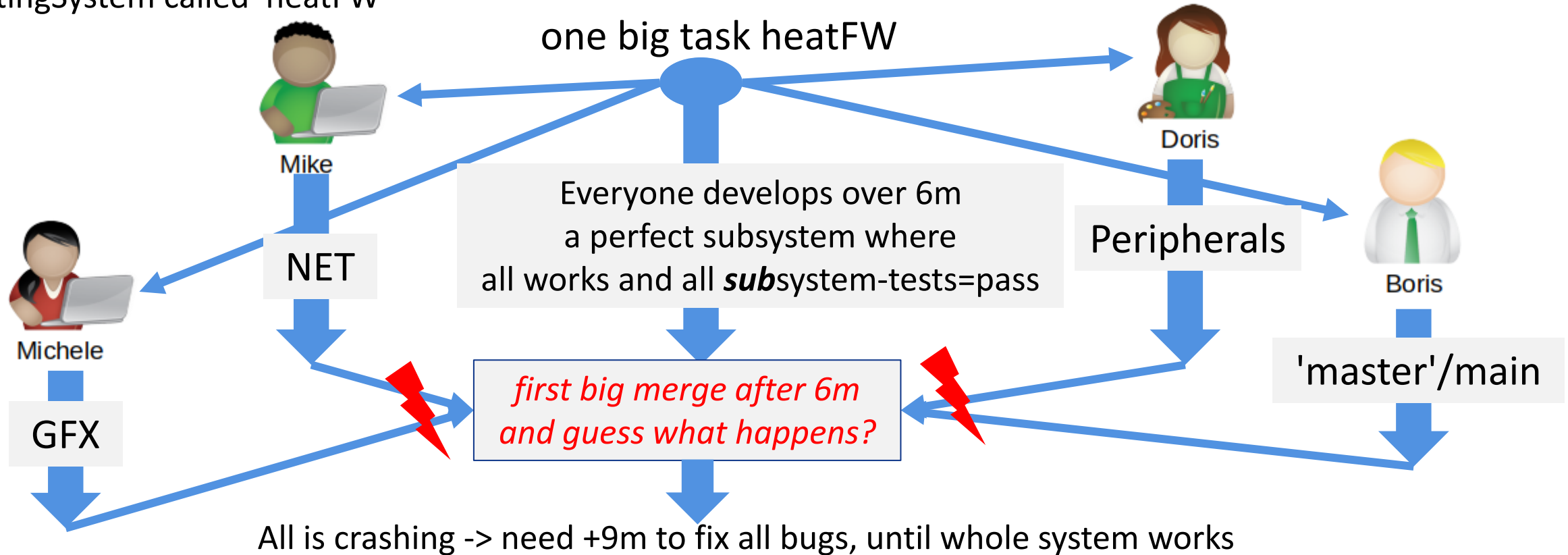
- The MPLAB®X CICD wizard
 - What is the MPLAB®X CICD wizard?
 - Some Q&A, but means what?
 - Result is two files 'Jenkinsfile', 'Dockerfile'



- But, what can I do with these files? Print them maybe? Or make paper planes out of it....?
- Further MPLAB's CICD-wizard userguide uses tool *cicdw.exe* - but what is it?
- First creates DockerContainer -> inside Jenkins-Master&-Node installed+started -> finally pipeline started ...
- Sounds complex and what if something goes wrong?
- **Goal of this class is to explain CICD so you can get the most out of MPLAB®X-CICD wizard**

Intro: what problem does CI/CD solve?

Doris & Boris (location1) and Michele & Mike (location2) get **one** big task to develop new FW for heatingSystem called 'heatFW'



-> overall time: 6+9=15m, not good and project like lost BUT *how to do better?*

-> instead of **one big** step, do **continuous** small&short 'increments' and merges (**CI/CD**)
and test all parts and provide small working increments (**CI/CD**)

Intro: capabilities needed for a CI/CD system

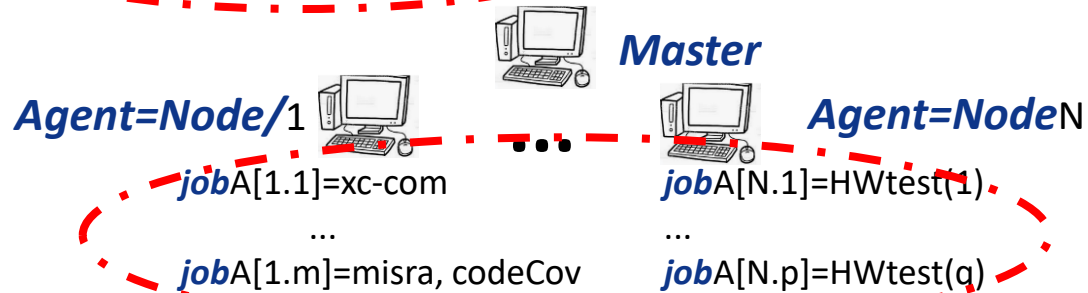
- What functionalities are needed for such a CI/CD-challenge?
 - First, need some way to handle different FW-releases
-> task of a Source-Control-Management/**SCM** system, like **git** (->CI/CD-pillar**1**)
 - Second, some automation with two major functions
 - define **list/sequence of tasks** to run, eg: get *FW-revA*, *compile&run on HW-rev4*
 - Some **automation** to **trigger** task-sequence regularly or on certain **events** like new check-into-release-branch OR everyWe@9pm
->task of the Automation system, like **Jenkins** (->CI/CD-pillar**2**)
 - Third need some **runtime environment** to execute all this on
-> set of **PCs** or **DockerContainer** (->CI/CD-pillar**3**)

Intro: capabilities needed for a CI/CD system

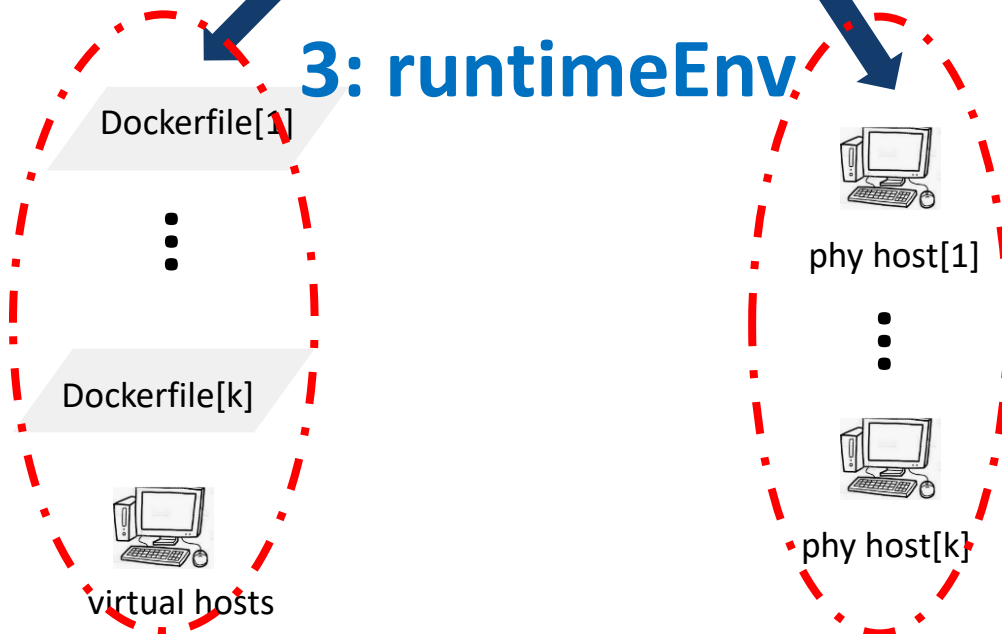
- Besides these 3 functions mentioned SCM->Automation->runTimeEnv, anything else?
- Well, besides a 'simple' fetch&compile&link what about Testing&Analysis?
- What and how depends on various topics but includes:
 - at least compile of all sources of a certain FW-revision but likely more, right?
 - MISRA compliance -> so MISRA check needed, but maybe not always!?
 - codeCoverage -> how often should this run?
 - Multiple HW-platforms? -> different FW-revision might have to run on different HW
 - ...
- So, it's always a task-sequence and some kind-of trigger
- And the task-sequence might include more than just a fetch&compile&link
- Therefore, will look at MPLAB®X analysis- and testing-capabilities, but first, let's look at the structure of a complete CI/CD system with Jenkins next

Intro: Major parts of a CI/CD-system (overview)

2: Automation



3: runtimeEnv

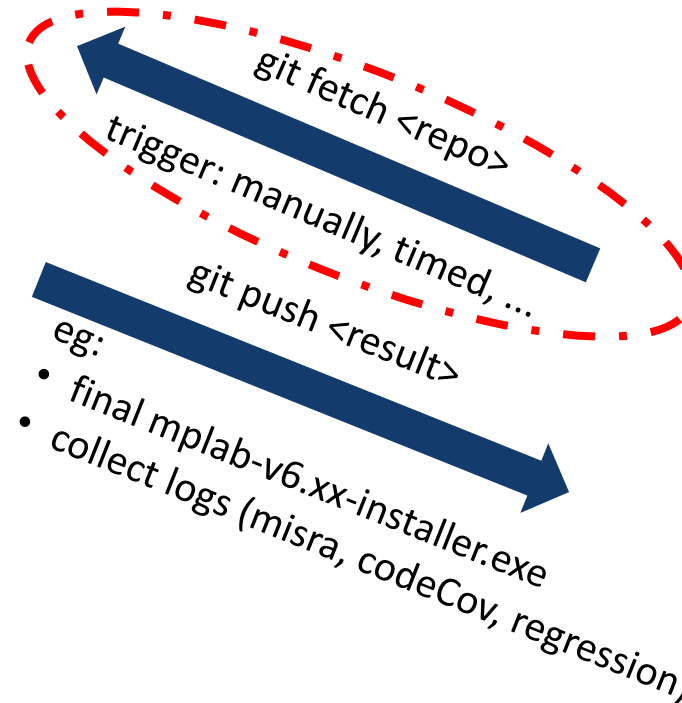


1: SCM



github.com

stefanluethin-microchip
|-project1\
|- ...
|-projectK\



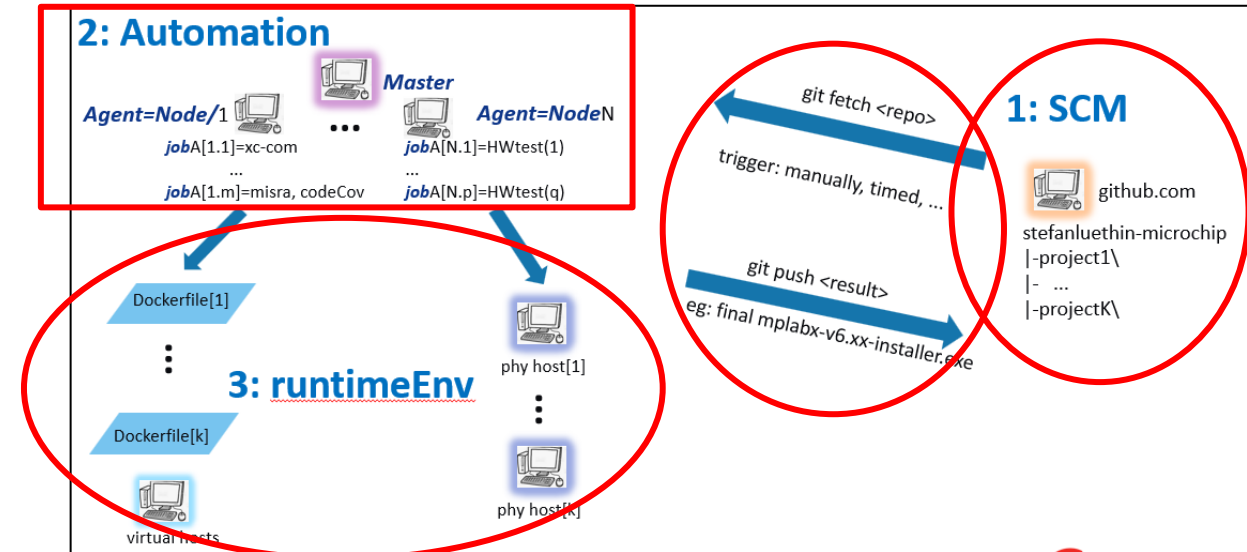
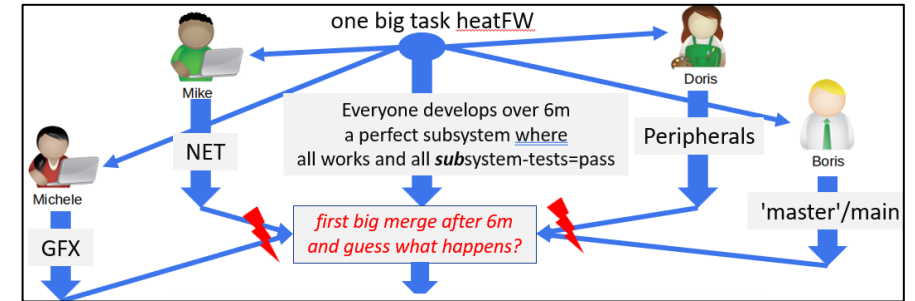
The Automation-pillar of a CI/CD-system must provide 2 functions:

- First, need to somehow define+run a **sequence of steps**
-> Jenkins-**Pipeline** used in Job-description
- Need to have ways to trigger this sequence on certain **events**
-> Jenkins-**Trigger** set in Job-description

Intro: a CI/CD-system (summary)

Summary why, what and how:

- **Why** CI/CD
 - Different terms but concept always same:
->divide&conquer, Scrum, Agile, ... -> small steps & continuous
- **What** problem solved
 - Reduce risk for unexpected delays
 - Reliable project planning
- **How**, the three pillars of a CI/CD system
 - Pillar**1** source-control-management/SCM:
 - FW-releases -> **git**
 - Pillar**2** Automation:
define+trigger a sequence of tasks -> **Jenkins**
 - Pillar**3** runtime environment:
realPC(s) or **DockerContainer**
- Two major functions of CI/CD-Automation part2/**Jenkins**
 - Provide some way to define a **sequence of tasks** -> Pipeline-definition
 - Provide ways to **trigger** defined task-sequence on **certain event** -> Pipeline-definition



Agenda

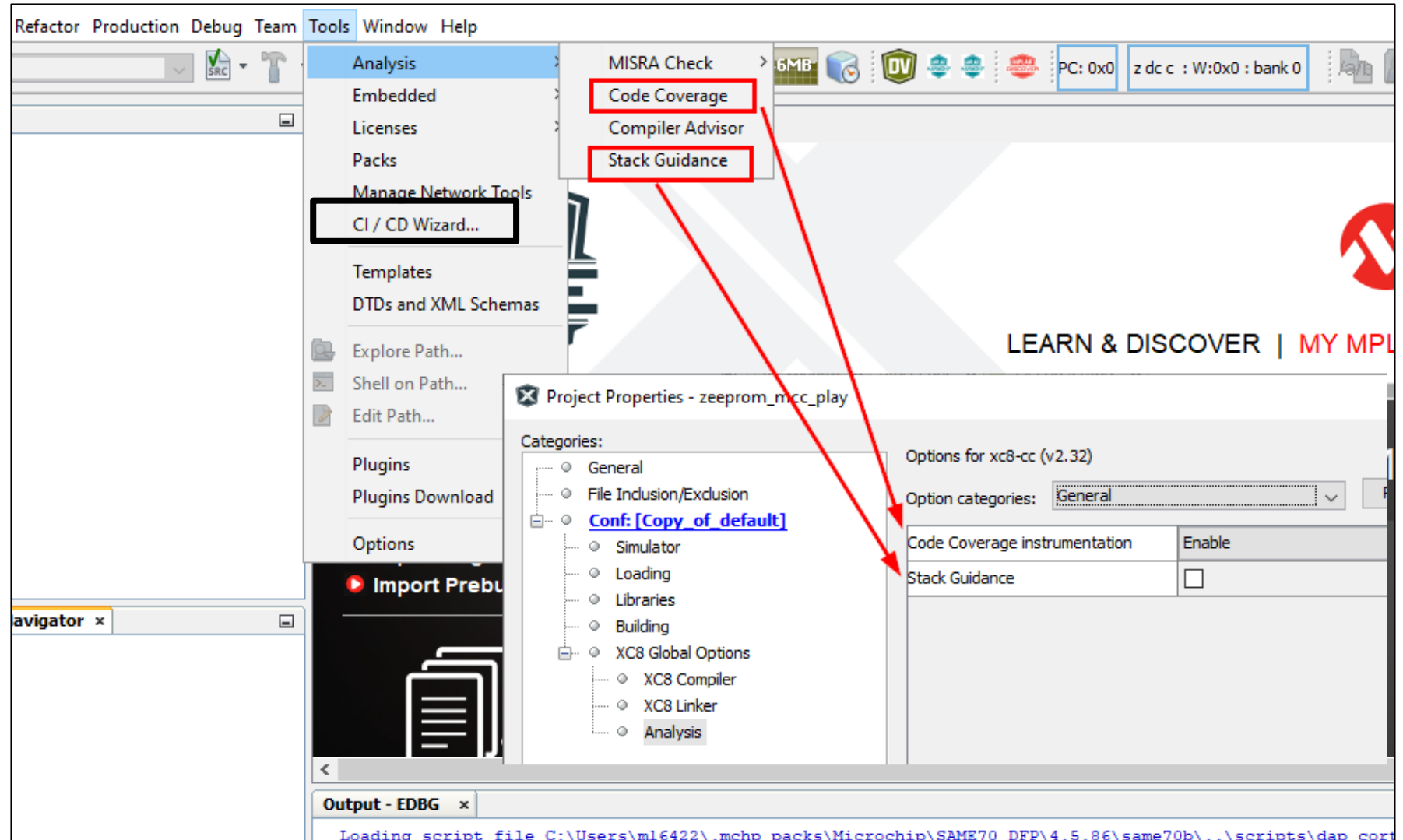
- Introduction to CICD
- **MPLAB®X ecosystem analysis capabilities and testing features**
- Demos (part1): used FW and expected result
- Three pillars of a CICD system:
 - SourceControlManagement=SCM/git
 - Automation/Jenkins: Jenkins-Master -> -Agent -> Pipeline -> Demo (bash-in-Pipe)
 - RuntimeEnv/PC-Docker
- Demos (part2): simple-pipeline, Jenkins-Trigger, errors+solutions
- MPLAB®X CICD wizard
- Demos (part3): HW-in-the-loop testing, DockerContainer
- Summary and Outlook

Analysis tool suite of MPLAB®X

MPLAB®X-analysis: Overview

New analysis tool suite start with MPLAB®X v6.00 (Q1.2022)

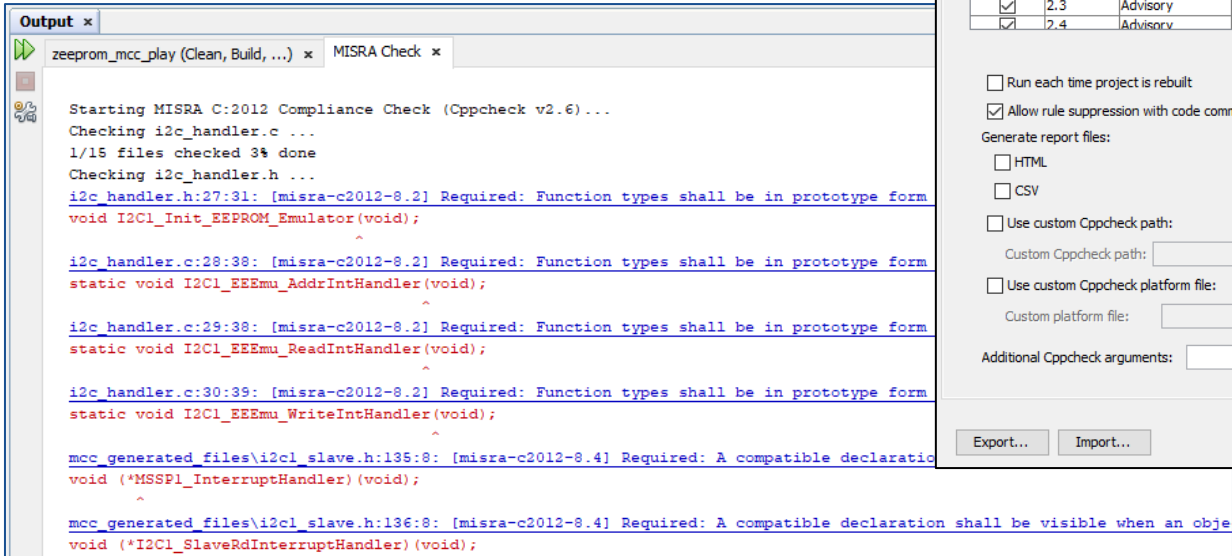
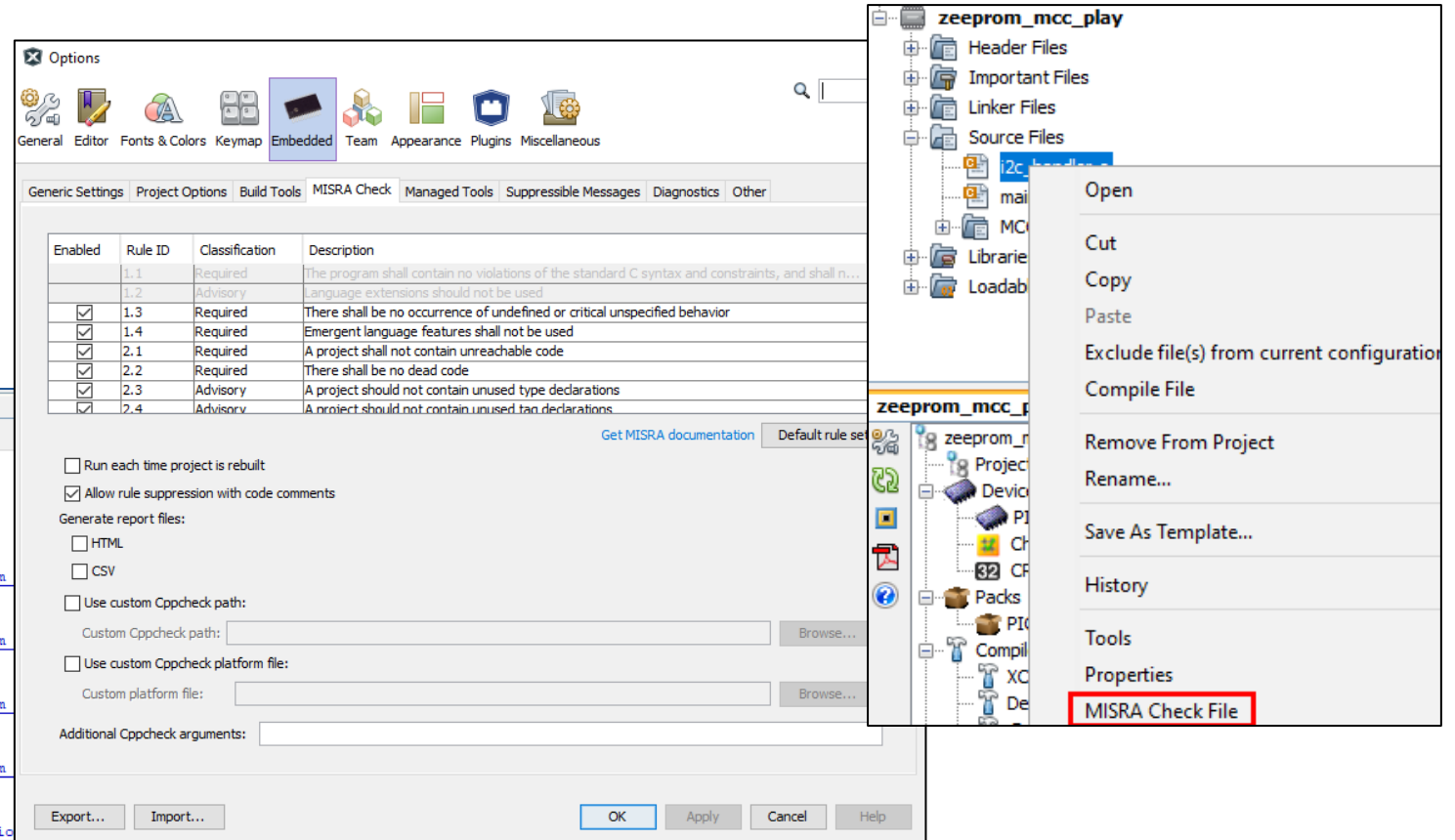
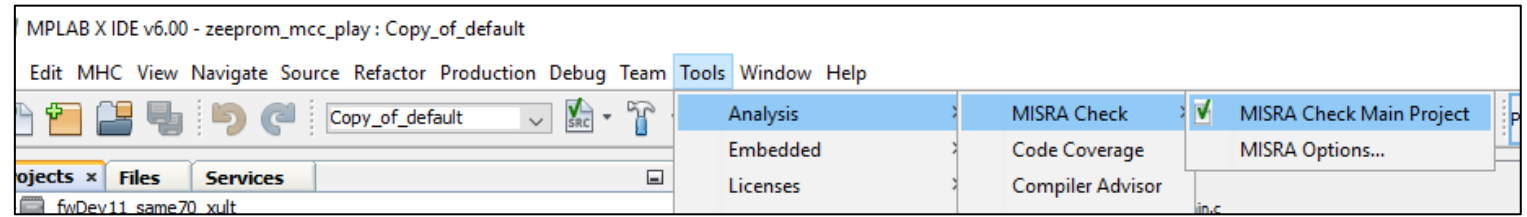
- MISRA checker
- Code Coverage
- Thread Safety
- CICD-wizard
- License:
*AnalysisToolSuite
license*



MPLAB®X-analysis: MISRA checker

MISRA-checker

- 3x types of MISRA-rules:
Mandatory, Required, Advisory
- Run on
 - prj-level
 - file-level
- Results with more details+examples available with errorID in Misra-doc (not free!)



MPLAB®X-analysis: Code Coverage

Code Coverage

- Run in IDE or on cmdLine (->Jenkins)
- Create html-report ->Jenkins artefacts

2:run

Color-coding AFTER CC-results are loaded -> disable CC to remove color-coding of source code

File Name	Address Units Covered	Address Units Not Covered	Coverage
pin_manager.c	55	0	100%
i2c_handler.c	28	52	35%
mccc	45	0	100%
OSCILLATOR_Initialize	11	0	100%
WDT_Initialize	6	0	100%
PMD_Initialize	10	0	100%
SYSTEM_Initialize	18	0	100%
main.c	11	0	100%
memory.c	0	46	0%
i2c1_slave.c	34	271	11%
interrupt_manager.c	0	42	0%

1:enable

3:load results

Debugging

Code Coverage

4:export

Code Coverage

Compiler Advisor

Stack Guidance

Options for xc8-cc (v2.32)

Code Coverage instrumentation

Enable

Stack Guidance

Output - EDBG

Loading script file C:\Users\ml6422\mchp\packs\Microchip\SAME70 DFP\4.5.06\sam70b\...\scripts\dap_cort

Coverage Report - Mobile F700

Source files

Package	Address units covered	Address units not covered	Coverage
Source files	173	411	29%
i2c_slave.c	34	271	
i2c_handler.c	28	52	
interrupt_manager.c	0	42	
main.c	11	0	
mccc	45	0	
memory.c	0	46	
pin_manager.c	55	0	

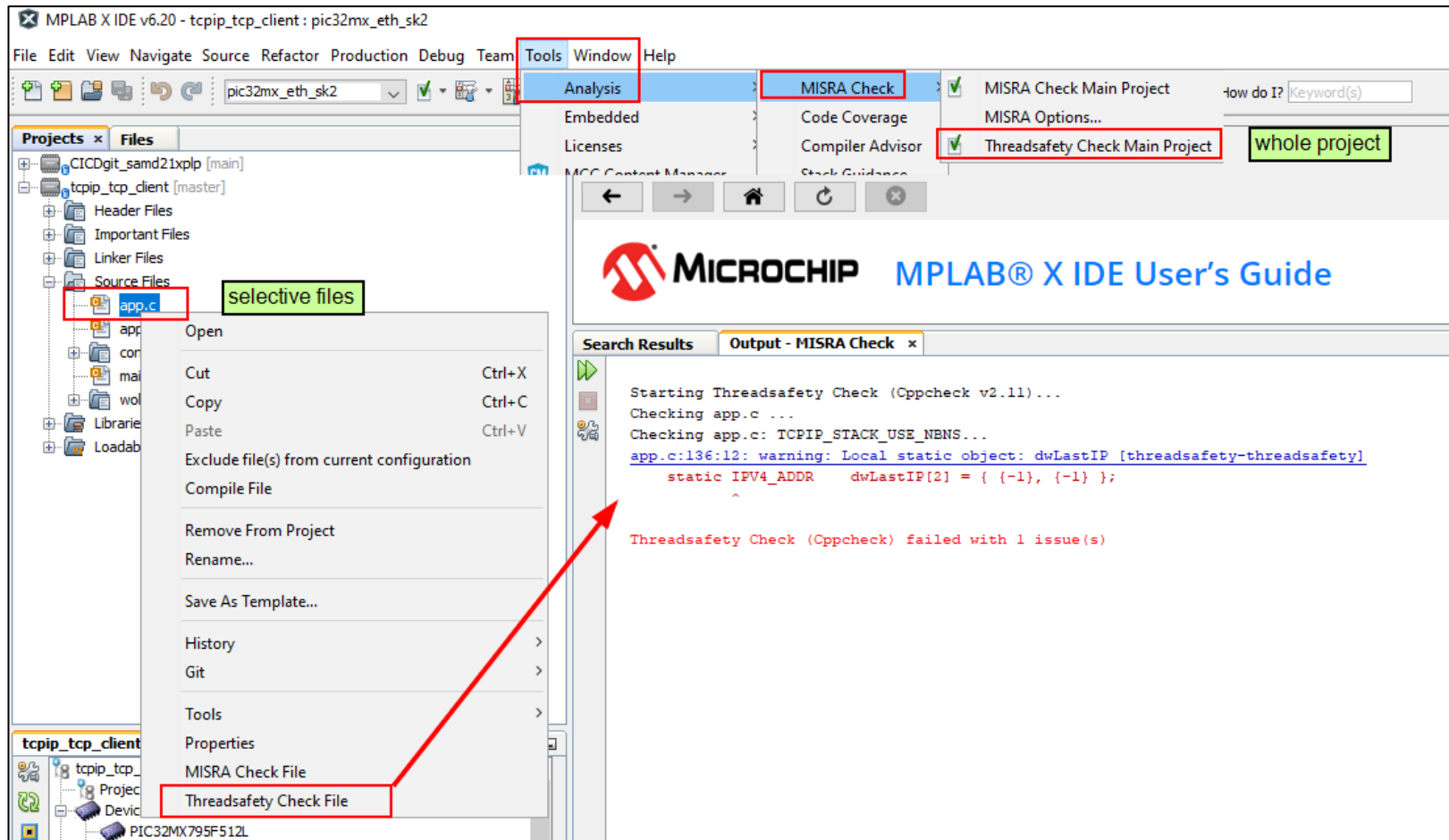
All Source files

Package	Address units covered	Address units not covered	Coverage
pin_manager.c (100%)	55	0	100%
i2c_handler.c (35%)	28	52	35%
mccc (100%)	45	0	100%
main.c (100%)	11	0	100%
memory.c (0%)	0	46	0%
i2c_slave.c (11%)	34	271	11%
interrupt_manager.c (0%)	0	42	0%

Functions in this source file(s)

Function	Coverage
DATAEE_ReadByte	0%
DATAEE_WriteByte	0%
I2C_EEEmu_AddressHandler	0%
I2C_EEEmu_ReadWriteHandler	0%
I2C_EEEmu_WriteWriteHandler	0%
I2C_Init_EEPROM_Emu	100%
I2C_Initialize	100%
I2C_Iter	0%

MPLAB®X-analysis: Threadsafety check



Besides these MPLAB®X analysis-capabilities, maybe you need more external tools/check:

- For open-sources SW with tools like **blackDuck®**
- Or do some static- and/or dynamic analysis with tools like **Coverity®**

Testing capabilities of MPLAB®X

MPLAB®X-testing: Overview

- Before diving into details, some thoughts about a Testing -> strategy and Goal-of-testing?
 - start with regular check if whole project compiles+links but maybe more, right?
 - FW-revA must run on HW-revB or FW-revC must run on HW-revB & HW-revC ...
 - Build regression: start with one test and add more, to create regression test suite
 - Automatically re-run regression-test suite to check new features don't break previous
 - pick up a result-log after maybe 50+h to find 48 tests passed and only 2 failed
- Needs for a regression test suite:
 - unique testID & testEndCriteria eg: buttonPress, certain-char-on-UART1
 - pass/fail criteria, eg: regA=0->test=pass, regA=1-5 == some error occurred
 - timeout -> test[x] wouldn't stall whole regression on hangUp
 - Summary: listOfTests, testId, testEndCriteria, testPassFailCriteria, testTimeout
- Why important to think about?
 - tracking&keeping project milestones is easier and delays will show up early
 - Problems are small and therefore very likely easy to find&fix

RegressionRun(1)

<u>Test#</u>	<u>status</u>
Test[1]	pass
Test[2]	pass
Test[3]	fail
Test[4]	pass
...	
Test[50]	fail

MPLAB®X-testing: unitTesting

- unitTesting
 - Not a MPLAB®X capability but documented CICD-wizard guide
 - What is unitTesting?
 - Different frameworks exist, one is **Unity** or **assert.h**
- Very brief **Unity** example on the right
 - (1) `simple main()`
 - #-Unity defines functions
 - (2) `run_unit_tests()`
 - (3) `test_function*()`
 - (4) `setUp(), tearDown()`
- CICD-wizard userguide has more details on Unity-framework, like how to get + install
- Next slide shows two small examples

3

```
#define F_CPU 1000000UL

#include "mcc_generated_files/mcc.h"
#include <util/delay.h>

#include "unity.h"

void test_function1() {
    // Simple demo of working test
    TEST_ASSERT_TRUE(1);
}

void test_function2() {
    // Simple demo of failing test
    TEST_ASSERT_FALSE(1);
}
```

1

```
int run_unit_tests(void)
{
    UnityBegin("main.c");
    RUN_TEST(test_function1);
    RUN_TEST(test_function2);
    UnityEnd();
    return 0;
}
```

2

```
int main(void)
{
    // Initialize drivers from MCC
    SYSTEM_Initialize();
    _delay_ms(1000);

    run_unit_tests();
}
```

4

```
void setUp (void) {}
void tearDown (void) {}
```

MPLAB®X-testing: unitTesting

- unitTesting example for simple adder() with **Unity** framework

```
#include "unity.c"
#include "calc.h"

void test_add()
{
    int output = add(10, 20);
    TEST_ASSERT_EQUAL(30, output);
}

int main(void)
{
    UNITY_BEGIN();

    RUN_TEST(test_add);

    UNITY_END();

    return 0;
}
```

C-library *assert.h*

```
#include <assert.h>

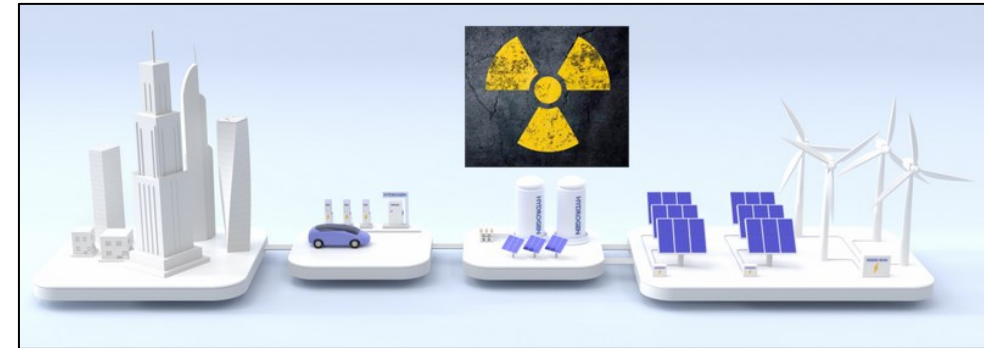
// In my_sum.c
int my_sum(int a, int b) {
    return a + b;
}

// In test_my_sum.c
int main(int argc, char *argv[]) {
    assert(2 == my_sum(1, 1));
    assert(-2 == my_sum(-1, -1));
    assert(0 == my_sum(0, 0));
    // ...
    return(0);
}
```

MPLAB®X-testing: Simulation

MPLAB®X Simulator for each MCU, but why Simulation anyway?

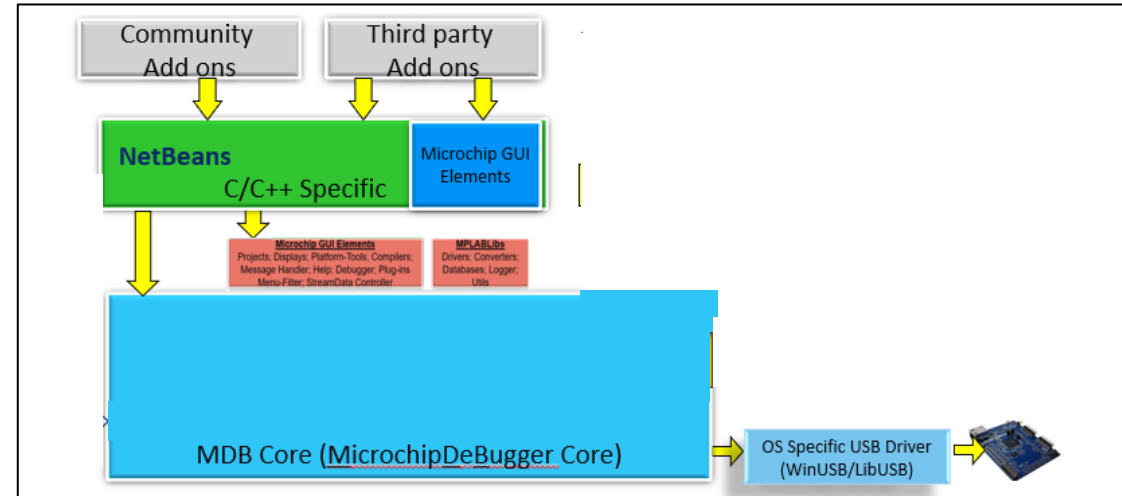
- Dangerous situations - ok, two worst-case situations, but ...
 - FW to monitor expensive Motor
 - > want to ruin N expensive motors before FW correct?
 - Atomic plant
 - > want to blowUp to find out FW not yet 100% ok?
- MPLAB®X Simulator 21014DEV3/Masters2017 (-> ask ESE)
 - Explains how to create **artificial peripherals**, like
 - To create a sine-wave is just one line of Java-code and feed into ADC-result-reg
 - smartUARTPeripheral: read file -> feed into UART-RX buffer -> regression
- A Simulation
 - Can run in DockerContainer and no real HW needed -> so potentially multiple parallel runs
 - With artificial peripherals MPLAB-simulator provide endless choices, only limited by time/effort spent
- Overall, several good reasons to look into simulation



MPLAB®X-testing: HW-in-the-loop testing

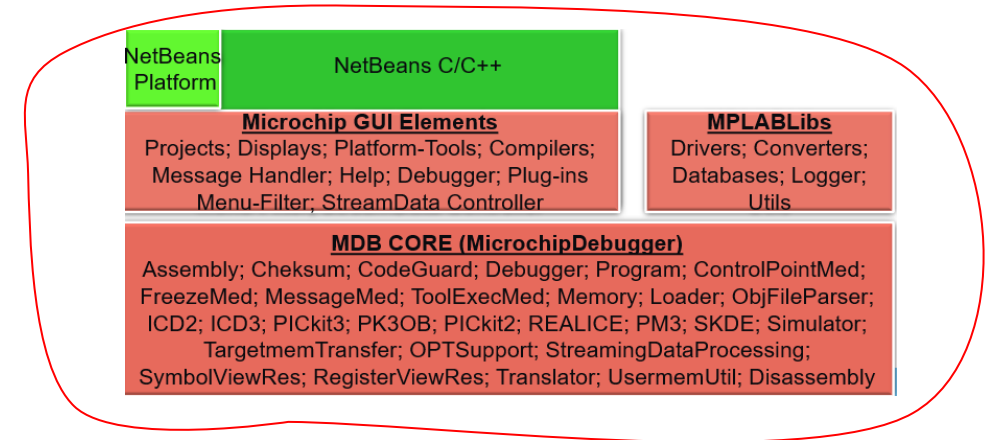
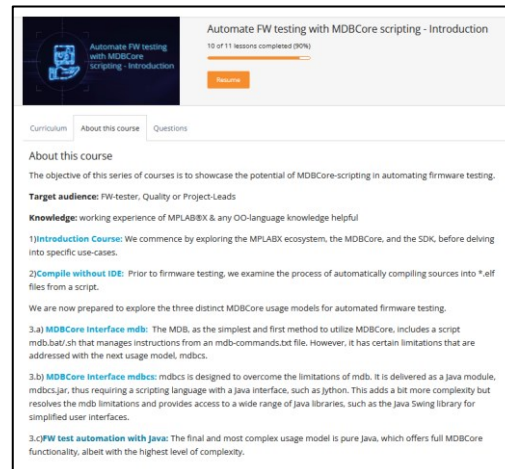
MPLAB®X = Netbeans + multi JavaLibs (~ 1000 Java modules), one is *MDBCore*, but why interesting?

- Can automate every task interactively done in IDE
-> easy to create regression
- compile w/ IDE -> prjMakefileGenerator.bat ∈ (MDBCore)
- Automation for CICD with three *MDBCore* interface:
 1. DOS/shell -> *mdb*
 2. Scripting/*Groovy* -> *mdbc*
 3. *Java*-IF -> access to full MDBCore



- Brief intro as MU-class available online:

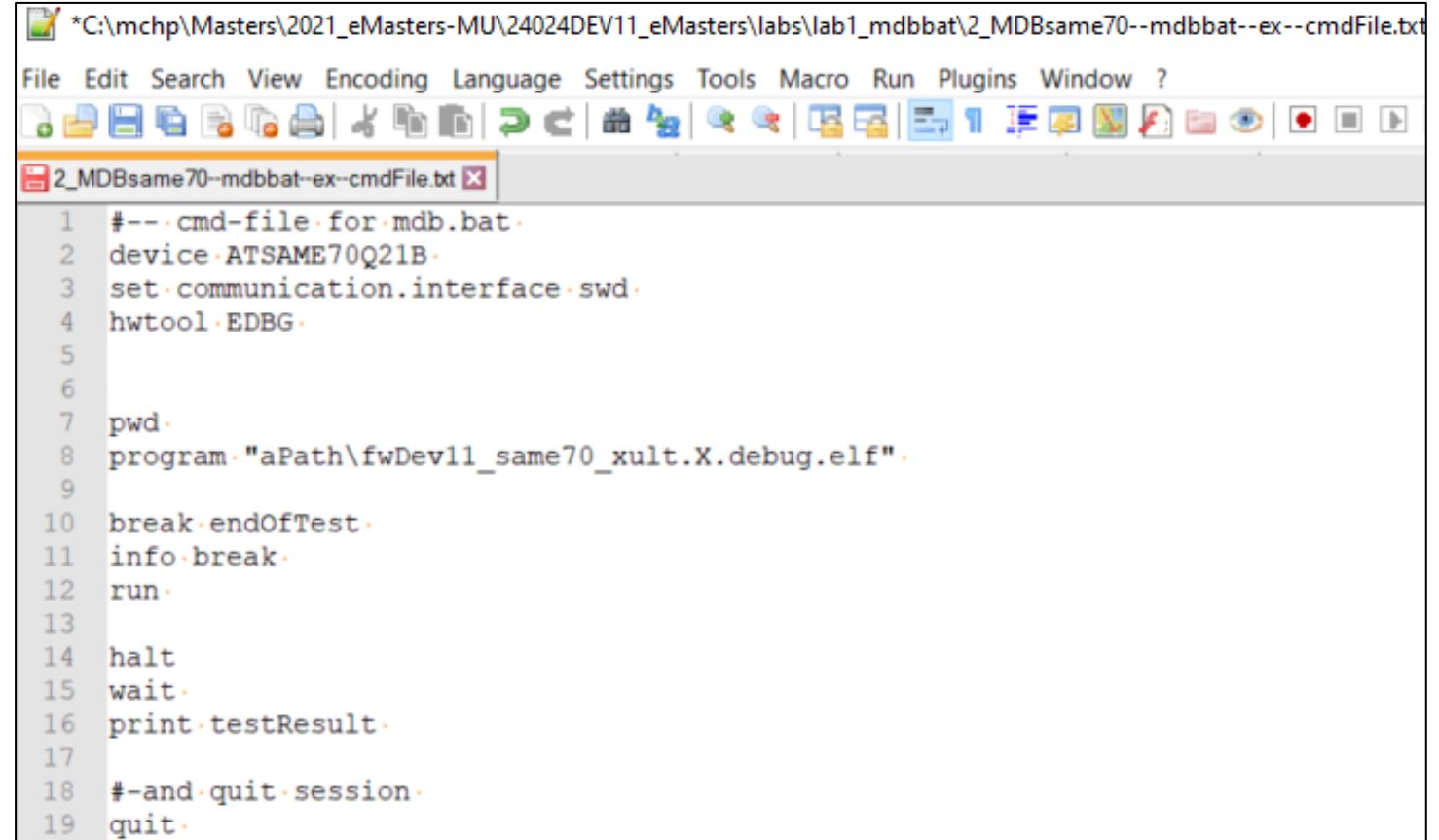
1. [mu/1: mdbcore-introduction](#)
2. [mu/2: compile w/ IDE](#)
3. [mu/3: mdb](#)
4. [mu/4: mdcbc](#)
5. [mu/5: Java](#)



MPLAB®X-testing: mdb

- MDBCore interface1 *mdb*
- Included in MPLAB®X installation, no extra installation
- Simple script mdb.bat/.sh
- Develop mdb-cmd sequence interactively at *mdb* prompt
- Store sequence in *mdb*-command file = text file with *mdb*-commands
- Run with

```
( ) > mdb.bat
      <path>\mdb_command.txt
```
- Comment-lines start with '#' and empty lines allowed
- Dedicated logfile -> | tee in mdb.bat



```
*C:\mchp\Masters\2021_eMasters-MU\24024DEV11_eMasters\labs\lab1_mdbbat\2_MDBsame70--mdbbat--ex--cmdFile.txt
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
2_MDBsame70--mdbbat--ex--cmdFile.txt
1  #-- .cmd-file .for .mdb.bat .
2  device .ATSAME70Q21B .
3  set .communication.interface .swd .
4  hwttool .EDBG .
5
6
7  pwd .
8  program ."aPath\fwDev11_same70_xult.X.debug.elf" .
9
10 break .endOfTest .
11 info .break .
12 run .
13
14 halt
15 wait .
16 print .testResult .
17
18 #-and .quit .session .
19 quit .
```

Updated

- ```
lab2bSol-groovy_debugger.groovy - GroovyConsole
File Edit View History Script Help

1 //- A simple groovy script that connects to a Debugger
2 package org.masters.DEV0.lab2b
3
4 import com.microchip.mdbcs.Debugger
5
6 class testHarness {
7 Debugger debugger = null
8
9 //- this is the static member of a groovy class
10 //- it will be called first when this script is run
11 static void main(args) {
12 new testHarness().performTest()
13 }
14
15 //- perform the test
16 def performTest() {
17 try {
18 println "Creating debugger"
19 //-debugger = new Debugger("PIC32MX795F512L", "Real ICE", true)
20 debugger = new Debugger("PIC32MX795F512L", "ICD3", true)
21 println "Connecting to debugger"
22 debugger.connect()
23 //- The Data should also represent an ASCII art image so print it
24 println "ASCII version of the data"
25 index = 0
26 char ch
27 for(row in 0..7) {
28 for(column in 0..15) {
29 ch = (char) buffer[index]
30 if (Character.isISOControl(ch)) {
31 print "."
32 } else {
33 print Character.toString(ch)
34 }
35 }
36 println
37 index++
38 }
39 } catch (Exception e) {
40 println "Error: " + e.getMessage()
41 }
42 }
43 }
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
9
```



# MPLAB®X analysis&testing: summary

- MPLAB®X includes several analysis-tools: Misra, Thread-Safety, CodeCoverage
- Talked about FW-testing strategy and needs (testList, testId, testEndCriteria, testPassFailCriteria, testTimeout)
- Looked into MPLAB®X testing capabilities:
  - unitTesting (not MPLAB®X specific!)
  - Simulation and why reasonable
  - HW-in-the-loop testing with MDBCore (-> MU-class)
    - mdb: easy, no extra installation, but limits
    - mdbcs: some setup required, solves mdb-limits, endless options in Java-world
    - Java: full access but most complex
- All these test- and analysis-capabilities can (or should) be used in a CI/CD-Jenkins pipeline
- Besides MPLAB-testing&-analysis there are 3<sup>rd</sup> party tools for eg Static/Dynamic Testing like eg: [LDRA®](#)
- Next let's look at a CI/CD-system in more details



## *Everyone still following?*

### *Seen so far:*

- 1. Intro, what is CI/CD, structure, ...*
- 2. MPLAB-analysis & -testing capabilities*

### *Next*

- Used FW, how to build (part1) & run + expected results (part2)*
- CI/CD-pillar1: SCM/git*

# Agenda

- Introduction to CICD
- MPLAB®X ecosystem testing features and analysis capabilities
- **Demos (part1): used FW and expected result**
- Three pillars of a CICD system:
  - SourceControlManagement=SCM/git
  - Automation/Jenkins: Jenkins-Master -> -Agent -> Pipeline -> Demo (bash-in-Pipe)
  - RuntimeEnv/PC-Docker
- Demos (part2): simple-pipeline, Jenkins-Trigger, errors+solutions
- MPLAB®X CICD wizard
- Demos (part3): HW-in-the-loop testing, DockerContainer
- Summary and Outlook

# Demos in git repository

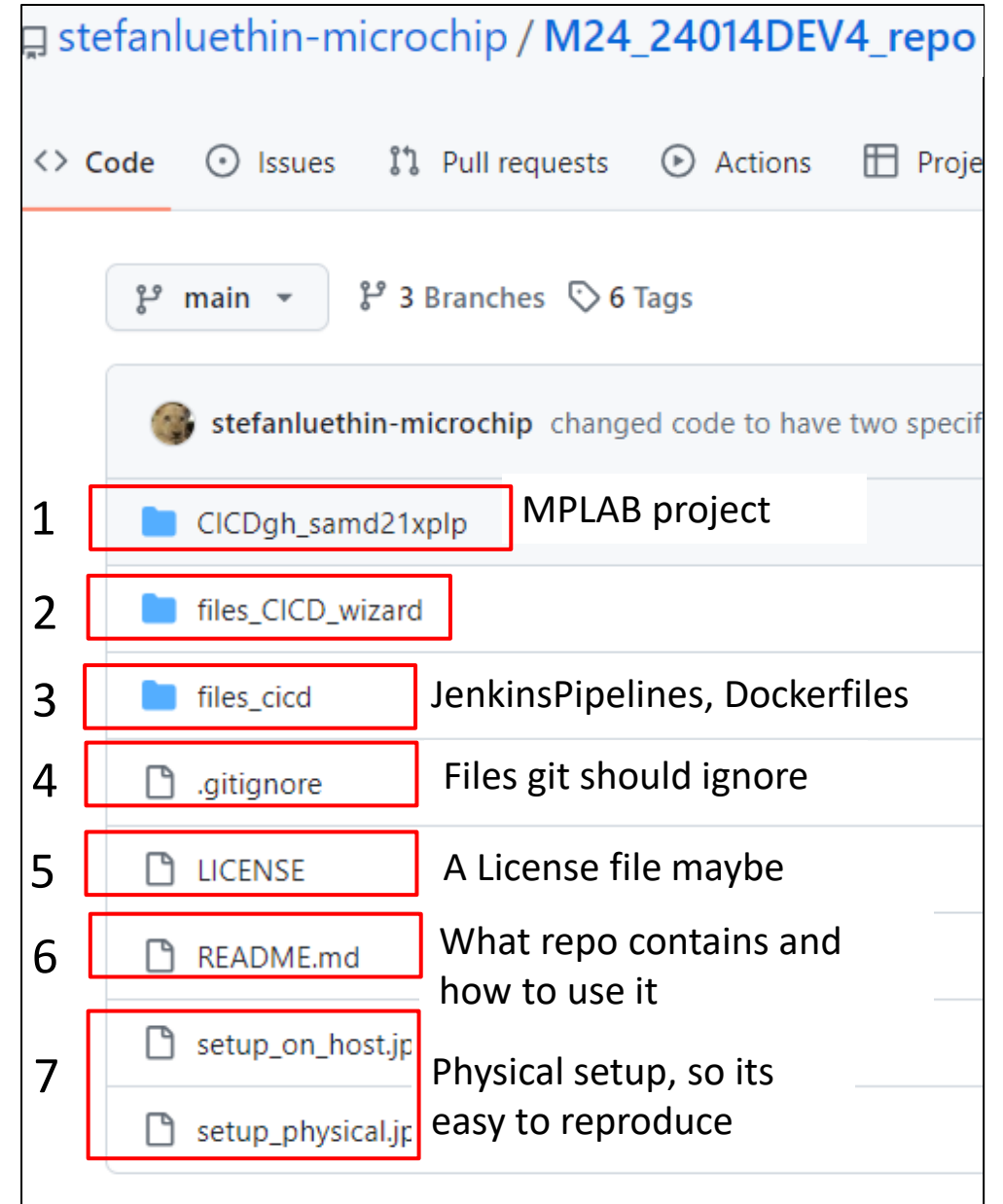
All labs/files are on the following github-repo:

[www.github.com/stefanluethin-microchip/M24\\_24014DEV4\\_repo](https://www.github.com/stefanluethin-microchip/M24_24014DEV4_repo)

<path>/M24\_24014DEV4\_repo/

- 1 |   |- CICDgh\_samd21xplp/
  - 1 |     |-firmware/       = MPLAB-project
  - 1 |     |-mdb/            = MDBCore-scripting
- 2 |   |- files\_cicd\_wizard = Output of MPLAB®X CICD-wizard
- 3 |   |- files\_cicd        = JenkinsPipelines, Dockerfiles of this class
- 4 |   |- .gitignore        = git ignore file
- 5 |   |- LICENSE           = a license file
- 6 |   |- README.md         = Readme for repo
- 7 |   |- setup\_\*.jpg       = pictures of setup

*A Demo/LabManual with all the details to setup+runs all, is available on request  
-> send me a mail [stefan.Luethin@microchip.com](mailto:stefan.Luethin@microchip.com)*



# Demo0: used FW

```
54 int main (void)
55 {
56 /* Initialize all modules */
57 SYS_Initialize (NULL);
58 printf("\n\r");
59 printf("\n\r START CICDtesting ...");
60
61 //--SL: inits
62 myCnt=0; //--SL: central counter
63 finalLedCnt=0;
64 sysTickTmrExFlag=false; //--SL: SysTickWrap-event flag
65 //--SL: SW0 defined as GPIO-In but still need to call this fct to enable In
66 SW0_InputEnable();
67 char2prntIdx=0; //-- default char-to-print = 'b'
68
69 SYSTICK_TimerStart();
70 SYSTICK_TimerCallbackSet(&sysTickTimeout_handler, (uintptr_t) NULL);
71
72 while (true)
73 {
74 /* Maintain state machines of all polled MPLAB Harmony modules. */
75 if(sysTickTmrExFlag)
76 {
77 sysTickTmrExFlag=false;
78 LED0_Toggle();
79 finalLedCnt++;
80
81 myCnt++;
82 if(cEchoCnt/2>=myCnt)
83 {
84 printf("a");
85 } else if ((cEchoCnt/2<myCnt) & (cEchoCnt>=myCnt))
86 {
87 //printf("b");
88 printf("%c",charArray[char2prntIdx]);
89 if (cEchoCnt==myCnt)
90 myCnt=0;
91 } else
92 printf("ERROR1");
93 }
94
95 if(SW0_Get() == PRESSED)
96 {
97 testEnd();
98 }
99 }
100
101 /* Execution should not come here during normal operation */
102
103 return (EXIT_FAILURE);
104 }
```

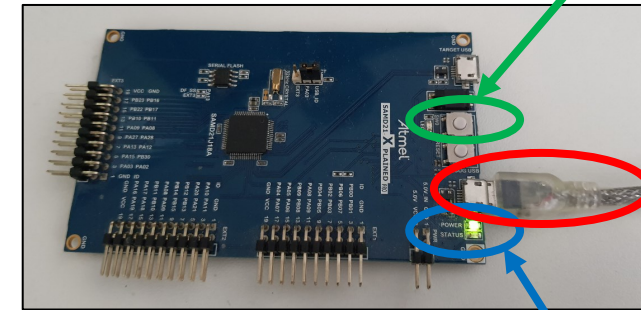
```
37 volatile uint32_t myCnt;
38 volatile uint32_t finalLedCnt;
39 #define cEchoCnt 10
40 #define PRESSED 0
41 #define RELEASED 1
42 volatile bool sysTickTmrExFlag;
43
44 char charArray[3] = {'b','c','d'};
45 volatile uint32_t char2prntIdx; //-- index for 'charArray'
46 void testEnd(void);
47
48
49 void sysTickTimeout_handler(uintptr_t context)
50 {
51 sysTickTmrExFlag=true;
52 }
```

```
COM8:115200bps - Tera Term VT
File Edit Setup Control Window Help

START CICDtesting ...aaaaabbbbbaaaaabbcccaaaaaacccccaaaaa
aaaccccccaaaaaaccccccaaaaaaccccccaaaaaac
testEnd with finalLedCnt=116
```

```
106 void testEnd(void)
107 {
108 printf("\n\r testEnd with finalLedCnt=%d\n\r ",(int)finalLedCnt);
109 printf("\n\r");
110 while(1){}
111 }
112
```

Used HW: [SAMD21 XPRO](#)



## Three FW functionalities:

1. LED blinks
2. echo a char on uart-console
3. SW0 press to end

# Demo0: run mdb-demo

Demo now *(or skip4time)*

## To run demo from Windows-PC:

1. Open DOS-shell and cd into new directory - eg: c:\test\

2. Get git-repo with

```
(win)> git clone https://github.com/stefanluethin-microchip/M24_24014DEV4_repo
```

3. Setup env:

```
(M24_24014DEV4_repo)> CICDgh_samd21xplp\mdb\CICDgh_samd21xplp_test1a_Setup_env.bat
```

4. Compile:

```
(M24_24014DEV4_repo)> CICDgh_samd21xplp\mdb\CICDgh_samd21xplp_test2a_comp.bat
```

<project-path>\firmware\

| -<projectName>.X

| | -Makefile

**4.1** | | -mcc\_prj.mc3

| | -nbproject\

| | | -project.xml

| | | -configuration.xml

| -src\ -> your sources

**4.2**

create/update Makefiles

->**prjMakefilesGenerator** (MDBCore)

**4.3**

compile using **make**

(MPLAB®X GnuTools)

**4.4**

result in

<prj>.X\dist\...\\*.elf

5. Run-on-HW: ***mdb mdb\_cmds.txt*** (MDBCore)

```
(M24_24014DEV4_repo)> CICDgh_samd21xplp\mdb\ CICDgh_samd21xplp_test3a_mdbbat-run.bat
```

# Demo0: summary

- Demo0 contains two parts and is run from a script

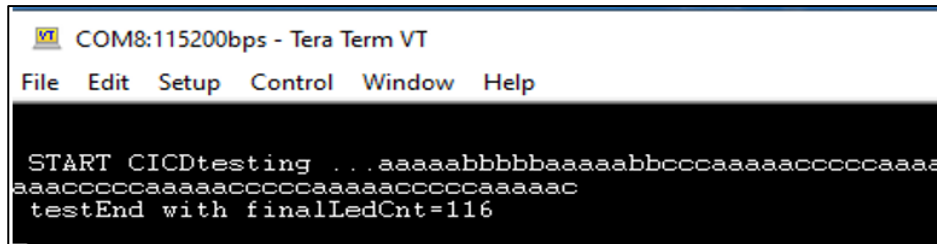
Part1 = compile: preparation -> create Makefiles -> compile

```
()> C:\CICDgh_samd21xplp_test1a_Setup_env.bat && C:\CICDgh_samd21xplp_test2a_comp.bat
```

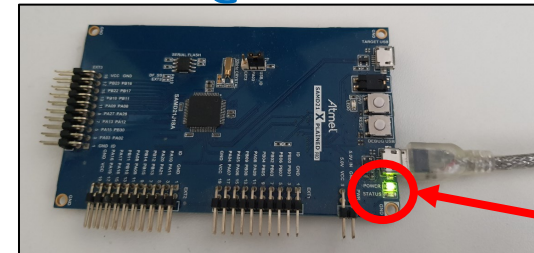
Part2 = Run-on-HW: connect-2-debugger -> download \*.elf -> run

```
()> ...\\CICDgh_samd21xplp_test3a_mdccat-run.bat
```

- Demos to come:
  - Demo1 will re-do Demo0/part1=compile but from **Jenkins-pipeline**
  - Demo2 will just show typical errors&solutions
  - Demo3 will re-do Demo0/part1&part2=compile & HW-run but from **Jenkins-pipeline**  
*printf()/vCOM/console -> msg on UART*



*LEDblink @ 1Hz*



- Demo4 will re-do Demo0/part1=compile but from **Jenkins-pipeline inside a DockerContainer**

# Agenda

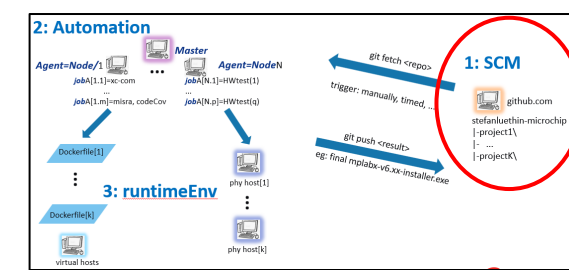
- Introduction to CICD
- MPLAB®X ecosystem testing features and analysis capabilities
- Demos (part1): used FW and expected result
- **Three pillars of a CICD system:**
  - **SourceControlManagement=SCM/git**
  - Automation/Jenkins: Jenkins-Master -> -Agent -> Pipeline -> Demo (bash-in-Pipe)
  - RuntimeEnv/PC-Docker
- Demos (part2): simple-pipeline, Jenkins-Trigger, errors+olutions
- MPLAB®X CICD wizard
- Demos (part3): HW-in-the-loop testing, DockerContainer
- Summary and Outlook



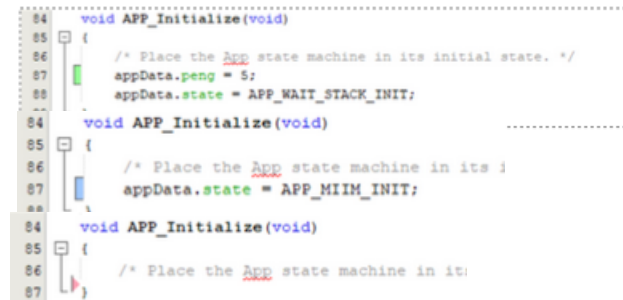
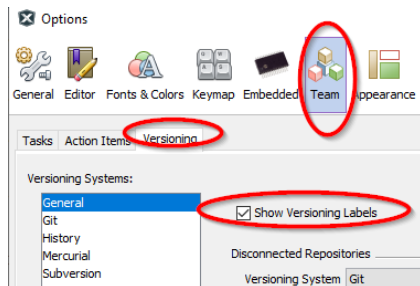
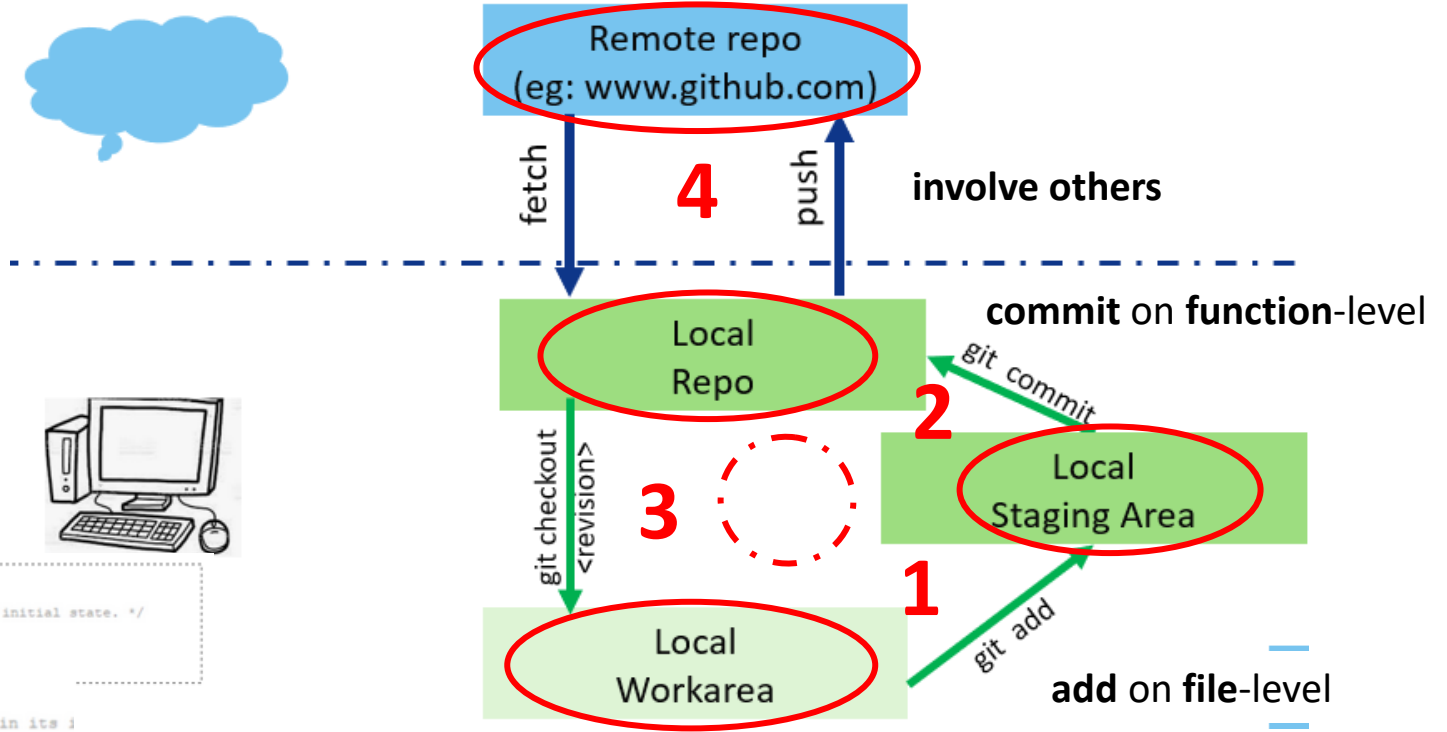
# CICD-p1/SCM: SCM concept

FW revisions with SourceControlManagement/SCM

*just simple intro, teaching  
git beyond scope of class*



- Different tools **git**, **subversion**,... available -> hold various FW-revisions
- SCMs have 3 major components: localWK->localRepo->remoteRepo (+stagingArea in git)
- **git** workflow
- Common to all: tags, branches (=what is FW-revision-A?), ...
- git **tags** 'communication ok'
- git **branches** 'uartRX polled' vs 'uartRX ISR-driven'
- MPLAB®X color marks for git-states



## *Everyone still following?*

### *Seen so far:*

- 1. Intro, what is CI/CD, structure, ...*
- 2. MPLAB-analysis & -testing capabilities*
- 3. Used FW, how to build (part1) & run + expected results (part2)*
- 4. CI/CD-pillar1: SCM/git*

### *Next*

- CI/CD-pillar2: Jenkins*

# Agenda

- Introduction to CICD
- MPLAB®X ecosystem testing features and analysis capabilities
- Demos (part1): used FW and expected result
- **Three pillars of a CICD system:**
  - SourceControlManagement=SCM/git
  - **Automation/Jenkins: Jenkins-Master -> -Agent -> Pipeline -> Demo (bash-in-Pipe)**
  - RuntimeEnv/PC-Docker
- Demos (part2): simple-pipeline, Jenkins-Trigger, errors+solutions
- MPLAB®X CICD wizard
- Demos (part3): HW-in-the-loop testing, DockerContainer
- Summary and Outlook

# CICD-p2/Jenkins: setup (overview)

A Jenkins system needs three parts:

## 1. Jenkins *Master*

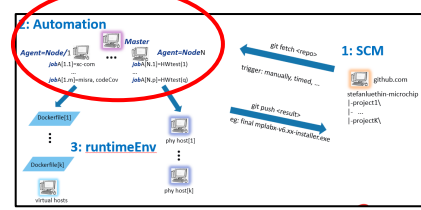
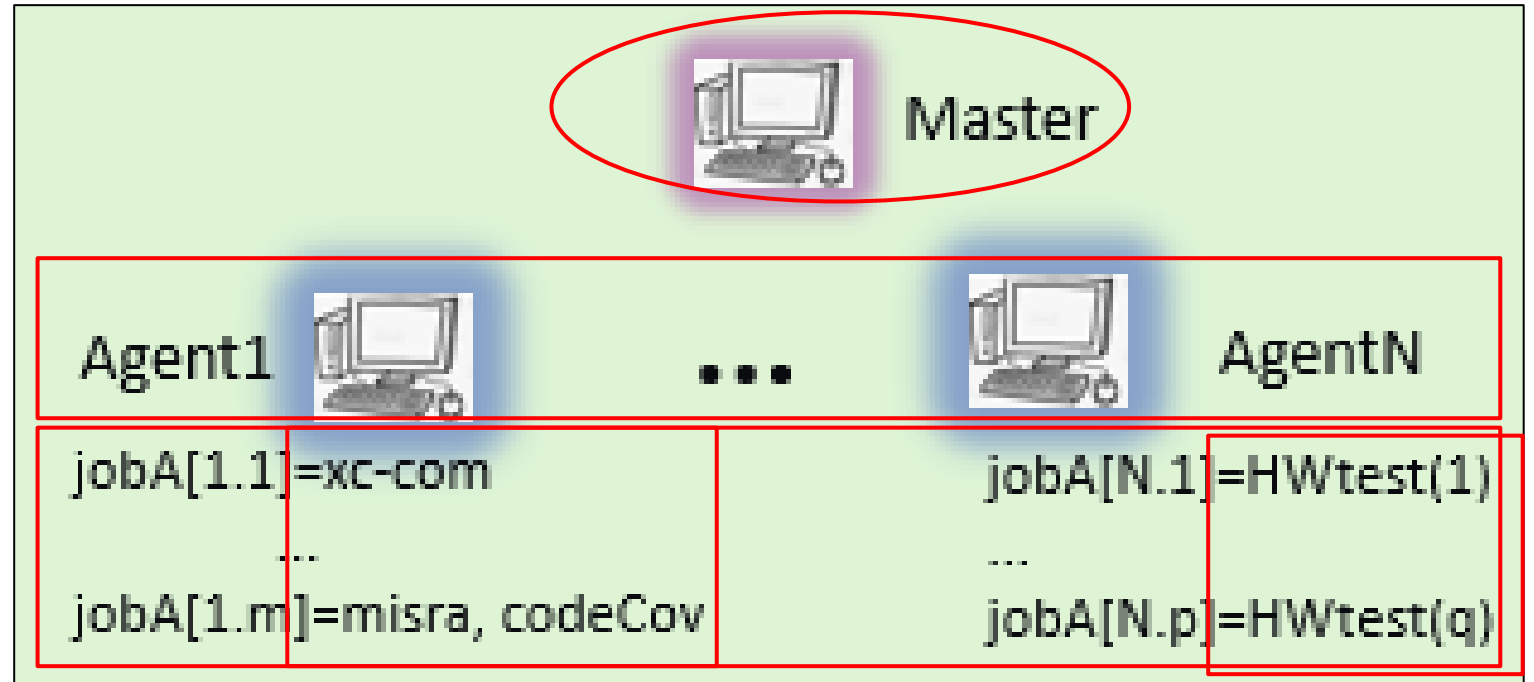
- webIF to setup&configure complete Jenkins-setup

## 2. Jenkins *Node/Agent*

- work horse running the *Jenkins-Jobs*
- Each Jenkins-Jobs is defined by a *Jenkins-pipeline*

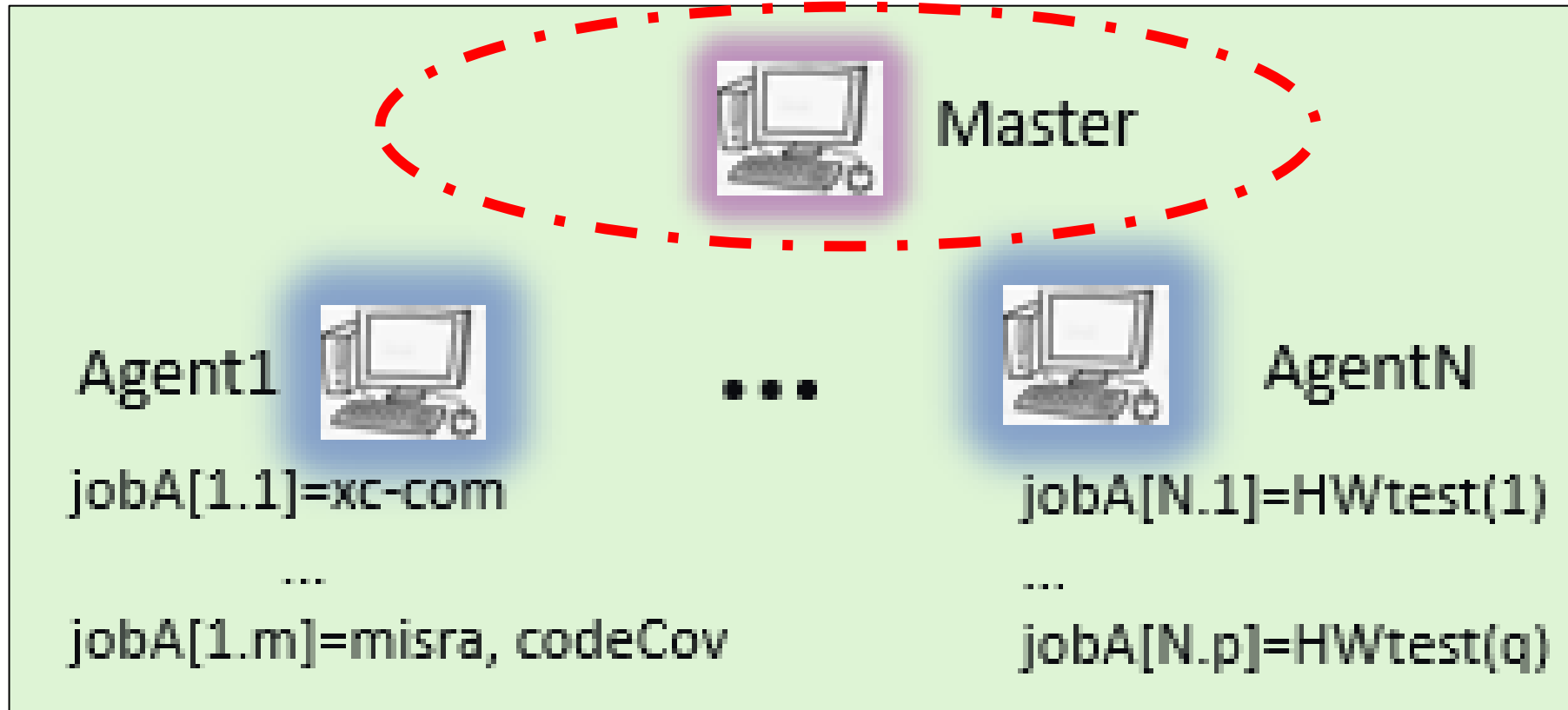
## 3. Jenkins *Pipeline*

- contains *link to git-repository* as well as *event-triggers* when to run a certain *task-sequence* (eg: compile+analysis OR compile+HWrun ...) and *agent-2-use* -> all this is defined in a *Jenkins-pipeline*
- The task-sequence itself is described in a *Jenkins-pipeline-script*
- A Jenkins-Pipeline-script has *Stages* (Build->Analysis->Test...) and each Stage has *Steps* (steps of build-stage: prep -> createMakefiles -> compile -> cleanup)



# CICD-p2/Jenkins: Jenkins *Master*

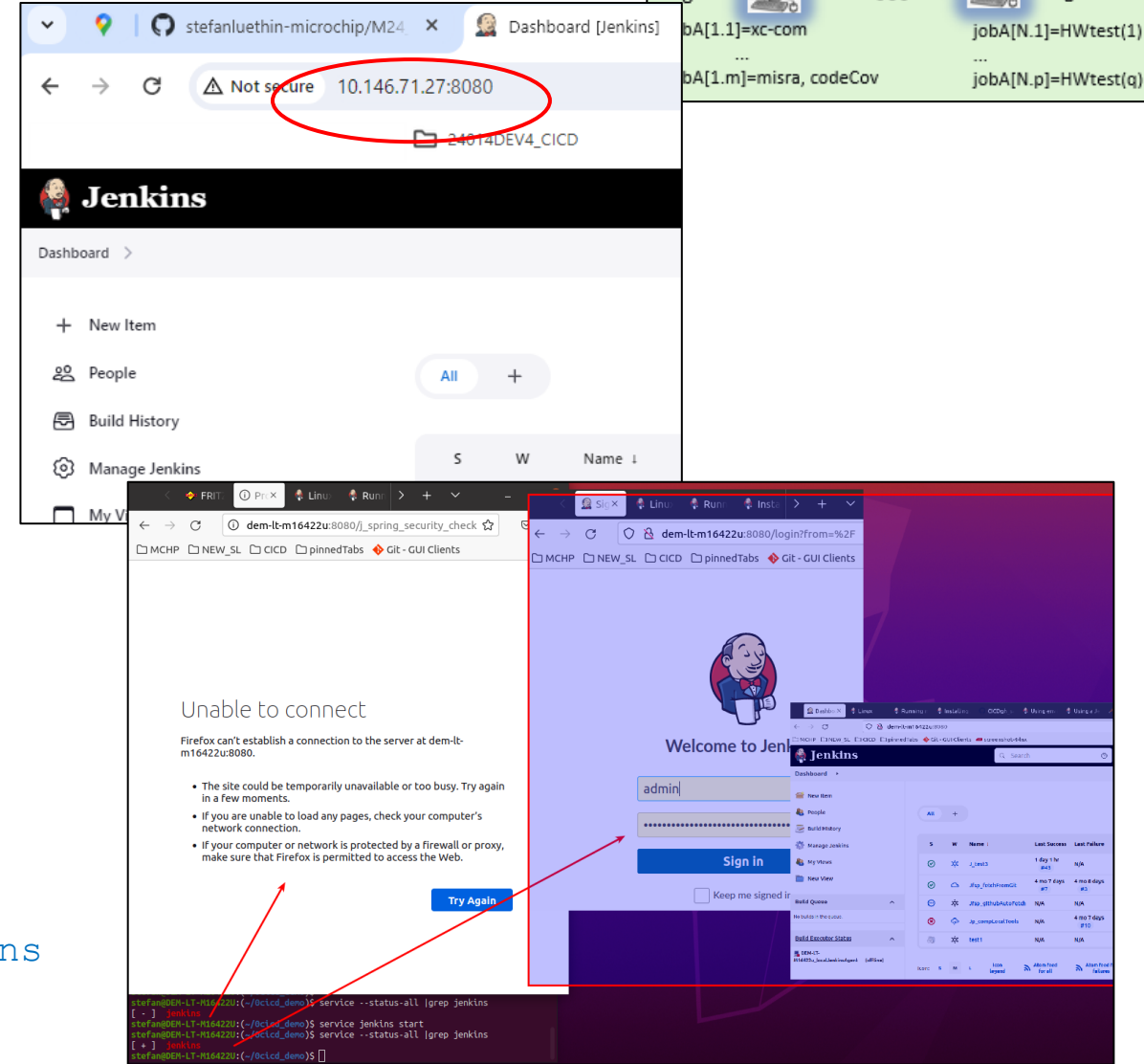
Jenkins *Master* (on Linux)



# CICD-p2/Jenkins: Jenkins *Master* (1)

## Jenkins *Master* (on Linux)

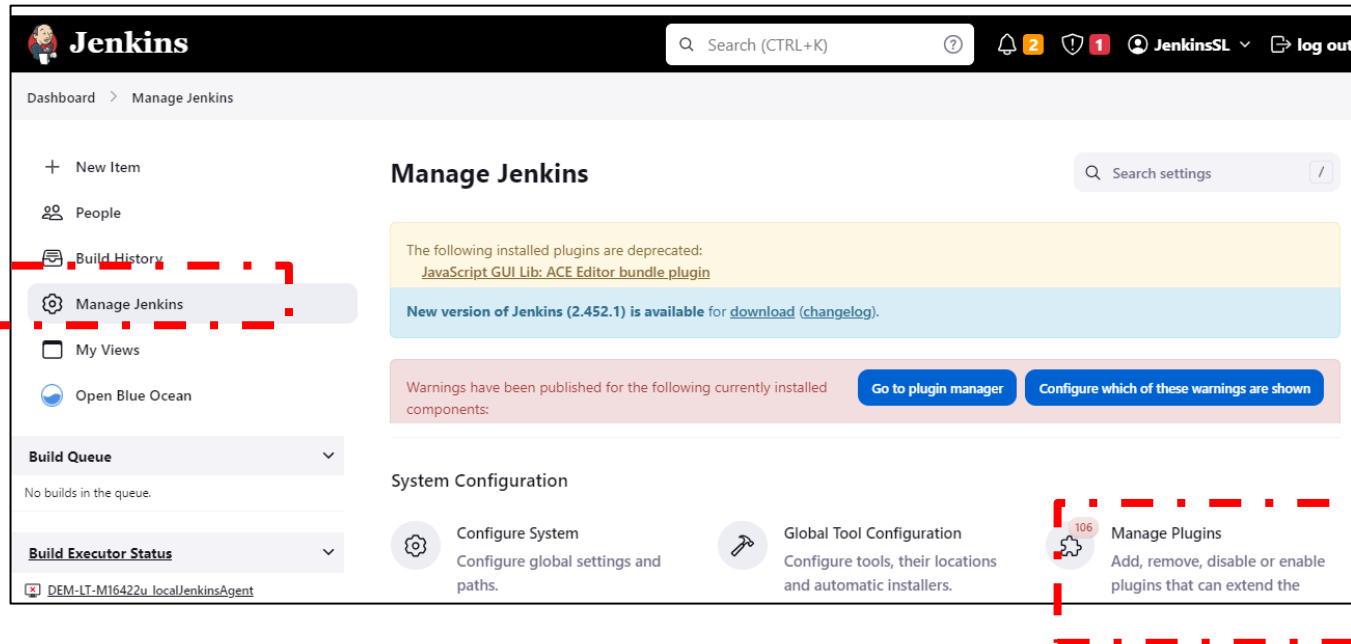
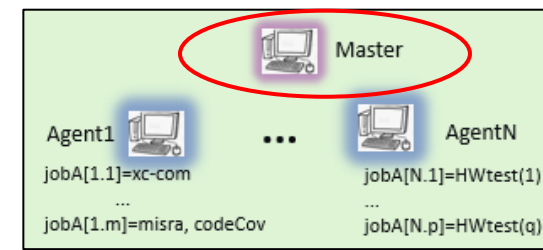
1. Task of a Jenkins-Master is:
  - configure Jenkins-system
  - main interface is J-Masters dashboard
  - Create Nodes in J-Masters dashboard
  - Create and configure pipelines
2. Install: `(lnx)$ ...&& apt-get install Jenkins`  
(-> find exact syntax for your Lnx on the web!)
3. Jenkins-*Master*/service start/stop  
`(~/0cicd_demo)$ service jenkins start|stop`
4. Connect to J-Master -> open URL  
URL: `<ip-address>:8080`
5. Check if Jenkins-*Master* is up with:  
`(~/0cicd_demo)$ service --status-all | grep jenkins`  
`[ + ] jenkins`
  - **service** running == `[+]`
  - **service** down == `[-]` and can't connect to web-page



# CICD-p2/Jenkins: Jenkins *Master* (2)

## Jenkins *Master* (continued)

- Installation automatically creates Linux runlevel-script to start service, so your Jenkins-Master (=service) should be running after reboot
- Add helpful addOns, like BlueOcean plugin (next slide)

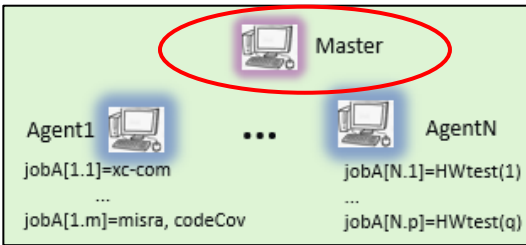




# CICD-p2/Jenkins: Jenkins *Master* (3)

## Jenkins *Master* (continued)

- Various addOns to ease Jenkins-usage - eg: *BlueOcean* plugin to visualize your runs



Dashboard > m24\_24014dev4\_pipeline >

Status

Changes

Build Now

Configure

Delete Pipeline

Open Blue Ocean

Rename

Jenkins

PipelinesAdministration

Logout

m24\_24014dev4\_pipeline

ActivityBranchesPull Requests

Run

| STATUS | RUN | COMMIT | MESSAGE         |
|--------|-----|--------|-----------------|
| ✓      | 55  | —      | Started by user |
| ✓      | 54  | —      | Started by user |
| ✓      | 53  | —      | Started by user |
| ✓      | 51  | —      | corrected com   |
| ✗      | 52  | —      | Start d by user |

m24\_24014dev4\_pipeline < 52 >

PipelineChangesTestsArtifacts

Branch: —

8s

No changes

Commit: —

5 days ago

Started by user JenkinsSL

StartBuildEnd

Restart Build

Build - 5s

✓ > Check out from version control

✓ > Check out from version control

✓ > Checks if running on a Unix-like node

✓ > files\_cicd/Dockerfile\_4\_d2\_docker\_24014dev4 — Read file from workspace

✗ > docker build -t 6fb9fcf4a802d371bd73eebf331e50873923c17d -f "files\_cicd/Dockerfile\_4\_d2\_docker\_24014dev4" — Shell Script

1 + docker build -t 6fb9fcf4a802d371bd73eebf331e50873923c17d -f files\_cicd/Dockerfile\_4\_d2\_docker\_24014dev4 .

2 ERROR: permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http://%2Fvar%2Frun%2Fdocker.sock/\_ping": dial u

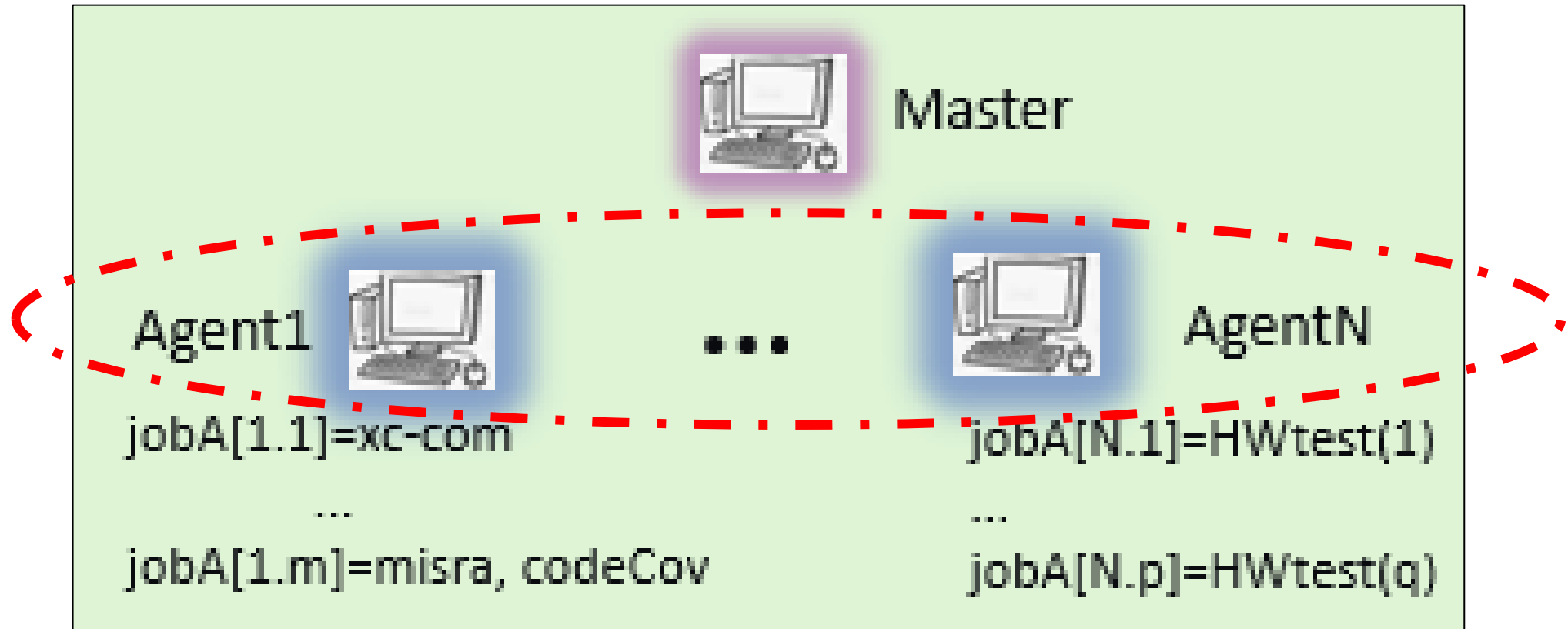
3 /var/run/docker.sock: connect: permission denied

script returned exit code 1

✓ > ###-SL: ===== — Print Message

# CICD-p2/Jenkins: Jenkins *Node/Agent*

Jenkins *Node/Agent* (on Linux)



# CICD-p2/Jenkins: Jenkins *Node/Agent* (0)

Jenkins *Node/Agent* (on Linux) - overview:

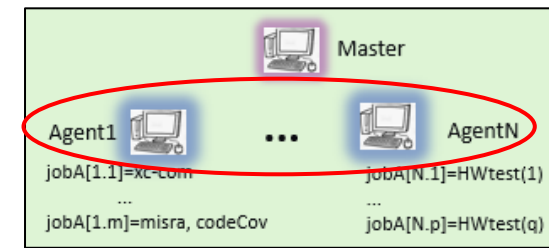
1. Task of a Jenkins-Node/Agent is

2. Will see how to **Create** a J-Node/Agent (in J-Master dashboard)

3. Will **configure** the J-Node/Agent (in J-Master dashboard) and define eg: how many builds to keep (discards) and specify a label used in the J-pipelines

4. Explain how to **start/stop** a J-Node/Agent

5. how to **check** if a certain a J-Node/Agent is up or down (in J-Master dashboard)



2

3

4

5

# CICD-p2/Jenkins: Jenkins *Node/Agent* (1)

Jenkins *Node/Agent* (on Linux)

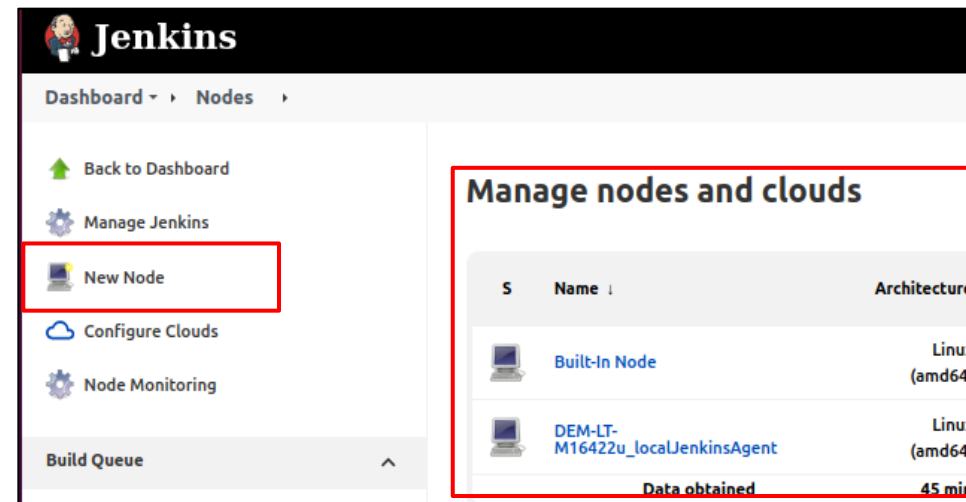
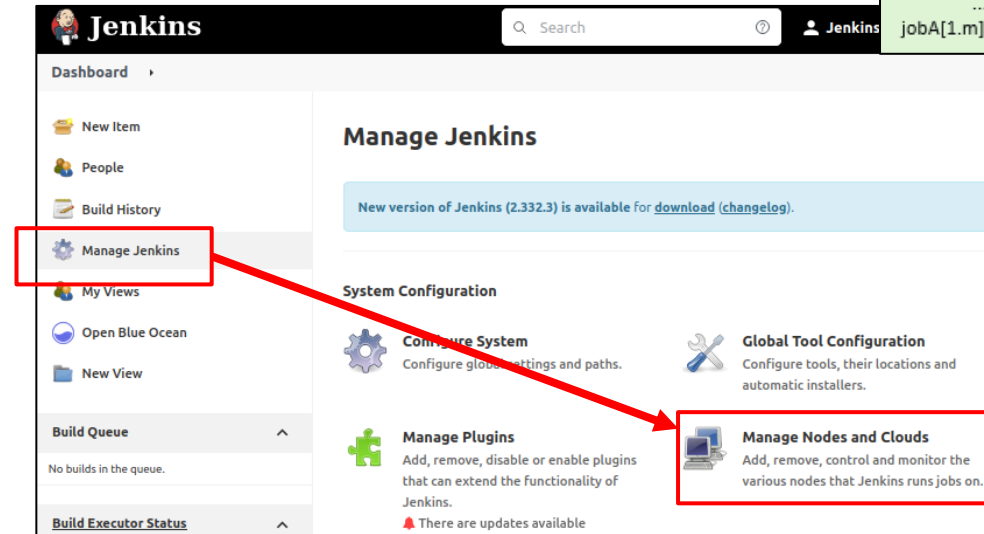
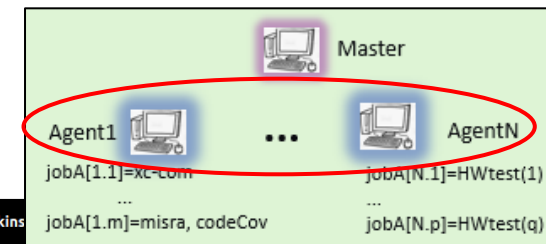
1. **Task of** a Jenkins-Node/Agent is:

- work horse running the jobs
- physicalPC or virtualHost (=docker)
- Node ~= Agent but
- Node = some execution-host for Master  
-> Label used in Pipeline-Script
- Agent = actual process of started script (later!) on the actual host

2. **Create/install**

- Start on J-Master dashboard
- Open 'Manage Jenkins'
- 'Manage Nodes & Clouds'
- Create 'New Node'
- Next configure & start

2. **Create**
3. **Configure**
4. **start/stop**
5. **check up/down**



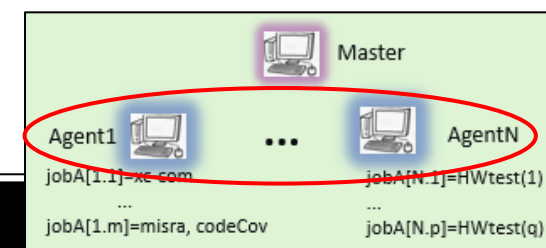
# CICD-p2/Jenkins: Jenkins Node/Agent (2)

Jenkins **Node/Agent** (continued)

## 3. **Configure** node/agent

- Name of node/agent and description
- # **executors** = how many jobs can run in parallel  
eg: host-w-lots-of-RAM ok to run many parallel compiles  
BUT host-w-HW can only run one job
- **Remote-directory**  
->'git clone' will store here  
->look here, in case of errors
- **Labels** used in Pipeline-agent statement to send a job to a specific node/agent eg: HW-jobs should run on node/agent with attached HW

2. **Create**
3. **Configure**
4. **start/stop**
5. **check up/down**



The screenshot shows the Jenkins 'Configure' page for a node named 'DEM-LT-M16422u\_localJenkinsAgent'. The page has a sidebar on the left with options: Status, Delete Agent, Configure (selected), Build History, Load Statistics, Script Console, Log, System Information, Disconnect, and Open Blue Ocean. The main content area has several fields, each highlighted with a red box: 'Name' (DEM-LT-M16422u\_localJenkinsAgent), 'Description' (local jenkins-agent on DEM-LT-M16422u), 'Number of executors' (2), 'Remote root directory' (/var/lib/jenkins/nodes/DEM-LT-M16422u\_localJenkinsAgent), and 'Labels' (DEM-LT-M16422u\_localLnxPC). At the bottom, there is a 'Usage' section. A red arrow points from the 'Number of executors' field to the 'Build Executor Status' section, which shows two idle executors.

# CICD-p2/Jenkins: Jenkins *Node/Agent* (3)

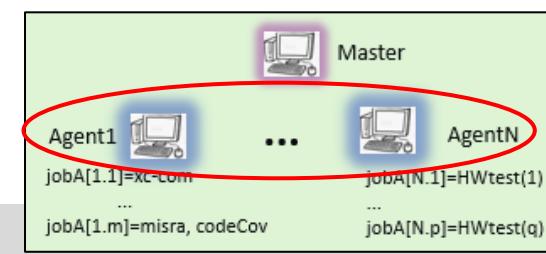
## Jenkins *Node/Agent* (continued)

### 4. **Start** the node/agent

- **node**-configuration (last slide) finished -> get Java-call
- put call into script and start **agent** on **node**
- Java-call creates process and this is the **Agent**
- So **Node** is physicalHost or DockerContainer whereas **Agent**=process of running java-call

The screenshot shows the Jenkins web interface at the URL `dem-lt-m16422u:8080/computer/testNode/`. The page title is "Agent testNode (testNode description)". It provides instructions on how to connect the agent to Jenkins, including a command to run from the agent command line: `java -jar agent.jar -jnlpUrl http://DEM-LT-M16422U:8080/computer/testNode/jenkins-agent.jar -secret-file e292d782bebc6b20eb25751291e4d54a63b2cd370dc9586326bf09872c9229fe -workDir "/var/lib/jenkins/nodes/testNode"`. A red box highlights this command. The "Build Executor Status" section shows the agent is "Idle".

2. **Create**
3. **Configure**
4. **start/stop**
5. **check up/down**



```
#!/usr/bin/bash
#- Jenkinsfile
JENKINS_HOST_N="DEM-LT-M16422U" #hostname of computer running Jenkins
...
###----- MAIN -----###
echo "#####Starting Jenkins-agent ..."
echo ${JENKINS_AGENT_S} > secret-file
...
echo "#####"
java -jar ${JENKINS_AGENT_F} -jnlpUrl ${JENKINS_URL}/jenkins-agent.jnlp \
-secret @secret-file -workDir "${JENKINS_AGENT_WORKDIR_P}"
```

The screenshot shows the Jenkins web interface at the URL `DEM-LT-M16422u_localJenkinsAgent > Configure`. The page title is "Jenkins". It provides a configuration page for the agent "DEM-LT-M16422u\_localJenkinsAgent". The "Name" field is set to "DEM-LT-M16422u\_localJenkinsAgent". The "Description" field is set to "local jenkins-agent on DEM-LT-M16422u". The "Number of executors" is set to "2". The "Remote root directory" is set to `/var/lib/jenkins/nodes/DEM-LT-M16422u_localJenkinsAgent`. The "Labels" field is set to "DEM-LT-M16422u\_localLinuxPC". The "Build Executor Status" section shows the agent is "Idle".

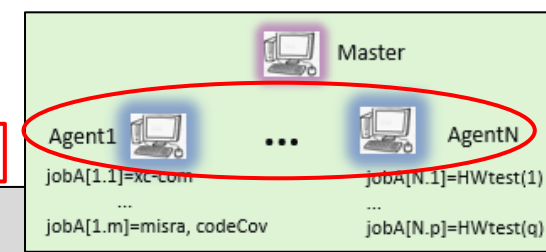
# CICD-p2/Jenkins: Jenkins *Node/Agent* (4)

## Jenkins *Node/Agent* (continued)

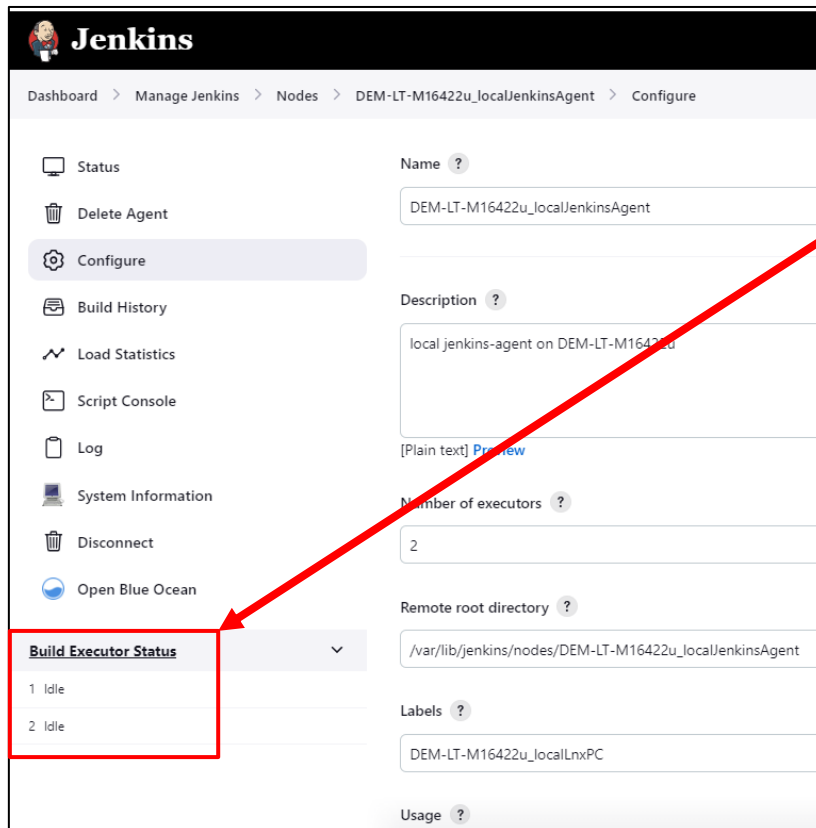
### 5. up/down-*check*

- Starting the script obviously starts the Node/Agent

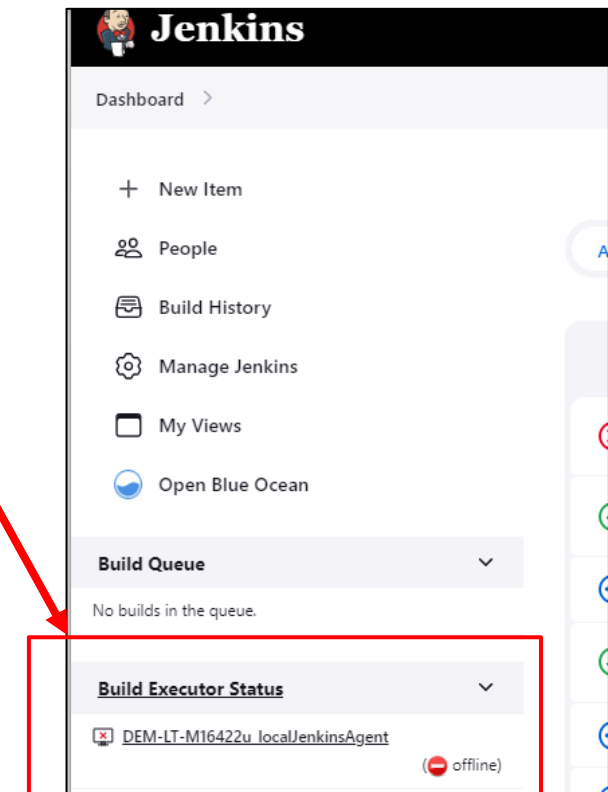
2. **Create**
3. **Configure**
4. **start/stop**
5. **check up/down**



```
#!/usr/bin/bash
#- Jenkinsfile
JENKINS_HOST_N="DEM-LT-M16422U" #-hostname of computer running Jenkins
...
###----- MAIN -----###
echo "#####Starting Jenkins-agent ..."
echo ${JENKINS_AGENT_S} > secret-file
...
echo "#####"
java -jar ${JENKINS_AGENT_F} -jnlpUrl ${JENKINS_URL}/jenkins-agent.jnlp \
 -secret @secret-file -workDir "${JENKINS_AGENT_WORKDIR_P}"
```



- Once started, on J-Masters dashboard see num executors specified idle
- To stop the Node/Agent simply stop the script -> on J-Dashboard you'd see this
- if this is your only Node/Agent, you'd now have no available Node/Agents to run on -> demo2

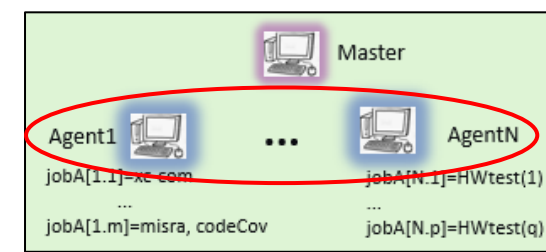




# CICD-p2/Jenkins: Jenkins *Node/Agent* (5: summary)

Jenkins *Node/Agent* (continued) - summary of steps:

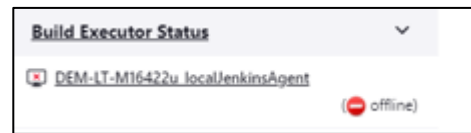
1. First **create** a new node/agent: J-Masters dashboard -> manage nodes -> new node -> manage nodes -> new node
2. Next **configure** (remote-dir, labels, ...) node -> message with java-call to start agent -> put call into script 'myAgent1.sh'
3. Copy script 'myAgent1.sh' onto Node and execute script to **start** agent
4. Once up **check** Agents' (build-)executors on J-Masters dashboard
5. To **stop** Agent, simply stop script



The screenshot shows the Jenkins 'Configure' page for a node named 'DEM-LT-M16422u\_localJenkinsAgent'. The left sidebar contains links: Status, Delete Agent, Configure (selected), Build History, Load Statistics, Script Console, Log, System Information, Disconnect, and Open Blue Ocean. The main content area has the following fields:

- Name**: DEM-LT-M16422u\_localJenkinsAgent
- Description**: local jenkins-agent on DEM-LT-M16422u
- Number of executors**: 2
- Remote root directory**: /var/lib/jenkins/nodes/DEM-LT-M16422u\_localJenkinsAgent
- Labels**: DEM-LT-M16422u\_localLnxPC
- Usage**: ?

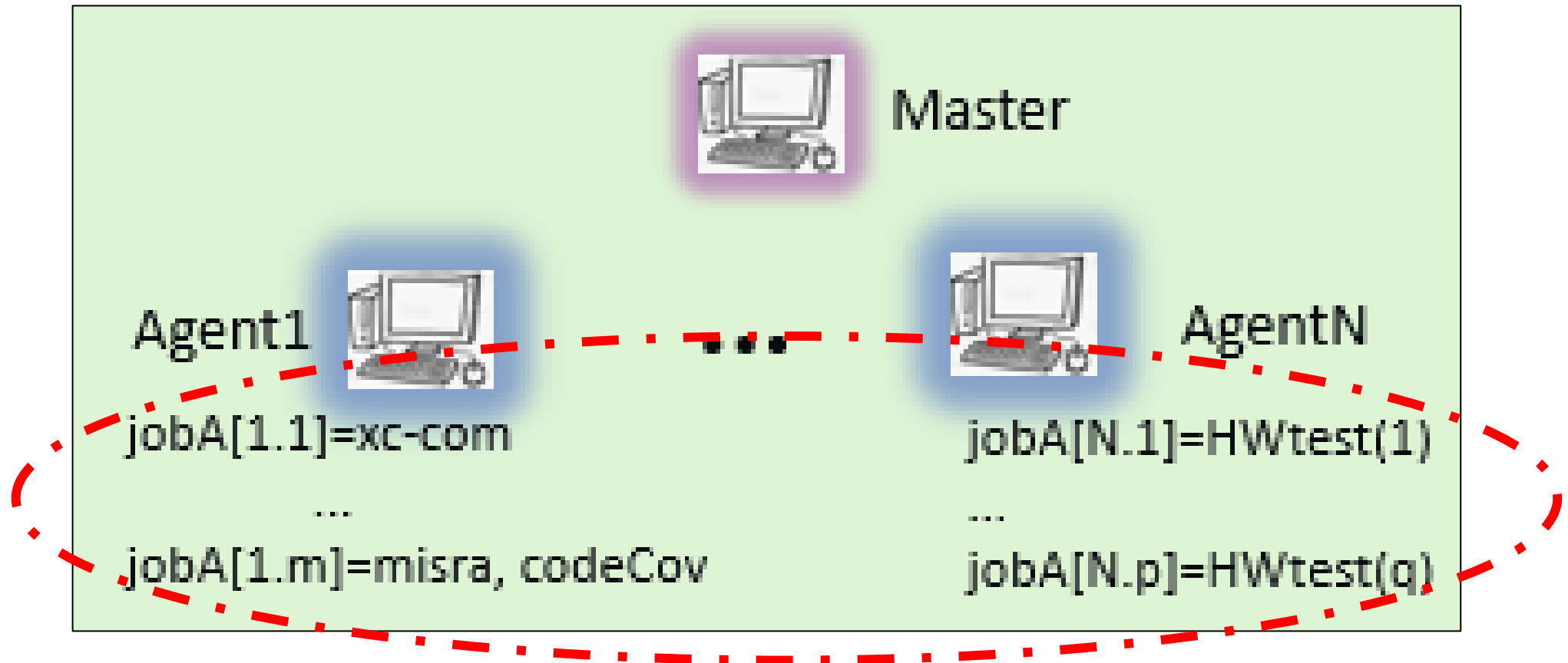
At the bottom, the 'Build Executor Status' section shows two executors, both in an 'Idle' state. A red box highlights this section, and a red arrow points from it to a smaller inset window.



*Short BREAK  
if needed !?*

# CICD-p2/Jenkins: Jenkins-Pipeline

Jenkins **Pipeline**: last part of Jenkins-system is the Pipeline where you finally define the sequence of tasks-to-execute or task-list

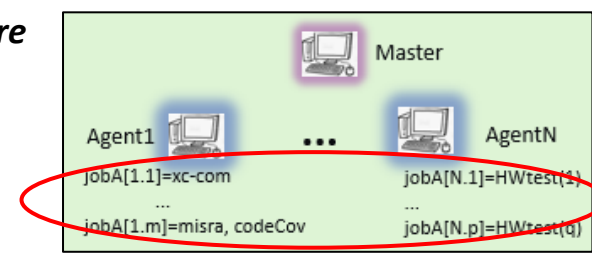


# CICD-p2/Jenkins: J-Pipeline Creation (0)

Jenkins *Pipeline* - overview:

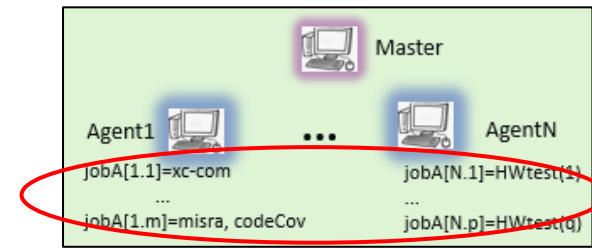
1. First we'll look at the Jenkins *Pipeline creation* with these steps:
  1. **Task** of a Jenkins-/J-Pipeline
  2. **Create&configure** a J-Pipeline
  3. **Start/stop** or run a J-Pipeline
  4. Result **check** - did my J-Pipeline run or fail and why?
  5. **Debugging** a J-Pipeline
2. In the second part we'll look at the Jenkins *Pipeline Script* or content, syntax, structure to understand what happens where

2. **Create & Configure**
3. **start/stop**
4. **Resultcheck**
5. **debug**



# CICD-p2/Jenkins: J-Pipeline Creation (1)

Jenkins *Pipeline* creation



1. Task of a Jenkins-/J-Pipeline is:

- The bread&butter = the sequence-of-tasks-to-run are defined in the Jenkins-**Pipeline script**
- But the Jenkins-**Pipeline-script** is just one part of the overall Pipeline configurations, which includes:
  - Type of Pipeline, like bash-script, J-Pipeline-Script, Multi-Stage, Freestyle, ...
  - How many builds to keep
  - When or how to trigger this pipeline -> **Jenkins-trigger**
  - which repository/URL to use and which branch to work on
  - Finally, the tasks-to-run in the J-Pipeline-**Script** (->this is where you define what should be done)
  - Jenkins-Pipelines might have:

- |                                         |                           |                            |
|-----------------------------------------|---------------------------|----------------------------|
| ▪ Pipe1: fetch + compile + misra        | & use Agent_BigRAM        | & run this 1x/wk on Th@8pm |
| ▪ Pipe2: fetch + compile + codeCoverage | & use Agent_any           | & run 1x/m on Mo@10pm      |
| ▪ Pipe3: fetch + compile + HWrun        | & on Agent HWrev1attached | & run every on Th@8am      |
| ▪ ...                                   |                           |                            |

->defined in J-Pipeline-Script

->defined in J-Pipeline Configuration

# CICD-p2/Jenkins: J-Pipeline Creation (2a)

Jenkins *Pipeline* creation (continued)

## 2. *create&configure* a J-Pipeline

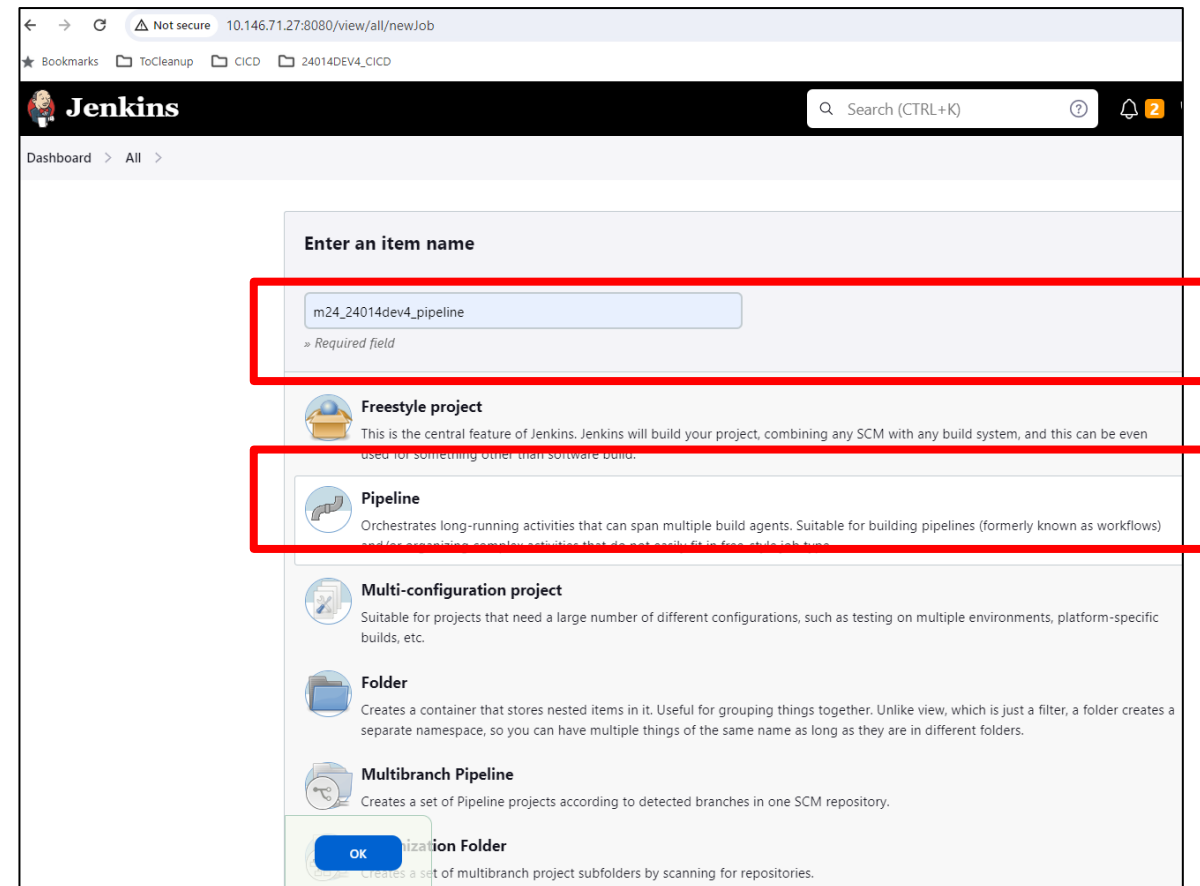
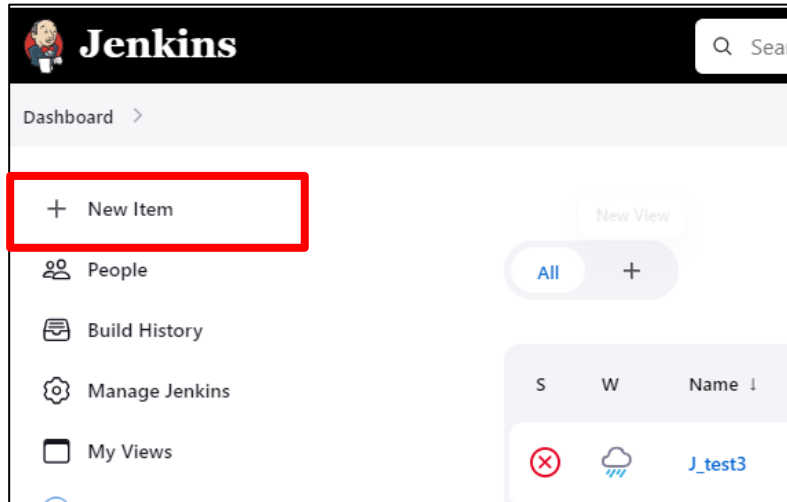
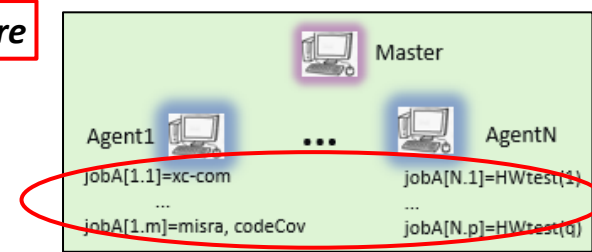
- Start from J-Master dashboard and create newItem
- Provide a *name* and a *type* (in class use 'Pipeline')

2. *Create & Configure*

3. *start/stop*

4. *Resultcheck*

5. *debug*



# CICD-p2/Jenkins: J-Pipeline Creation (2b)

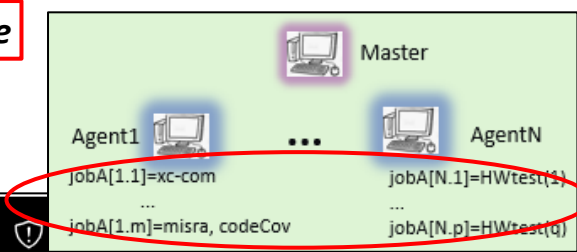
Jenkins *Pipeline* creation (continued)

2. create&configure a simple *Pipeline* (continued)

Depending on pipeline type you get more/less configuration entries, for Type=Pipeline you get:

- **Description**
- **Discard** strategy  
= Num-buids to keep

2. **Create & Configure**
3. **start/stop**
4. **Resultcheck**
5. **debug**



The screenshot shows the Jenkins Configuration page for a Pipeline. The page is titled 'm24\_24014dev4\_pipeline > Configuration'. The 'General' tab is selected. The 'Description' field is highlighted with a red box and contains the text 'Pipeline used for Masters2024 class 24014DEV4 on CICD'. The 'Discard old builds' checkbox is checked. The 'Strategy' dropdown is set to 'Log Rotation'. The 'Days to keep builds' field is empty. The 'Max # of builds to keep' field is set to '3'. The 'Advanced' dropdown is visible at the bottom.



# CICD-p2/Jenkins: J-Pipeline Creation (2c)

Jenkins *Pipeline* creation (continued)

2. create&configure a J-Pipeline (continued)

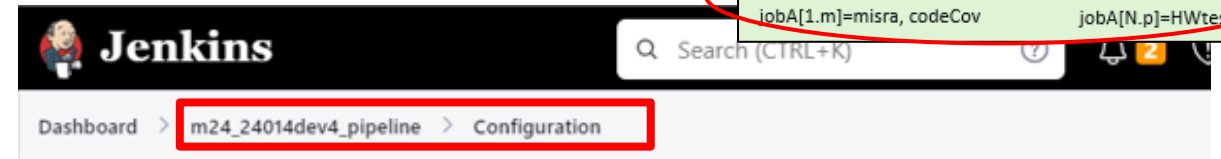
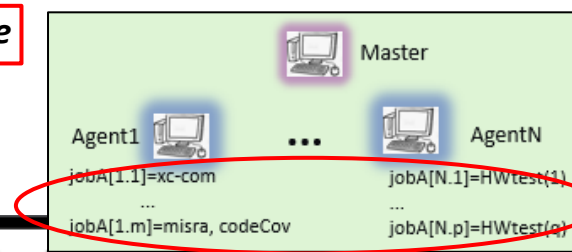
- In **Build Triggers** specify when to build (=start)
  - Manually (->later slide!)
  - Timed, like every Th@8pm
  - webHooks -> commit on branch main

2. **Create & Configure**

3. **start/stop**

4. **Resultcheck**

5. **debug**



## Configure

General

Advanced Project Options

Pipeline

### Build Triggers

☐ Build after other projects are built ?

☐ Build periodically ?

☐ GitHub Branches

☐ GitHub Pull Requests ?

☐ GitHub hook trigger for GITScm polling ?

☐ Poll SCM ?

☐ Quiet period ?

☐ Trigger builds remotely (e.g., from scripts) ?

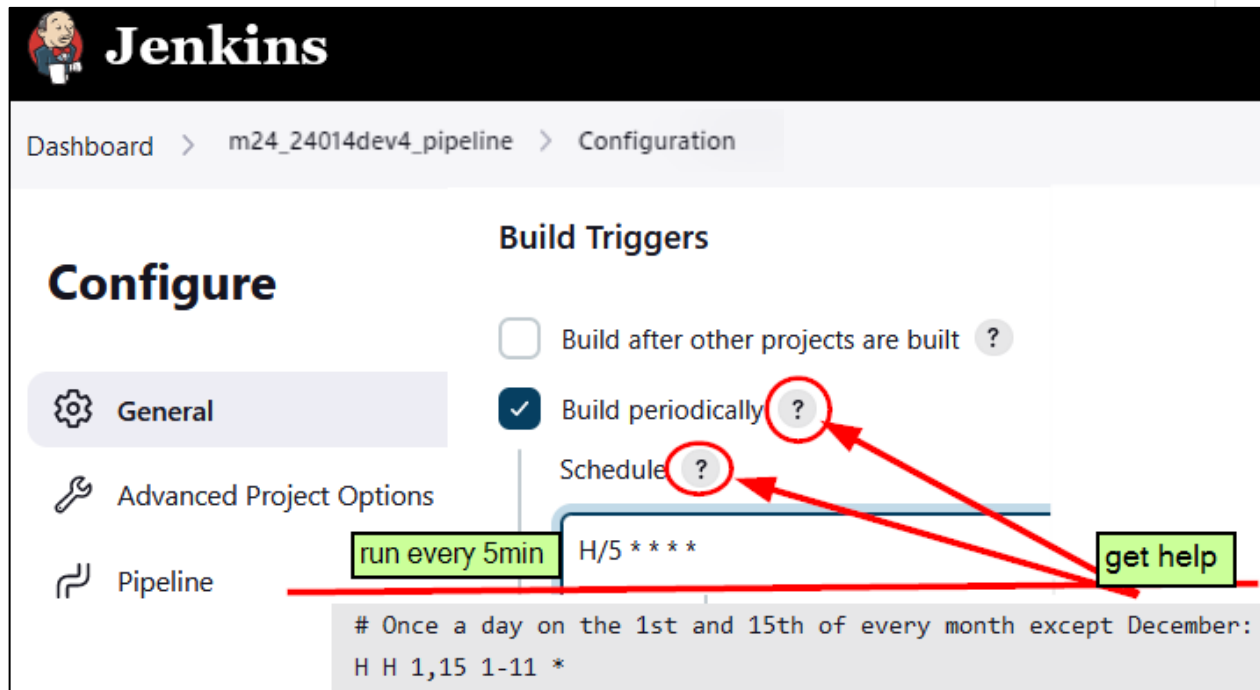
### Advanced Project Options

Advanced ▾

Max # of builds to keep

if not empty, only up to this number of build records are kept

© 2024 Microchip Technology Inc. and its subsidiaries



# CICD-p2/Jenkins: J-Pipeline Creation (2d)

Jenkins *Pipeline* creation (continued)

## 2. create&configure a J-Pipeline (cont)

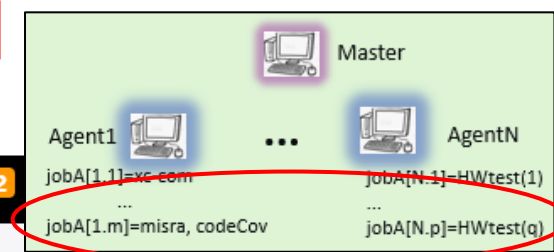
- In *Pipeline* define
  - SCM-to-use: git, subversion, ...
  - URL-2-repo & credentials for private repos
  - git-branch-2-use
  - Pipeline-file to run (or for a start, just a simple bash-scr with echos)

2. **Create & Configure**

3. **start/stop**

4. **Resultcheck**

5. **debug**



The screenshot shows the Jenkins 'Configure' page for a pipeline named 'm24\_24014dev4\_pipeline'. The 'Pipeline' tab is selected in the left sidebar. The main configuration area is divided into several sections, each highlighted with a red box:

- Definition:** 'Pipeline script from SCM' is selected.
- SCM:** 'Git' is selected.
- Repositories:** The 'Repository URL' is set to 'https://github.com/stefanluethin-microchip/M24\_24014DEV4\_repo'. The 'Credentials' dropdown is set to '- none -'.
- Branches to build:** The 'Branch Specifier (blank for 'any')' is set to '\*/main'.
- Script Path:** The path is set to 'files\_cicd/Jenkinsfile\_d2\_24014dev4'.

At the bottom, the 'Lightweight checkout' checkbox is checked. The 'Save' and 'Apply' buttons are visible at the bottom right.

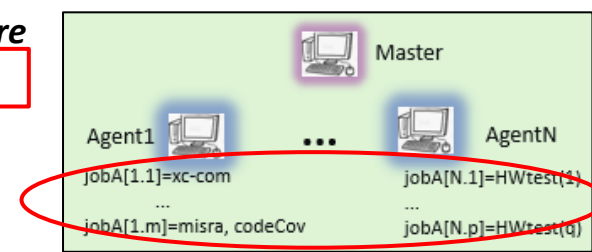
# CICD-p2/Jenkins: J-Pipeline Creation (3)

Jenkins *Pipeline* creation (continued)

## 3. *Start/stop* or run a J-Pipeline

- Touched how to trigger a pipeline *automatically* using Jenkins-Triggers in Pipeline Configuration
- Simplest method is to start it *manually* from the Pipeline-context

2. *Create & Configure*
3. *start/stop*
4. *Resultcheck*
5. *debug*



The screenshot shows the Jenkins web interface for the pipeline 'm24\_24014dev4\_pipeline'. The left sidebar contains a list of actions: 'Status', 'Changes', 'Build Now' (highlighted with a red box), 'Configure', 'Delete Pipeline', 'Open Blue Ocean', 'Rename', and 'Pipeline Syntax'. The main content area displays the pipeline name, a description 'Pipeline used for Masters2024 class 24014DEV4 on CICD', and a list of 'Permalinks' including 'Last build (#55)', 'Last stable build (#55)', 'Last successful build (#55)', 'Last failed build (#52)', 'Last unsuccessful build (#52)', and 'Last completed build (#55)'. At the bottom, the 'Build History' section shows the last two builds, #55 and #54, with their respective timestamps.

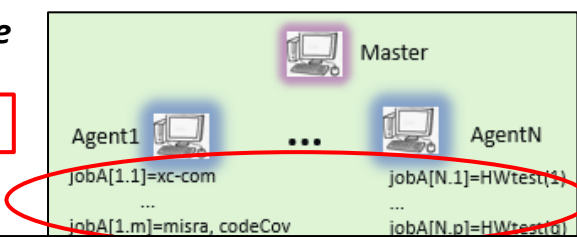
# CICD-p2/Jenkins: J-Pipeline Creation (4)

Jenkins *Pipeline* creation (continued)

## 4. *Result check*

- Either open 'console output' from run-context
- Or use addOn BlueOcean

2. *Create & Configure*
3. *start/stop*
4. *Resultcheck*
5. *debug*



The screenshot shows the Jenkins Pipeline view for the 'm24\_24014dev4\_pipeline'. The top navigation bar includes 'Pipelines', 'Administration', and 'Logout'. The main header displays the pipeline name 'm24\_24014dev4\_pipeline' with a star icon and a gear icon. Below the header, there are tabs for 'Activity', 'Branches', and 'Pull Requests'. A 'Run' button is visible on the left, and a 'Disable' button is on the right. The main content area shows a table of pipeline runs with columns for 'STATUS', 'RUN', 'COMMIT', 'MESSAGE', 'DURATION', and 'COMPLETED'. The table lists runs 55, 54, 53, 51, and 52. Run 52 is marked with a red 'X' in the status column. Below the table, there is a section for 'Build - 5s' showing a progress bar with 'Start', 'Build', and 'End' stages. The 'Build' stage is highlighted with a red 'X'.

The screenshot shows the console output for build #52 of the 'm24\_24014dev4\_pipeline'. The output is displayed in a red box with a white 'X' icon. The text shows the build steps and their results. The first step is 'Check out from version control', which succeeded. The second step is 'Check out from version control', which also succeeded. The third step is 'Checks if running on a Unix-like node', which succeeded. The fourth step is 'files\_cicd/Dockerfile\_4\_d2\_docker\_24014dev4 - Read file from workspace', which succeeded. The fifth step is 'docker build -t 6fb9fcf4a802d371bd73eebf331e50873923c17d -f "files\_cicd/Dockerfile\_4\_d2\_docker\_24014dev4" . - Shell Script', which failed with an error. The error message is: 'ERROR: permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http://%2Fvar%2Frun%2Fdocker.sock/\_ping": dial unix:///var/run/docker.sock: connect: permission denied script returned exit code 1'. The output ends with '###-SL: - Print Message'.

The screenshot shows the Jenkins Pipeline view for the 'm24\_24014dev4\_pipeline'. The top navigation bar includes 'Dashboard', 'm24\_24014dev4\_pipeline', and a search bar. The main header displays the pipeline name 'Pipeline m24\_24014dev4\_pipeline' with a description 'Pipeline used for Masters2024 class 24014DEV4 on CICD'. Below the header, there are tabs for 'Status', 'Changes', 'Build Now', 'Configure', and 'Delete Pipeline'. The 'Open Blue Ocean' button is highlighted with a red box. The 'Build History' section shows a list of builds with a 'trend' dropdown. The build #55 is highlighted with a red circle. The 'Permalinks' section on the right shows a list of links for the build, including 'Last build (#55)', 'Last stable build (#55)', 'Changes', 'Console Output', 'Edit Build Information', 'Delete build #55', 'Git Build Data', 'Open Blue Ocean', 'Restart from Stage', 'Replay', 'Pipeline Steps', and 'Workspaces'. The 'Console Output' link is highlighted with a red box.

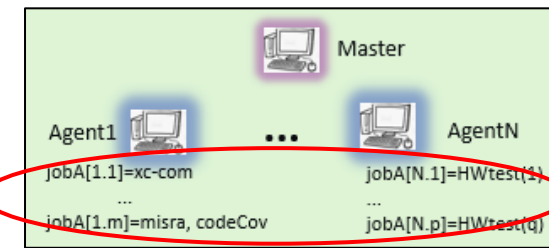
# CICD-p2/Jenkins: J-Pipeline Creation (5)

Jenkins *Pipeline* creation (continued)

## 5. *Debugging*

- On error first check console-output (last slide)
- Demo2 will show some example errors and solutions like
  - Syntax-errors in Pipeline-scripts (eg: wrong comment)
  - Wrong path+name to Dockerfile
  - Problem inside Dockerfile
  - ...
- Jenkins also provides some helpful tools like Script Console

2. *Create & Configure*
3. *start/stop*
4. *Resultcheck*
5. *debug*



The screenshot shows the Jenkins web interface. The left sidebar contains navigation options: Status, Delete Agent, Configure, Build History, Load Statistics, Script Console (highlighted with a red box), Log, System Information, Disconnect, and Open Blue Ocean. The main content area is titled 'Script Console' and contains a text input field for Groovy scripts. Below the input field, there is a section for the script's execution result, showing the output of the script.

```
println System.getenv("PATH")

println "uname -a".execute().text
```

This execution happens in the agent JVM.  
All the classes from all the plugins are visible. jenkins.\*, jenkins.model.\*, hudson.\*, and hudson.model.\*

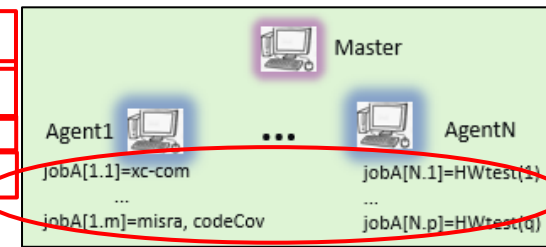
```
1 printf("test print (START)\n")
2 //-this is a comment
3
4 //def folder = new File('./files_cicd')
5 //if (folder.exists()) {
6 def readmefile = new File('README.md')
7 if (! readmefile.exists()) {
8 printf("no\n")
9 } else {
10 printf("yes\n")
11 }
12 printf("END\n")
```

**Result**

```
test print (START)
no
END
```

# CICD-p2/Jenkins: Pipeline Creation (6: sum)

2. **Create & Configure**
3. **start/stop**
4. **Resultcheck**
5. **debug**



Jenkins **Pipeline** creation (continued) - summary on setup+configuration:

1. Task of a J-Pipeline: this is where the work is done and also how (discards, triggers, URL2git,...)
2. First **create** a J-Pipeline from J-Master dashboard via 'new item' and then configure it:
  - General infos like Name and Discard-strategy
  - Triggers = when-to-run pipeline, like every Th@8pm
  - Pipeline-section defines SCM-to-use and URL-2-repo and finally the actual task-list = pipeline-script  
->this class will use one Pipeline and for different demos just use different pipeline-scripts/files
3. **Start/stop** a Pipeline:
  - start manual from J-Master dashboard
  - Start automatically with
    - timedTrigger = timing-event, like every Th@8pm+every-2.week
    - Webhooks = events like 'commit to release branch'
4. **Result-check**: find results
  - in 'console output' via run-context menu
  - Or use plugins like BlueOcean
5. **Debugging**:
  - in Demo2 will discuss some common error+solutions

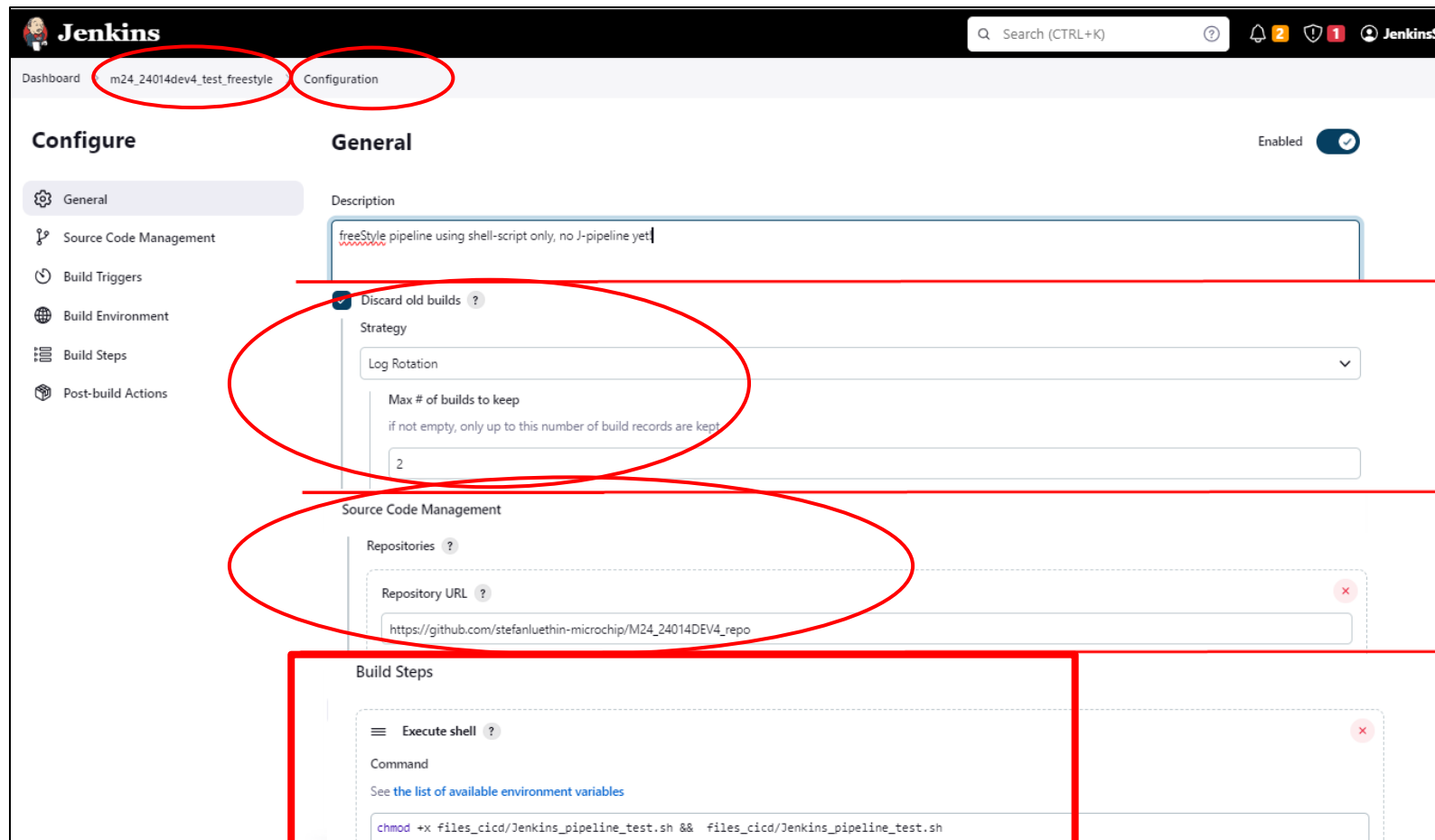
**Last part is the Pipeline-script = bread&butter where all the work is finally done**

# CICD-p2/Jenkins: J-Pipeline simple demo



Jenkins *Pipeline* - a simple start

- By now know how to start J-Master, create a J-Node & start it (->Agent) and we know how to create a J-Pipeline (type->discards->URL-2-repo) and finally where to define the task-sequence =J-Pipeline-Script
- good idea to start with simple Jenkins-Pipeline script=bash-script *instead* of full featured pipeline (next)



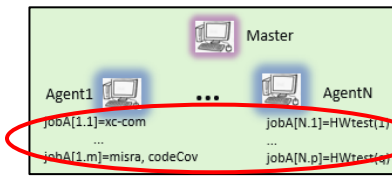
- Pipeline type: **FreeStyle**
- **Configure**
  - Discards: 3x
  - URL-2-repo: M24 git-repo
  - Build step: bash-script (->part of repo!)



# CICD-p2/Jenkins: J-Pipeline simple demo

Running this simple pipeline with a simple bash-script results in

Run demo now



Jenkins Dashboard: test1\_freestyle > #5 > Console Output

Console Output (Green checkmark icon):

```
Started by user JenkinsSL
Running as SYSTEM
Building remotely on DEM-LT-M16422u_localJenkinsAgent (DEM-LT-M16422u_localLinuxPC) in workspace /var/lib/jenkins/nodes/DEM-LT-M16422u_localJenkinsAgent/workspace/test1_freestyle
The recommended git tool is: git
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/nodes/DEM-LT-M16422u_localJenkinsAgent/workspace/test1_freestyle/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/stefanluethin-microchip/M24_24014DEV4_repo # timeout=10
Fetching upstream changes from https://github.com/stefanluethin-microchip/M24_24014DEV4_repo
> git --version # timeout=10
> git --version # 'git version 2.34.1'
> git fetch --tags --force --progress -- https://github.com/stefanluethin-microchip/M24_24014DEV4_repo +refs/heads/*:refs/remotes/origin/* # timeout=10
Checking out Revision 371ab7833bafa4331b943e9c75e6c35381c7c3fa (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 371ab7833bafa4331b943e9c75e6c35381c7c3fa # timeout=10
Commit message: "simple bash for first simple Jenkins-pipeline"
> git rev-list --no-walk 371ab7833bafa4331b943e9c75e6c35381c7c3fa # timeout=10
[test1_freestyle] $ /bin/sh -xe /tmp/jenkins8881717103904642542.sh
+ chmod +x files_cicd/Jenkins_pipeline_test.sh
+ files_cicd/Jenkins_pipeline_test.sh

##--- start Jenkins-pipeline @Mo 10. Jun 17:05:33 CEST 2024
##--- Step1: check env...
##... run CheckEnv ...

##--- Step2: generate/update Makefiles...
##... run prjMakefileGen ...

##--- Step3: compile...
##... run make ...

##--- Step4: check if compile=successful...
##... run resultCheck ...

##--- Step5: run HW-in-the-loop ...
##... run mdb myTest_mdbcmds.txt ...

##--- done with Jenkins-pipeline @Mo 10. Jun 17:05:38 CEST 2024
Finished: SUCCESS
```

Jenkins\_pipeline\_test.sh

```
1 #!/usr/bin/bash
2 echo "##--- start Jenkins-pipeline @`date`"
3 echo ""
4 echo "##--- Step1: check env..."
5 echo "##... run CheckEnv ..."
6 sleep 1
7 echo ""
8 echo "##--- Step2: generate/update Makefiles..."
9 echo "##... run prjMakefileGen ..."
10 sleep 1
11 echo ""
12 echo "##--- Step3: compile..."
13 echo "##... run make ..."
14 sleep 1
15 echo ""
16 echo "##--- Step4: check if compile=successful..."
17 echo "##... run resultCheck ..."
18 sleep 1
19 echo ""
20 echo "##--- Step5: run HW-in-the-loop ..."
21 echo "##... run mdb myTest_mdbcmds.txt ..."
22 sleep 1
23 echo ""
24 echo "##--- done with Jenkins-pipeline @`date`"
25 echo ""
```



## *Everyone still following?*

### *Seen so far:*

- 1. Intro, what is CI/CD, structure, ...*
- 2. MPLAB-analysis & -testing capabilities*
- 3. Used FW, how to build (part1) & run + expected results (part2)*
- 4. CI/CD-pillar1: SCM/git*
- 5. CI/CD-pillar2: Jenkins-Master & -Node/Agent & -Pipeline*

### *Next*

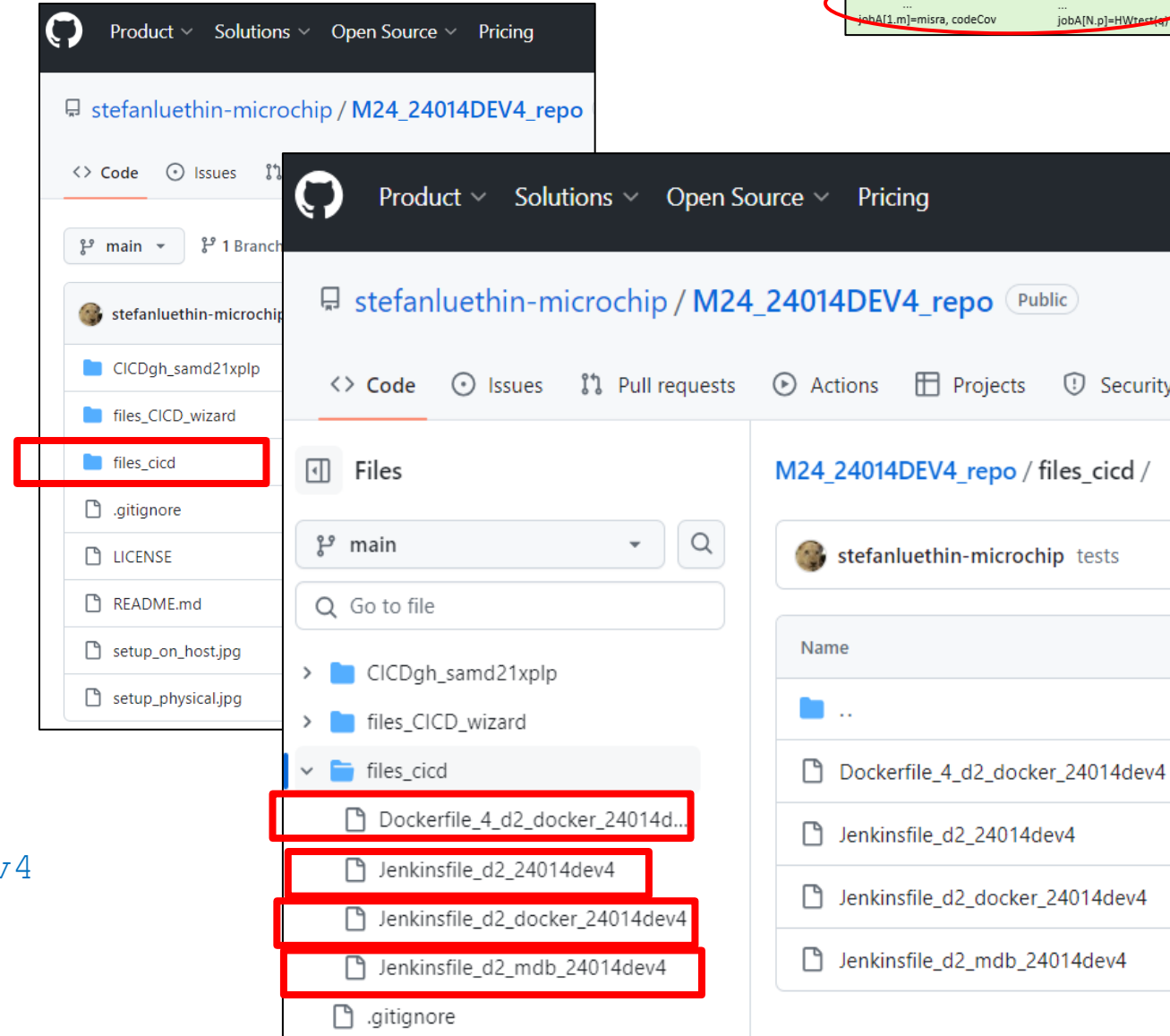
- CI/CD-pillar2: Jenkins-Pipeline-Script (=sequence-of-tasks)*

# CICD-p2/Jenkins: J-Pipeline Script (0)

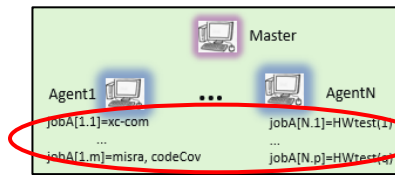


Jenkins **Pipeline script** or content (=task-sequence):

1. recap: clear how to create&config J-pipeline but what's the content of a J-Pipeline?
2. Next let's look at the first pipeline used in demo1 '**Jenkinsfile\_d2\_24014dev4**'
3. All pipeline-scripts used, are in repo folder **files\_cicd\**
4. This class will re-use same Pipeline **m24\_24014dev4\_pipeline** and just change actual pipeline-script executed
5. Pipeline-Scripts used are:  
Pipeline-Script1 **Jenkinsfile\_d2\_24014dev4**  
=Demo0-part1 **compile** in J-Pipeline  
Pipeline-Script2 **Jenkinsfile\_d2\_docker\_24014dev4**  
=Pipe1 BUT running in DockerContainer  
**Dockerfile\_4\_d2\_docker\_24014dev4**  
Pipeline-Script3 **Jenkinsfile\_d2\_mdb\_24014dev4**  
= Demo0-part2 **HW-run** in J-Pipeline



# CICD-p2/Jenkins: J-Pipeline Script (1)



## Jenkins *Pipeline-Script* (cont)

### 1. Initial configuration

- Language format  
-> groovy

### 2. Start of Pipeline `pipeline {...}`

### 3. Environment `environment{...}` eg: searchpath

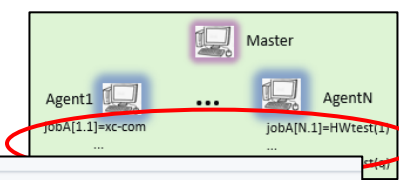
```
#!/usr/bin/env groovy

// -SL: This file was generated by the CI/CD Wizard version 1.0.375.
// -SL: See the user guide for information on how to customize and use this file.
// -SL: Jenkins Pipeline syntax type = Declarative
// -SL: Pipeline used for 24014DEV4-cicd class for Masters2024

pipeline {
 environment {
 DEMO_V = "demo_simple_d1"
 JPIPELINE = "files_cicd/Jenkinsfile_d2_24014dev4"
 // -SL: root path to tools and IDE
 MPLABX_ROOT = "/opt/microchip/mplabx"

 // -prjMakefilesGenerator to read MPLABX-configuration.xml and create Makefiles
 PRJ_MK_GEN_CALL = "${MPLABX_ROOT}" + "/" + "${MPLABX_V_2_USE}" + "/mplab_platform/bin/" + "prjMakefilesGenerator"
 // -SL: simple bash-scrp to find *.elf-file and print path+name
 ELF_TEST_SCR = "." + "/" + "result_check.sh"
 // -SL: extend searchpath with above set tools
 //PATH = "${XC32_BIN_PATH}:${MPLABX_BIN_PATH}:${env.PATH}"
 PATH = "/opt/microchip/xc32/v4.40/bin:/opt/microchip/mplabx/v6.20/mplab_platform/bin:${env.PATH}"
 }
}
```

# CICD-p2/Jenkins: J-Pipeline Script (2)



Jenkins **Pipeline** content (cont)

4. Where to run this job on?

`agent any`

5. What to do anyway?

->defined in **Stages**, like  
Build, Analysis, HWrun,...

Each Stage has 1-n **Steps**,  
like Build-stage has steps:  
envCheck->prep  
->CreateMakefiles->compile  
->cleanup

=>Stage-Build/Step1=envCheck

```
stage ('Build') {...
 steps {envCheck...
 sh (
 label: 'env check'
)
 }
}
```

->simple bash-syntax 'sh'

```
45 }
46 agent any
47
48 stages {
49 stage('Build') {
50 steps {
51 sh(
52 label: 'environment check',
53 script: ""
54 set +x
55 echo "###-SL: =====
56 echo "###-SL: Jenkins-Pipeline ${env.JPIPELINE}"
57 echo "###-SL: from ${env.JOB_NAME}"
58 echo "###-SL: START stage:build / step:environment check
59 echo "###-SL: =====
60 #-SL-comment: full env (only for debugging)
61 #-printenv
62 echo "###-SL: "
63 echo "###-SL: Running ${env.BUILD_ID} on ${env.JENKINS_URL}"
64 echo "###-SL: prj_root_p = ${env.PRJ_ROOT_P}"
65 echo "###-SL: WS = ${env.WORKSPACE}"
66 echo "###-SL: PATH = ${env.PATH}"
67 which make
68 which xc32-gcc
69 which prjMakefilesGenerator.sh
```

# CICD-p2/Jenkins: J-Pipeline Script (3)



Jenkins *Pipeline* content (cont)

- Next **Step** of **Stage** 'Build' is 'cleanup' -> just removes temp-stuff

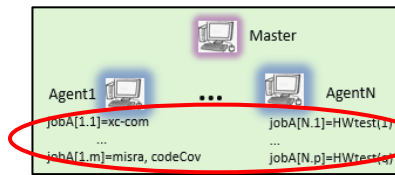
```
stage{ 'Build'
steps {
...
sh (
 label: 'cleanup'
```

```
M24_24014DEV4_repo / files_cicd / Jenkinsfile_d2_24014dev4 in main
Cancel changes Commit changes..

Edit Preview Code 55% faster with GitHub Copilot Tabs 8 No wrap

48 stages {
49 stage('Build') {
50 steps {
51 sh(
52 label: 'environment check',
53 script: """
54 set +x
55 echo "###-SL: =====
56 echo "###-SL: Jenkins-Pipeline ${env.JPIPELINE}"
57 echo "###-SL: from ${env.JOB_NAME}"
58 echo "###-SL: START stage:build / step:environment check"
59 echo "###-SL: =====
60 #-SL-comment: full env (only for debugging)
61 #-printenv
62 """
63) // -SL: end 'step: env-check'
64 }
65 }
66 }
67
68 stage('Cleanup') {
69 steps {
70 sh(
71 label: 'cleanup (if needed)',
72 script: """
73 set +x
74 echo "###-SL: =====
75 echo "###-SL: START stage:build / step:cleanup"
76 echo "###-SL: =====
77 cd ${env.WORKSPACE}
78 rm -rf ./build
79 """
80)
81 }
82 }
83 }
```

# CICD-p2/Jenkins: J-Pipeline Script (4)



Jenkins **Pipeline** content (cont)

7. Next **Step** of **Stage** 'Build' is 'Build Makefiles' using MDBCore-module `prjMakefileGenerator.sh`
- ```
stage{'Build'  
  steps {  
    ...  
    sh (  
      label: '..makefiles'
```

```
M24_24014DEV4_repo / files_cicd / Jenkinsfile_d2_24014dev4 in main  
[Cancel changes] [Commit changes]  
[Edit] [Preview] Code 55% faster with GitHub Copilot  
[Tabs] 8 [No wrap]  
97 sh(  
98     label: 'Generate build makefiles',  
99     script: ""  
100         set +x  
101         echo "###-SL: ===== "  
102         echo "###-SL:      START stage:build / step:re-creating makefiles for ${PRJ_MK_STR}"  
103         echo "###-SL: ===== "  
104         #-SL: prjMakefilesGenerator.sh reads the MPLABX-file configuration.xml and out of it creates Makefiles  
105         #-/opt/microchip/mplabx/v6.20/mplab_platform/bin/prjMakefilesGenerator.sh -v -f \  
106         #- ./CICDgh_samd21xplp/firmware/CICDgit_samd21xplp.X@samd21xplp  
107         "${PRJ_MK_GEN_CALL}" -v -f ${PRJ_WS_REL_P}@\${env.MPLABX_CFG_N}  
108         echo "###-SL: ===== "  
109         echo "###-SL:      END stage:build / step:re-creating makefiles"  
110         echo "###-SL: ===== "  
111         set +x  
112         ""  
113     ) //-SL: end 'step: update/build Makefiles'
```

Compile:

```
(M24_24014DEV4_repo)> CICDgh_samd21xplp\mdb\CICDgh_samd21xplp_test2a_comp.bat
```

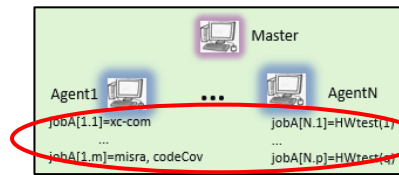
```
<project-path>\firmware\  
| -<projectName>.X  
| | -Makefile  
| | -mcc_prj.mc3  
4.1 | -nbproject\  
| | | -project.xml  
| | | -configuration.xml  
| -src\ -> your sources
```

4.2
create/update Makefiles
->**prjMakefilesGenerator** (MDBCore)

4.3
compile using **make**
(MPLABX-GnuTools)

4.4
result in
<prj>.X\dist\...*.elf

CICD-p2/Jenkins: J-Pipeline Script (4)



Jenkins **Pipeline** content (cont)

8. Next **Step** of **Stage** 'Build' is the actual compile using **make**

```
stage{ 'Build'
  steps {
    ...
    sh (
      label: 'make'
```

```
M24_24014DEV4_repo / files_cicd / Jenkinsfile_d2_24014dev4 in main

Edit Preview Code 55% faster with GitHub Copilot

114 sh(
115     label: 'compile = running Makefile',
116     script: ""
117         set +x
118         echo "###-SL: =====
119         echo "###-SL:      START stage:build / step: compile=make"
120         echo "###-SL: =====
121         cd "${env.PRJ_WS_REL_P}"
122         echo "###-SL: current path `pwd`"
123         make clean
124         make CONF="${env.MPLABX_CFG_N}"
125         echo "###-SL: =====
126         echo "###-SL:      END stage:build / step: compile=make"
127         echo "###-SL: =====
128         set +x
129         ""
130     ) //--SL: end 'step: compile-w-make'
```

Compile:

(M24_24014DEV4_repo)> CICDgh_samd21xplp\mdb\CICDgh_samd21xplp_test2a_comp.bat

4.1	4.2	4.3	4.4
<pre><project-path>\firmware\ -<projectName>.X -Makefile -mcc_prj.mc3 -nbproject\ -project.xml -configuration.xml -src\ -> your sources</pre>	create/update Makefiles -> prjMakefilesGenerator (MDBCore)	compile using make (MPLABX-GnuTools)	result in <prj>.X\dist\...*.elf

CICD-p2/Jenkins: J-Pipeline Script (5)



Jenkins **Pipeline** content (cont)

9. Last **Step** of **Stage** 'Build' is to check result was *.elf created?

```
stage{'Build'
steps {
...
sh (
label: 'compile success'
...

chmod +x result_check.sh
result_check.sh
```

10. End of
stage{'Build'

```
M24_24014DEV4_repo / CICDgh_samd21xplp / firmware / CICDgit_samd21xplp.X / result_check.sh
stefanluethin-microchip chmod +x

Code Blame Executable File · 15 lines (14 loc) · 360 Bytes Code 55% faster with GitHub Copilot

1 #!/bin/bash
2 #- Description: simple find to get path+name of elf-file
3 #- v1.0: first version (SL, 10.5.22)
4
5 #resultf_n=find ./dist -name *.elf
6 resultf_n=find . -name *.elf
7 #echo "result check - start"
8 if [ "${resultf_n}" != "" ]; then
9     echo "Resultfile = ${resultf_n}"
10    exit 0
11 else
12     echo "no resultfile"
13    exit 1
14 fi
15 #echo "result check - done"
```

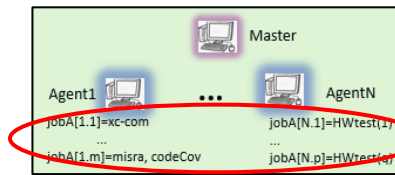
```
M24_24014DEV4_repo / files_cicd / Jenkinsfile_d2_24014dev4 in main
Edit Preview Code 55% faster with GitHub Copilot

131 sh(
132     label: 'compile successful?',
133     script: ""
134         set +x
135         echo "###-SL: =====
136         echo "###-SL:      START stage:build / step: result-check"
137         echo "###-SL: =====
138         echo "###-SL (start): result check..."
139         cd "${env.PRJ_WS_REL_P}"
140         chmod +x ${ELF_TEST_SCR}
141         ${ELF_TEST_SCR}
142         echo "###-SL: =====
143         echo "###-SL:      END stage:build / step:result-check"
144         echo "###-SL: =====
145         set +x
146         ""
147     ) //-SL: end 'step: post-proc, check elf exists with bat-script'
148 } //-end 'steps' in 'stage=build'
149 } //-end stage=build
150 } //-end 'stages'
```

Compile:			
(M24_24014DEV4_repo)> CICDgh_samd21xplp\mdb\CICDgh_samd21xplp_test2a_comp.bat			
4.1	4.2	4.3	4.4
<pre><project-path>\firmware\ <-projectName>.X <-Makefile <-mcc_prj.mc3 <-nbproject\ <-project.xml <-configuration.xml <-src>.<-> your sources</pre>	<pre>create/update Makefiles ->prjMakefilesGenerator (MDBCore)</pre>	<pre>compile using mak (MPLABX-GnuTools)</pre>	<pre>result in <prj>.X\dist\...*.elf</pre>



CICD-p2/Jenkins: J-Pipeline Script (5)



Jenkins *Pipeline* content (cont)

11. Final part of each Pipeline is (not a Step-of-a-Stage)

- post-stage contains
- a part *always* run
- a part only run on *success*

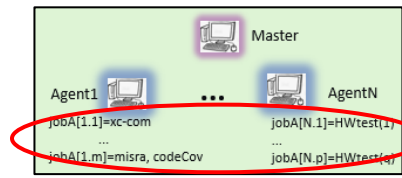
12. And last is the closing quote of the pipeline

```
M24_24014DEV4_repo / files_cicd / Jenkinsfile_d2_24014dev4 in main

Edit Preview Code 55% faster with GitHub Copilot Tabs

152     post {
153         //-SL: there're different outcomes 'sucess, failure,..., always' of the
154         //-SL: stages-step that are optional, hence don't have to be specified
155         always {
156             // Clean workspace after build
157             echo "###-SL: ====="
158             echo "###-SL: post / always from current Jenkins-Pipeline ${env.JPIPELINE}"
159             echo "###-SL: ====="
160             //cleanWs() //-Jenkins-buildin fct to cleanup Workspace
161         } //-end 'post/always'
162         success {
163             echo "###-SL: ====="
164             echo "###-SL: post / success from current Jenkins-Pipeline ${env.JPIPELINE}"
165             echo "###-SL: ====="
166         } //-end 'post/success'
167     } //-end 'post'
168
169 } //- end 'pipeline'
```

CICD-p2/Jenkins: J-Pipeline Script (6: summary)



Jenkins *Pipeline* content - summary

1. Initial settings, language
2. Start `pipeline` {...
3. Env `environment`{...}
4. Where to run -> `agent`...
5. What to do -> `stages`{...
Stage contains steps, 1.step
`step{envCheck ...`
6. `step{cleanup ...`
- ...
7. `step{GenMakefiles...`
8. `step{compile/make ...`
9. `step{resultCheck...`
10. End of Stage('build')
11. Final pipeline stage **POST** with
 - run always x
 - run only on success
 - run only on failure
12. End of pipeline

Sample tasks for the final wrapup-stage

- **always** send a mail that pipeline finish with result=pass, fail,...
- On **success** collect compile.log, hwrn.log, misra.log,... and store centrally
- ...

CICD-p2/Jenkins: J-Pipeline summary (1)

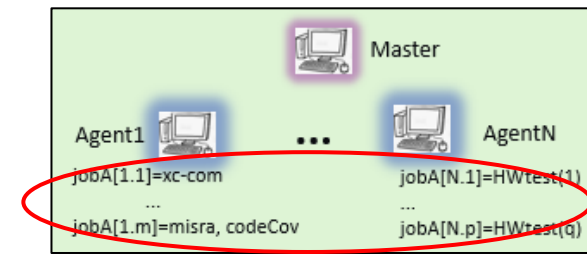
Jenkins **Pipeline-Script** = content - summary

1. J-Pipeline **creation** - major steps

- Create from J-Master dashboard with 'new item'
- In J-Pipeline **configuration** you specify
 - Name, discard-strategy or when-to-run/trigger
 - Connection to git-repo and which branch to monitor
 - The pipeline-script to execute
- **Start** a J-Pipeline automatically with **Jenkins-Trigger** or manually from J-Master dashboard
- Monitor execution on console-output or with addOn OceanBlue

2. J-Pipeline **Script** = content = your sequence-of-tasks with **Stages+Steps**, major parts are

- Starts with `pipeline{...`
- Environment, like searchpath in `environment{...`
- Where run on -> `agent ...`
- What to do consists of `stages{...` and `steps{...`
- Final wrapup stage with targets to run `always` or only `onSuccess`



CICD-p2/Jenkins: overall summary

- Jenkins-**Master** down -> cannot login into Jenkins-**Master** webIF, so check with:

```
(lnx)$ service --status-all | grep -i jenkins  
[ + ] jenkins
```

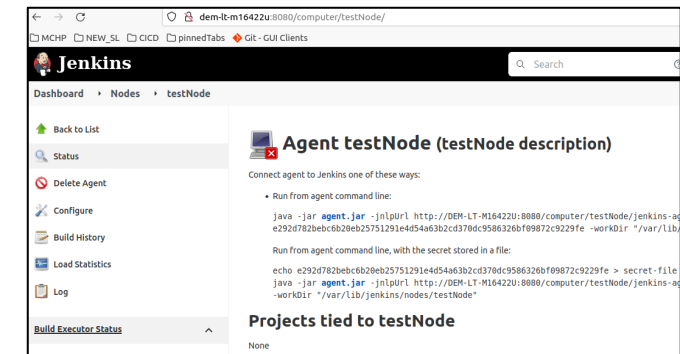
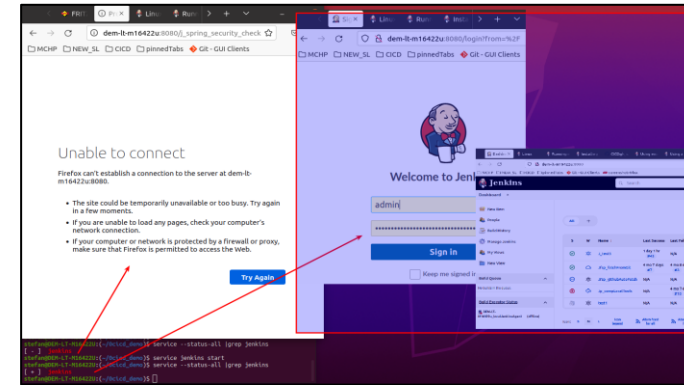
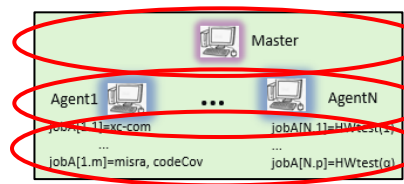
- Jenkins-**Node/Agent** is work-horse to run the jobs

- Create from J-Master dashboard -> manageNodes
- Start with java-call - running script = **Agent**
- Configure how many jobs can run on one agent
 - computer-w-bigRAM = Simulation
 - computer-w-HWrev1 attached -> HWrun

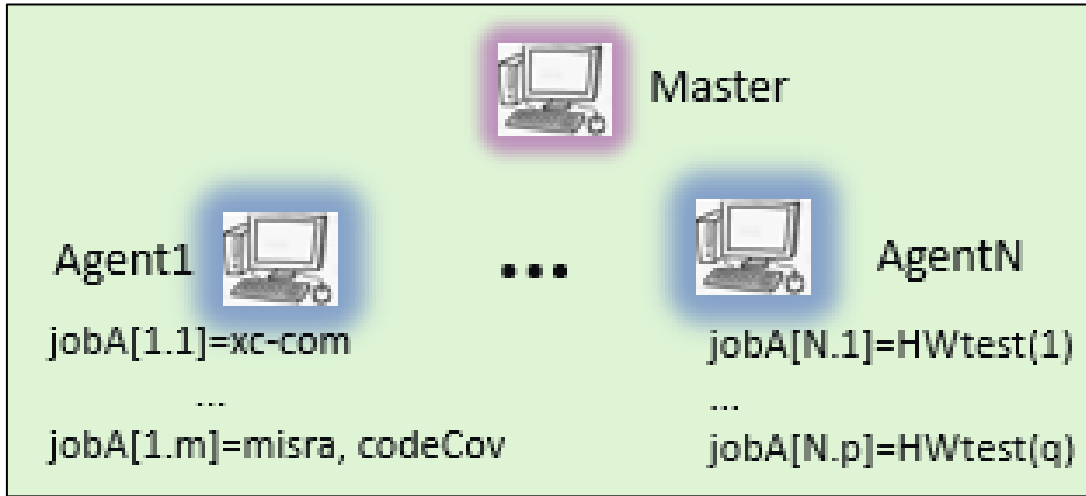
- Jenkins-**Pipeline**

- Create from J-Master -> newItem
- Configure: name, URL-2-gitrepo, pipeline-script
- **Pipeline-script** finally contains the sequence-of-tasks
- Pipeline has stages and steps and Pipeline-Stages can be:
 - Build (prep->createMakefiles->compile)
 - Analysis (misra, thread-safety-check, codeCoverage..)
 - Testing (unitTests , Simulation , Hwrun)
 - Post (onFailure, onSuccess, Always) collect report, ...

- Next we'll look at the last CICD-pillar, the runtime-engine



CICD-p2/Jenkins: Jenkins in a nutshell



1. Jenkins **Master** (on Linux)
 1. webIF -> IP:8080
 2. service -> `service --status-all | grep -i jenkins`
2. Jenkins **Item->Node->Agent**
 1. J-Master-dashboard: new **Item**
 2. **Node**=virtualJob with config (#Executors,...)
-> '`java -jar <node-config>`' call to start Node
 3. **Agent**=actual process of java-call/Node
3. Jenkins **Pipeline**
 1. URL-2-repo, branch-2-use, discard-strategy, eventTriggers
 2. What to do: bash-script, pipeline-script,...
4. Jenkins **Pipeline-Script**
 1. Actual Task-Sequence: build, analysis, hwrn,...
 2. Stages and Steps
 3. Final stage POST with Always,Success,...

Everyone still following?

Seen so far:

- 1. Intro, what is CI/CD, structure, ...*
- 2. MPLAB-analysis & -testing capabilities*
- 3. Used FW, how to build (part1) & run + expected results (part2)*
- 4. CI/CD-pillar1: SCM/git*
- 5. CI/CD-pillar2: Jenkins-Master & -Node/Agent & -Pipeline and last looked at the Pipeline-script (=the bread&butter)
=>summary on next slide*

Next

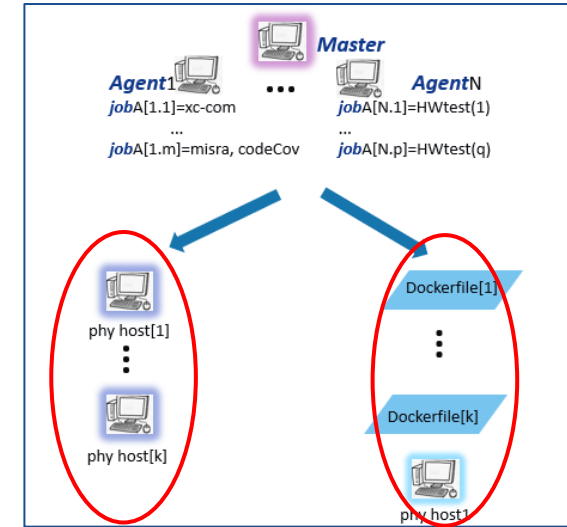
- CI/CD-pillar3: runtime environment (phyPC or Docker)*

Agenda

- Introduction to CICD
- MPLAB®X ecosystem testing features and analysis capabilities
- Demos (part1): used FW and expected result
- **Three pillars of a CICD system:**
 - SourceControlManagement=SCM/git
 - Automation/Jenkins: Jenkins-Master -> -Agent -> Pipeline -> Demo (bash-in-Pipe)
 - **RuntimeEnv/PC-Docker**
- Demos (part2): simple-pipeline, Jenkins-Trigger, errors+solutions
- MPLAB®X CICD wizard
- Demos (part3): HW-in-the-loop testing, DockerContainer
- Summary and Outlook

CICD-p3/Docker: Runtime Engine & Docker

- **Third pillar is the actual runtime environment**
 - basically, need *some* runtime environment to run your compiles on
 - Runtime Environment can be
 - a set of physical PCs (manually install tools !)
 - a set of DockerContainer
- **What actually is Docker?**
 - some kind of light-weight-VM on OS-level compared to 'real' VM like VirtualBox
 - So, creating a DockerContainer basically means to setup a VM and install tools
 - These instructions are stored in a Dockerfile - will review a full Dockerfile next
- **Physical PC vs Docker**
 - you can attach real HW to a physical PC, BUT not to a VM (well with ICE4 via IP you could...)
 - Easy to maintain as it is only a simple txt-file =DockerFile, instead of physicalPC



CICD-p3/Docker: Working with DockerContainer (1)

- A Docker**Container** is (-> from <https://docs.docker.com/get-started/>)
 - a runnable instance of an Docker**Image**
 - you can create, start, stop, move, or delete a container using the Docker API or CLI
 - Can be run on local machines, virtual machines, or deployed to the cloud and is portable (run on any OS)
 - Is isolated from other containers and runs its own software, binaries, configurations, etc.
- Some suggestions working with DockerContainer
 - Sometimes difficult to debug problems of a DockerFile within a Jenkins-Pipeline
 - Better to test the DockerFile used in the JenkinsPipeline standalone on a Linux-computer or ideally the Lnx-computer running the Jenkins-Node/Agent ->next slide
 - Some more commands&hints in LabManual

Next, let's see a simple example, before we look at a complete Dockerfile

CICD-p3/Docker: Working with DockerContainer (3)

Install/start/stop and delete DockerContainers (continued) - a simple example:

```
#- create new directory 'initialDockerTest', cd into and create 2 files main.py + Dockerfile
#- check which DockerContainers are up
```

```
(lnx)$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	...	NAMES
107212ca7590	hello-world	"/hello"	...	jolly_Easley

```
#- build new DockerContainer
```

```
(.../initialDockerTest)$ docker build -t main_py-test .
[+] Building 50.5s (7/7)                                docker:default
...
=> => naming to docker.io/library/main_py-test 0.0s
```

```
#- which Containers up now? -> still same, we didn't start it yet!
```

```
(lnx)$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	...	NAMES
107212ca7590	hello-world	"/hello"	...	jolly_Easley

```
#- start new DockerContainer 'main_py-test' now
```

```
(lnx)$ docker run main_py-test
```

Docker is magic!

```
#- which DockerContainers are up now? -> still the same, as we didn't start it yet!
```

```
(lnx)$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	...	NAMES
a7d745545419	main_py-test	"python ./main.py"	...	fervent_margulis
107212ca7590	hello-world	"/hello"	...	jolly_Easley

```
#- 'main.py'
#!/usr/bin/env python3
print("Docker is magic!")
```

```
#- 'Dockerfile'
FROM python:latest
COPY main.py /
CMD [ "python", "./main.py" ]
```

CICD-p3/Docker: complete sample Dockerfile from CICD-wizard ^{updated}

- DockerFile on the right creates a DockerContainer running Debian-Lnx with MPLAB®X-v5.50 & XC32-v2.50 and various LNX-tools
- To create this 'VM' do
(lnx)\$ docker
build \
Dockerfile
- use MPLAB®X
CICD-wizard to create
complete Dockerfile
- Obviously storing such
an ASIC-file is easier
and smaller than a
VM-image or physicalPC

```
# Dockerfile generated by the CI/CD Wizard version 1.0.375.
#-start from a prepared docker-image with a debian-distro from hub.docker.com
FROM debian:buster-slim
ENV DEBIAN_FRONTEND noninteractive

#-install addOn-tools needed into the preloaded debian-image
USER root
RUN dpkg --add-architecture i386 && apt-get update -yq ...
    && apt-get clean && rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*

# Download and install MPLAB X IDE version 5.50
ENV MPLABX_VERSION 5.50
RUN curl -fSL -A "Mozilla/4.0" -o /tmp/mplabx-installer.tar \
    "https://ww1.microchip.com/downloads/en/DeviceDoc/MPLABX-v${MPLABX_VERSION}-linux-installer.tar" \
    && tar xf /tmp/mplabx-installer.tar -C /tmp/ && rm /tmp/mplabx-installer.tar \
    && USER=root ./tmp/MPLABX-v${MPLABX_VERSION}-linux-installer.sh ... \
    && rm ./tmp/MPLABX-v${MPLABX_VERSION}-linux-installer.sh ...
ENV PATH /opt/microchip/mplabx/v${MPLABX_VERSION}/mplab_platform/bin:$PATH

# Download and install toolchain
ENV TOOLCHAIN_VERSION 2.50
RUN curl -fSL -A "Mozilla/4.0" -o /tmp/${TOOLCHAIN}.run \
    "https://ww1.microchip.com/.../xc32-v${TOOLCHAIN_VERSION}-full-install-linux-installer.run" \
    ... && rm /tmp/${TOOLCHAIN}.run
ENV PATH /opt/microchip/${TOOLCHAIN}/v${TOOLCHAIN_VERSION}/bin:$PATH

# Download and install Microchip.SAMD21_DFP.3.6.133
RUN curl -fSL -A "Mozilla/4.0" -o /tmp/tmp-pack.atpack ...

ENV BUILD_CONFIGURATION samd21xplp
#-install more addOn-tools into debian-image needed eg for doxygen or simulation
RUN apt-get update -yq ... && apt-get clean && rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*
```

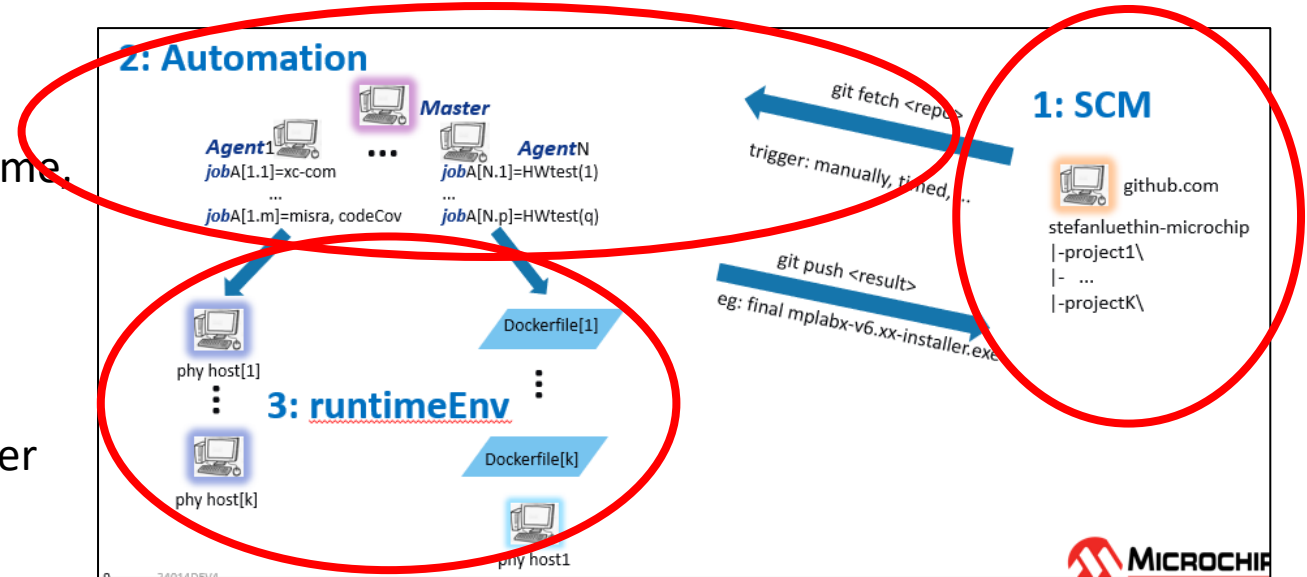
CICD-p3/Docker: summary

- Pro and Con of DockerContainer
 - A real PC or a setup of real PCs
 - Advantages: you can use usb dongle-license -> can attach real HW
 - Disadvantages: if only 1xPC (->Master+Agents) easy, BUT multiple PCs need management
 - A set of Dockerfiles
 - Advantages: just a few kB-files (->DockerFiles), easier to handle than real HW/PCs
 - Disadvantages: need network-licenses -> cannot attach real HW
- Summary on part3 'Runtime Environment'
 - The Jenkins Runtime-Environment can be a physicalPC or a virtualHost=DockerContainer
 - There are Pros&Cons for both realPCs or Docker
 - A DockerFile is just a description how to build a VM/DockerContainer or DockerImage
 - Create a DockerImage/Container on the fly with 'docker build -f DockerFile...'

CICD: overall summary on CICD

Summary on CICD system

- What is it?
 - a methodology to save overall development time, with divide-and-conquer
- 3 major pillars
 - **sources control management** -> SCM like git
 - **automation** with Jenkins
 - **runtime-environment** real or virtual with Docker
- **SCM**/SourceControlManagement
 - holds different FW-revisions
- **Jenkins** for automation
 - a Jenkins **Master** (provides webIF for configuration) and 1-n **Agents** (created in Master-webIF)
 - an **Agent** can be a real PC or a virtual environment, like a **DockerContainer**
 - on certain events (=Triggers, like manual->timed->branch-checkin) a sequence-of-tasks is executed
 - this sequence is called a **Pipeline**
 - a Pipeline consists of Stages (build->check->...) with Steps (stage-build: Makefiles+make+cleanup)
- **Docker**
 - like a VM and started or created on-the-fly
 - a DockerFile describes what the DockerContainer/VM contains
 - easier to handle than x real PCs



Everyone still following?

Seen so far:

- 1. Intro, what is CI/CD, structure, ...*
- 2. MPLAB-analysis & -testing capabilities*
- 3. Used FW, how to build (part1) & run + expected results (part2)*
- 4. CI/CD-pillar1: SCM/git*
- 5. CI/CD-pillar2: Jenkins-Master & -Node/Agent & -Pipeline and last looked at the Pipeline-script (=the bread&butter)*
- 6. CI/CD-pillar3: runtime environment (phyPC or Docker)*

Next

- Demos: compile-in-J-pipeline only 1xStage and no Docker*

Agenda

- Introduction to CICD
- MPLAB®X ecosystem testing features and analysis capabilities
- Demos (part1): used FW and expected result
- Three pillars of a CICD system:
 - SourceControlManagement=SCM/git
 - Automation/Jenkins: Jenkins-Master -> -Agent -> Pipeline -> Demo (bash-in-Pipe)
 - RuntimeEnv/PC-Docker
- **Demos (part2): simple-pipeline, Jenkins-Trigger, errors+olutions**
- MPLAB®X CICD wizard
- Demos (part3): HW-in-the-loop testing, DockerContainer
- Summary and Outlook

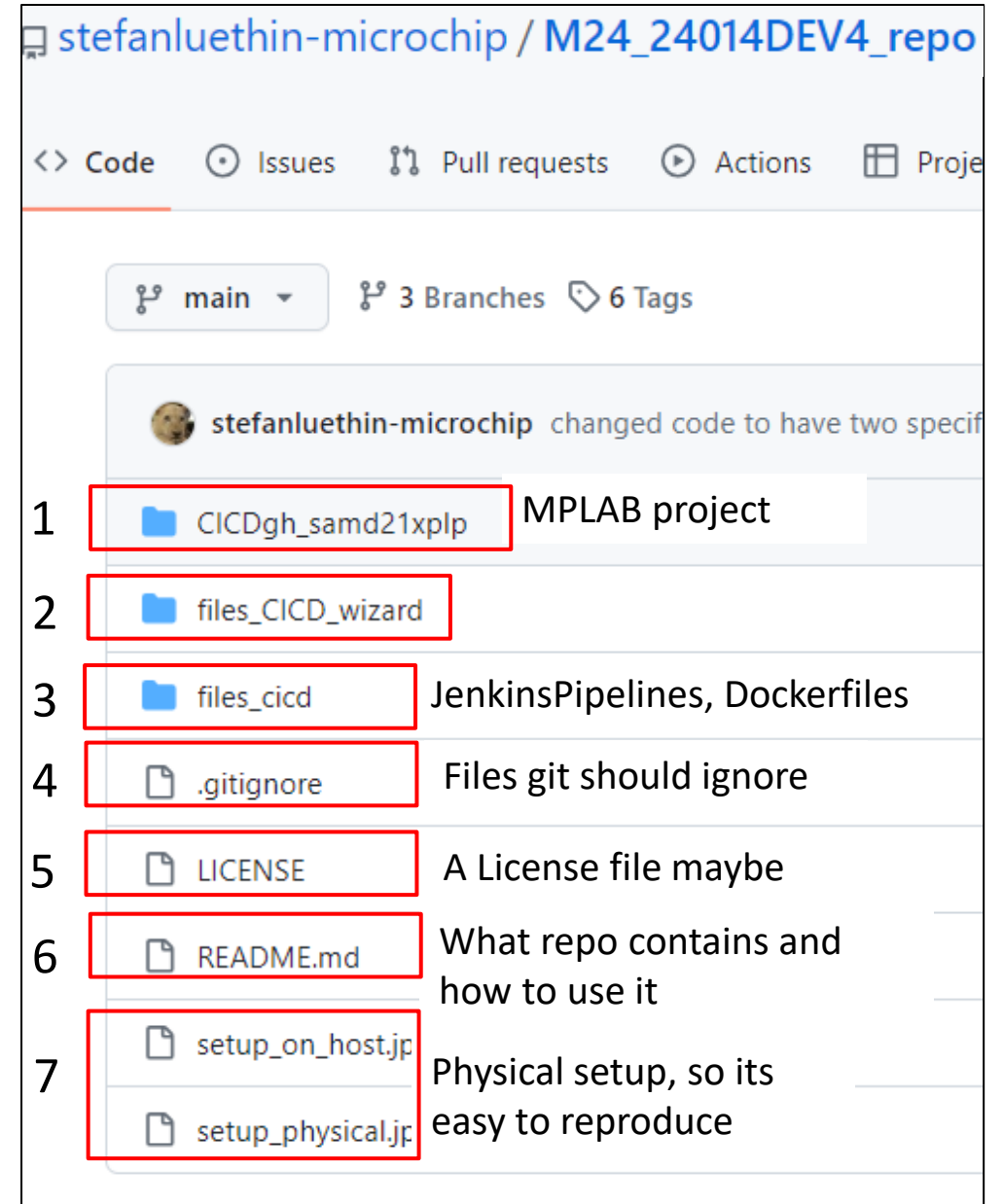
Demos on github (*just reminder*)

All labs/files are on the following github-repo:

www.github.com/stefanluethin-microchip/M24_24014DEV4_repo

<git_repo: M24_24014DEV4_repo>/

- 1 | |- CICDgh_samd21xplp/
 - 1 | |-firmware/ = MPLAB-project
 - 1 | |-mdb/ = MDBCore-scripting
- 2 | |- files_cicd_wizard = Output of MPLAB®X CICD-wizard
- 3 | |- files_cicd = JenkinsPipelines, Dockerfiles of this class
- 4 | |- .gitignore = git ignore file
- 5 | |- LICENSE = a license file
- 6 | |- README.md = Readme for repo
- 7 | |- setup_*.jpg = pictures of setup



Demo1: simple-pipeline & triggers (*just reminder*)

- Demo0 contains two parts

Part1 = compile: preparation -> create Makefiles -> compile

```
()> C1CDgh_samd21xplp_test1a_Setup_env.bat && C1CDgh_samd21xplp_test2a_comp.bat
```

Part2 = Run-on-HW: connect-2-debugger -> download *.elf -> run

```
()> ...\C1CDgh_samd21xplp_test3a_mdbbat-run.bat
```

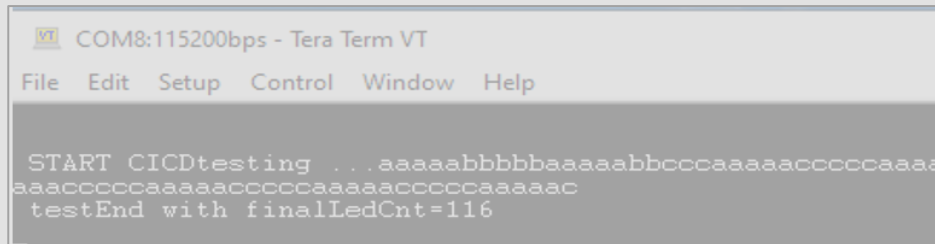
- Demo next:

- Demo1 will re-do Demo0/part1=compile but from **Jenkins-pipeline**

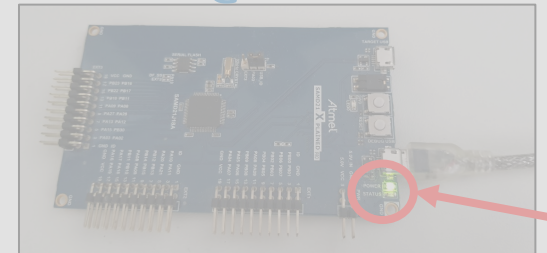
- Demo2 will just show typical errors&solutions

- Demo3 will re-do Demo0/part1&part2=compile & HW-run but from **Jenkins-pipeline**

printf()/vCOM/console -> msg on UART



LEDblink @ 1Hz



- Demo4 will re-do Demo0/part1=compile but from **Jenkins-pipeline inside a DockerContainer**

Demo1: simple-pipeline & triggers

Demo1: = simple pipeline1 'fetch+compile' and Jenkins-trigger

Task: demo1 basically re-runs Demo0 , only part1=compile

- A. Use pipeline `m24_24014dev4_pipeline` and
- as pipeline-script use pipeline1 (= *fetch+compile-only*)
 - `files_cicd\Jenkinsfile_d2_24014dev4`
 - agent = any (*show agent down effect*)
 - trigger execution: *manually*

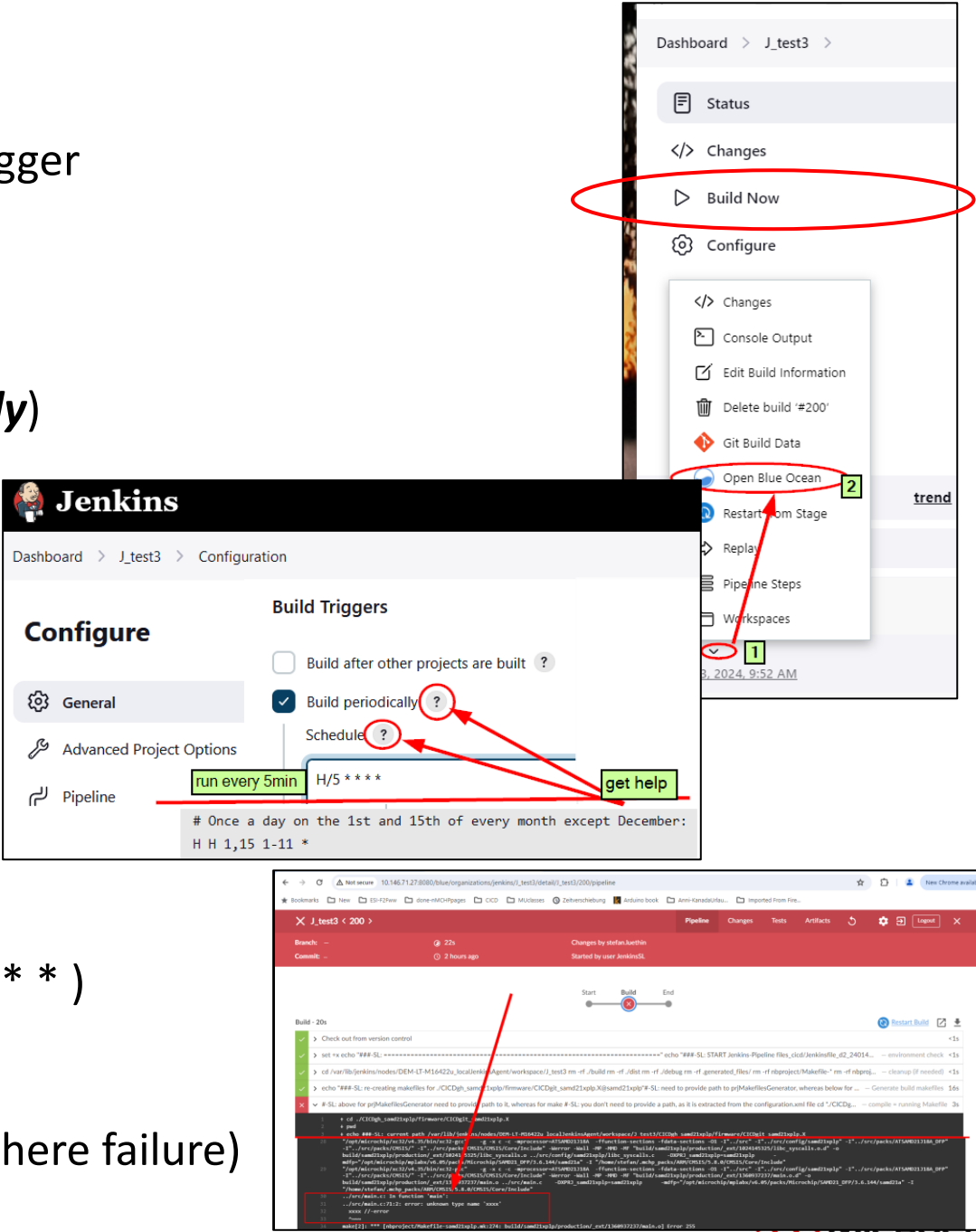
->run demo1.a now

- B. Improved trigger *timed*Trigger + showCommitError

- Keep pipeline `m24_24014dev4_pipeline` and pipeline-script '`Jenkinsfile_d2_24014dev4`'
- only enable trigger *timed*Trigger @3min (-> H/3 * * * *)

->run demo1.b now

(between 2 runs commit syntax-error -> 'BlueOcean' to find where failure)



Demo2: error+solutions

- Demo0 contains two parts

Part1 = compile: preparation -> create Makefiles -> compile

```
()> CICDgh_samd21xplp_test1a_Setup_env.bat && CICDgh_samd21xplp_test2a_comp.bat
```

Part2 = Run-on-HW: connect-2-debugger -> download *.elf -> run

```
()> ...\\CICDgh_samd21xplp_test3a_mdbbat-run.bat
```

- Demo next:

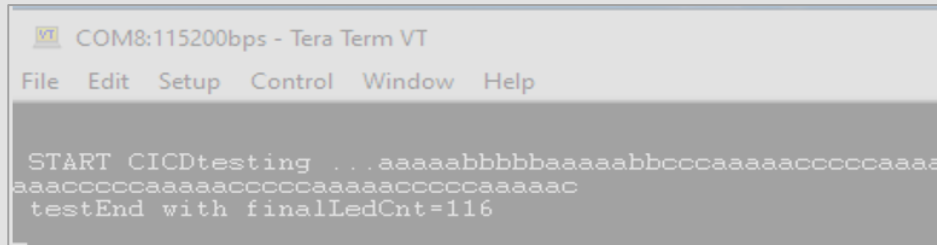
- Demo1 will re-do Demo0/part1=compile but from *Jenkins-pipeline*

- Demo2 will just show typical errors&solutions

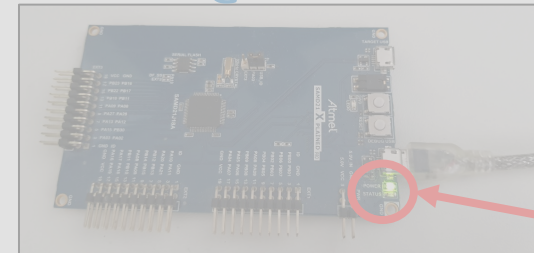
skip-4-time

- Demo3 will re-do Demo0/part1&part2=compile & HW-run but from *Jenkins-pipeline*

printf()/vCOM/console -> msg on UART



LEDblink @ 1Hz



- Demo4 will re-do Demo0/part1=compile but from *Jenkins-pipeline inside a DockerContainer*

Demo2: error+solutions

skip-4-time

Demo2: ==problems+solutions

Task: just show various issues a user might face AND how to solve it

1. Show effect if Jenkins-Masters is down and how to solve
2. Show effect if Jenkins-client/agent(s) are down and what to do
3. Various errors faced and how to solve
 - a. syntax-errors (like comments with '//' in stage 'sh()')...
-> easy to see in console-log
 - b. Wrong path+name of Dockerfile
-> difficult to find in console-log
 - c. Only one physicalPC as Node and running 2xAgent-Jobs
-> stop agentX.jar and start pipeline and in consoleOutput notice pipeline is waiting for 'any agent' to become available
 - d. Re-running on same DockerContainer/agent very fast, why?
-> DockerContainer already built+re-used



Demo2: error+solutions (summary)

What demos did we see

- Demo0 contains two parts

Part1 = compile: preparation -> create Makefiles -> compile

```
()> CICDgh_samd21xplp_test1a_Setup_env.bat && CICDgh_samd21xplp_test2a_comp.bat
```

Part2 = Run-on-HW: connect-2-debugger -> download *.elf -> run

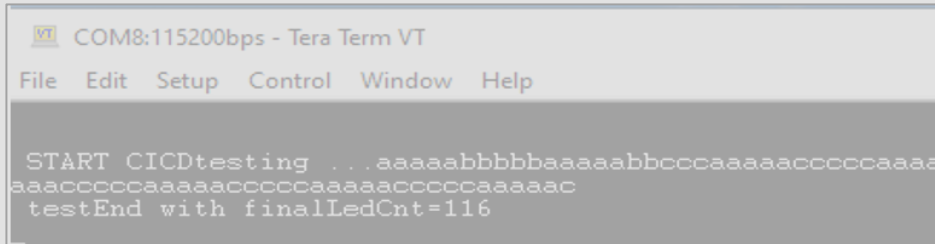
```
()> ...\\CICDgh_samd21xplp_test3a_mdbbat-run.bat
```

- Demos seen:

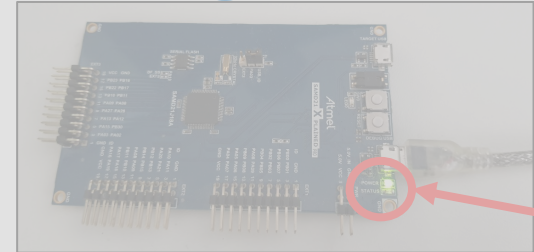
- Demo1 will re-do Demo0/part1=compile but from **Jenkins-pipeline**
- Demo2 will just show typical errors&solutions

- Demo3 will re-do Demo0/part1&part2=compile & HW-run but from **Jenkins-pipeline**

printf()/vCOM/console -> msg on UART



LEDblink @ 1Hz



- Demo4 will re-do Demo0/part1=compile but from **Jenkins-pipeline inside a DockerContainer**

->next loop back to start and again look at the MPLAB®X CICD-wizard output, now hopefully clear

Everyone still following?

Seen so far:

- 1. Intro, what is CI/CD, structure, ...*
- 2. MPLAB-analysis & -testing capabilities*
- 3. Used FW, how to build (part1) & run + expected results (part2)*
- 4. CI/CD-pillar1: SCM/git*
- 5. CI/CD-pillar2: Jenkins-Master & -Node/Agent & -Pipeline and last looked at the Pipeline-script (=the bread&butter)*
- 6. CI/CD-pillar3: runtime environment (phyPC or Docker)*
- 7. Demos: compile-in-J-pipeline only 1xStage and no Docker & error+olutions*

Next

- Go back and review CI/CD-wizard, but now hopefully you understand...*

*Short BREAK
if needed !?*

Agenda

- Introduction to CICD
- MPLAB®X ecosystem testing features and analysis capabilities
- Demos (part1): used FW and expected result
- Three pillars of a CICD system:
 - SourceControlManagement=SCM/git
 - Automation/Jenkins: Jenkins-Master -> -Agent -> Pipeline -> Demo (bash-in-Pipe)
 - RuntimeEnv/PC-Docker
- Demos (part2): simple-pipeline, Jenkins-Trigger, errors+olutions
- **MPLAB®X CICD wizard**
- Demos (part3): HW-in-the-loop testing, DockerContainer
- Summary and Outlook

MPLAB®X v6.x CI/CD-wizard: Steps and Meaning

Needed? -> Docker

1 **CI / CD Wizard**

Tools > CI / CD Wizard...

2 **CI / CD Wizard**

Steps: 1. Select Action, 2. ...

Select Action (1 of ...)

Projects:

- CICDgit_samd21xplp
- samv71xult_j1_mhc
- samv71xult_j1_mcc
- CICDgit_samd21xplp_v2

Description:

The CI/CD Wizard creates configuration files required to set up an automated build environment for an MPLAB X project.

Please select a generate option to proceed.

The Dockerfile enables creation of a Docker image with all dependencies build in a containerized environment.

Jenkins pipeline files will help you automate steps needed to achieve quality on your product.

Generate:

- Jenkins Pipeline File
- Dockerfile

3 **CI / CD Wizard**

Steps: 1. Select Action, 2. Build Environment, 3. Jenkins Settings, 4. Static Code Analysis, 5. Simulator Testing, 6. Hardware Testing, 7. Documentation, 8. Summary

Build Environment (2 of 8)

MPLAB X IDE version: 6.00

Build Configuration: samd21xplp : XC32

MPLAB XC32 Compiler version: 2.50

MPLAB XC Network License Server Address: [redacted]

Server Port: [redacted]

Connection verified

4 **CI / CD Wizard**

Steps: 1. Select Action, 2. Build Environment, 3. Jenkins Settings, 4. Static Code Analysis, 5. Simulator Testing, 6. Hardware Testing, 7. Documentation, 8. Summary

Jenkins Settings (3 of 8)

Build agent label: any

Docker host agent label: docker

☐ Use Custom Docker Registry Settings

Docker Registry URL: https://registry.hub.docker.com/

Docker Registry Credentials ID:

5 **CI / CD Wizard**

Steps: 1. Select Action, 2. Build Environment, 3. Jenkins Settings, 4. Static Code Analysis, 5. Simulator Testing, 6. Hardware Testing, 7. Documentation, 8. Summary

Static Code Analysis (4 of 8)

☒ Enable MISRA C Verification

Report Plugin: Warnings Next Generation (default)

This selection requires the Warnings Next Generation plugin installed on the Jenkins server. The same plugin can be used to compile warnings and error trends. For more information, see the Warnings Next Generation plugin documentation.

6 **CI / CD Wizard**

Steps: 1. Select Action, 2. Build Environment, 3. Jenkins Settings, 4. Static Code Analysis, 5. Simulator Testing, 6. Hardware Testing, 7. Documentation, 8. Summary

Simulator Testing (5 of 8)

☒ Enable Simulator Testing

Configuration to Build and Run on Simulator: samd21xplp

☒ Enable Simulator Code Coverage

☒ Scan Output for Unity Test Results

Capture Output from Serial Communication: uart1

7 **CI / CD Wizard**

Steps: 1. Select Action, 2. Build Environment, 3. Jenkins Settings, 4. Static Code Analysis, 5. Simulator Testing, 6. Hardware Testing, 7. Documentation, 8. Summary

Hardware Testing (6 of 8)

☒ Enable Hardware Testing

Configuration to Build and Run on ICE4: samd21xplp

IP Address of ICE4: [redacted]

☒ Enable MPLAB Code Coverage

☒ Scan Output for Unity Test Results

Hardware testing requires MPLAB ICE 4 and breakout board. For more information, see the Hardware Testing plugin documentation.

Dockerfile generated by CI/CD Wizard

```
#-using default URL: hub.docker.com
FROM debian:buster-slim
ENV DEBIAN_FRONTEND noninteractive
#-install addOn-tools needed ...
```

What does it do?

-> see CICD-jenkinsPipeline

What does it do?

-> will setup all for ICE4-based

What does it do?

Will add command line call to run codeCoverage

24014DEV4

Microchip

MPLAB®X v6.x CICD-wizard: output Jenkinsfile

Jenkinsfile

```
CICDwiz-Jenkinsfile
1  #!/usr/bin/env groovy
2
3  // This file was generated by the CI/CD wizard
4  // See the user guide for information on how to use it
5
6  pipeline {
7      environment {
8          BUILD_CONFIGURATION = 'samd21xp'
9          SUPPORT_SCRIPT_REPO = 'https://github.com/microchip/cicd-wizard'
10         SIMULATE_BUILD_CONFIGURATION = 'samd21xp'
11         SIMULATE_COVERAGE_DIR = 'simcov'
12         SIMULATE_COVERAGE_XML = 'simcov.xml'
13     }
14     agent any
15     stages {
16         stage('Build') {
17             agent {
18                 dockerfile {
19                     label 'docker'
20                     filename 'Dockerfile'
21                     registryUrl 'https://hub.docker.com/'
22                 }
23             }
24             steps {
25                 sh(
26                     label: 'Generate build makefiles',
27                     script: "prjMakefilesGenerator.sh -v -f ./@${env.BUILD_CONFIGURATION}"
28                 )
29                 sh(
30                     label: 'Running Makefile',
31                     script: """
32                         rm -rf ./build
33                         rm -rf ./dist
34                         make clean
35                         make CONF=${env.BUILD_CONFIGURATION}
36                     """
37                 )
38                 stash name: 'build',
39                     includes: 'dist/**/*',
40                     allowEmpty: true
41             }
42         }
43
44         stage('Analyse') {
45             agent {
46                 dockerfile {
47                     label 'docker'
48                     filename 'Dockerfile'
49                     registryUrl 'https://hub.docker.com/'
50                 }
51             }
52             steps {
53                 sh(
54                     label: 'Running MISRA Analysis',
55                     script: """
56                         rm -rf cppcheck-result.xml
57                         misracli.sh --xml cppcheck-result.xml ./ || true
58                     """
59                 )
60                 stash name: 'misra',
61                     includes: 'cppcheck-result.xml',
62                     allowEmpty: true
63             }
64         }
65
66         stage('Simulate') {
67             agent {
68                 dockerfile {
69                     label 'docker'
70                     filename 'Dockerfile'
71                     registryUrl 'https://hub.docker.com/'
72                 }
73             }
74             environment {
75                 SIMULATE_BUILD_CONFIGURATION = 'samd21xp'
76                 SIMULATE_COVERAGE_DIR = 'simcov'
77                 SIMULATE_COVERAGE_XML = 'simcov.xml'
78             }
79             steps {
80                 sh(
81                     label: 'Run MDB script on simulator',
82                     script: """
83                         mdb.sh ${env.SIMULATE_MDB_SCRIPT_FILE}
84                         rm -rf ${env.SIMULATE_BINARY_FILE}
85                     """
86                 )
87                 sh(
88                     label: 'Convert simulator coverage to xml and html using gcov / gcovr',
89                     script: """
90                         if [ -f "${env.SIMULATE_COVERAGE_FILE}" ]; then
91                             curl ${env.SUPPORT_SCRIPT_REPO}/simcov2gcov.js -o simcov2gcov.js
92                             node simcov2gcov.js ${env.SIMULATE_COVERAGE_FILE}
93
94                             rm -rf ${env.SIMULATE_COVERAGE_DIR}
95                             mkdir ${env.SIMULATE_COVERAGE_DIR}
96                             gcovr --use-gcov-files --keep --print-summary --xml --output ${env.SIMULATE_COVERAGE_XML}
97                             gcovr --use-gcov-files --keep --print-summary --html --html-details --output ${env.SIMULATE_COVERAGE_DIR}/index.html -r .
98                         else
99                             echo "Warning: The simulation did not generate any coverage data"
100                         fi
101                     """
102                 )
103                 stash name: 'simulate',
104                     includes: "${env.SIMULATE_COVERAGE_DIR}/**/*,${env.SIMULATE_COVERAGE_XML}",
105                     allowEmpty: true
106             }
107         }
108     }
109 }
```

misra check

compile

codeCoverage

MPLAB®X v6.x CI/CD-wizard: output Dockerfile

Dockerfile

```
# Dockerfile generated by the CI/CD Wizard version 1.0.375.
#-start from a prepared docker-image with a debian-distro from hub.docker.com
FROM debian:buster-slim
ENV DEBIAN_FRONTEND noninteractive

#-install addOn-tools needed into the preloaded debian-image
USER root
RUN dpkg --add-architecture i386 && apt-get update -yq ...
    && apt-get clean && rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*

# Download and install MPLAB X IDE version 5.55
ENV MPLABX_VERSION 5.50
RUN curl -fSL -A "Mozilla/4.0" -o /tmp/mplabx-installer.tar \
    "https://ww1.microchip.com/downloads/en/DeviceDoc/MPLABX-v\${MPLABX\_VERSION}-linux-installer.tar" \
    && tar xf /tmp/mplabx-installer.tar -C /tmp/ && rm /tmp/mplabx-installer.tar \
    && USER=root ./tmp/MPLABX-v${MPLABX_VERSION}-linux-installer.sh ... \
    && rm ./tmp/MPLABX-v${MPLABX_VERSION}-linux-installer.sh ...
ENV PATH /opt/microchip/mplabx/v${MPLABX_VERSION}/mplab_platform/bin:$PATH

# Download and install toolchain
ENV TOOLCHAIN_VERSION 2.50
RUN curl -fSL -A "Mozilla/4.0" -o /tmp/${TOOLCHAIN}.run \
    "https://ww1.microchip.com/.../xc32-v\${TOOLCHAIN\_VERSION}-full-install-linux-installer.run" \
    ... && rm /tmp/${TOOLCHAIN}.run
ENV PATH /opt/microchip/${TOOLCHAIN}/v${TOOLCHAIN_VERSION}/bin:$PATH

# Download and install Microchip.SAMD21_DFP.3.6.133
RUN curl -fSL -A "Mozilla/4.0" -o /tmp/tmp-pack.atpack ...

ENV BUILD_CONFIGURATION samd21xplp
#-install more addOn-tools into debian-image needed eg for doxygen or simulation
RUN apt-get update -yq ... && apt-get clean && rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*
```

MPLAB®X v6.x CICD-wizard (summary)

- So, what is the MPLAB®X CICD-wizard?
 - basically, a guided flow to create two outputs, a Jenkinsfile and a Dockerfile
 - without **a lot of further** investigation, it is difficult to understand what you actually get
 - This class is supposed to provide all knowledge needed to start with this great MPLAB®X extension
- Where, how does the MPLAB®X CICD-wizard help ?
 - Once you understand the concept of CICD, it is a great source of info
 - Provides syntax for all complex calls like misra-checker, doxygen-creation, codeCoverage
 - Shows how to do a very sophisticated simulation with stimuli creation via Jscript
 - Provides a complete Jenkins pipeline with all stages&steps
 - Though leaves out parts, like HW-in-the-loop testing with MDBCore, which is explained in this class
- ***So, once you have the big picture then the MPLAB®X CICD-wizard is very helpful
->exactly this is the goal of this class to***
 - explain the big picture of a CICD-system and its major pillars and provide easy start into CICD
 - Highlight some advantages of the MPLAB®X ecosystem with
 - MisraCheck, codeCoverage, ...
 - Simulation or HW-in-the-loop testing

Still following?

Seen so far:

- 1. Intro, what is CI/CD, structure, ...*
- 2. MPLAB-analysis & -testing capabilities*
- 3. Used FW, how to build (part1) & run + expected results (part2)*
- 4. CI/CD-pillar1: SCM/git*
- 5. CI/CD-pillar2: Jenkins-Master & -Node/Agent & -Pipeline and last looked at the Pipeline-script (=the bread&butter)*
- 6. CI/CD-pillar3: runtime environment (phyPC or Docker)*
- 7. Demos: compile-in-J-pipeline only 1xStage and no Docker & error+olutions*
- 8. review CI/CD-wizard, with better understanding now*

Next

- Demos: HWrun (part2 of first demo) -> run in DockerContainer*

Agenda

- Introduction to CICD
- MPLAB®X ecosystem testing features and analysis capabilities
- Demos (part1): used FW and expected result
- Three pillars of a CICD system:
 - SourceControlManagement=SCM/git
 - Automation/Jenkins: Jenkins-Master -> -Agent -> Pipeline -> Demo (bash-in-Pipe)
 - RuntimeEnv/PC-Docker
- Demos (part2): simple-pipeline, Jenkins-Trigger, errors+solutions
- MPLAB®X CICD wizard
- **Demos (part3): HW-in-the-loop testing, DockerContainer**
- Summary and Outlook

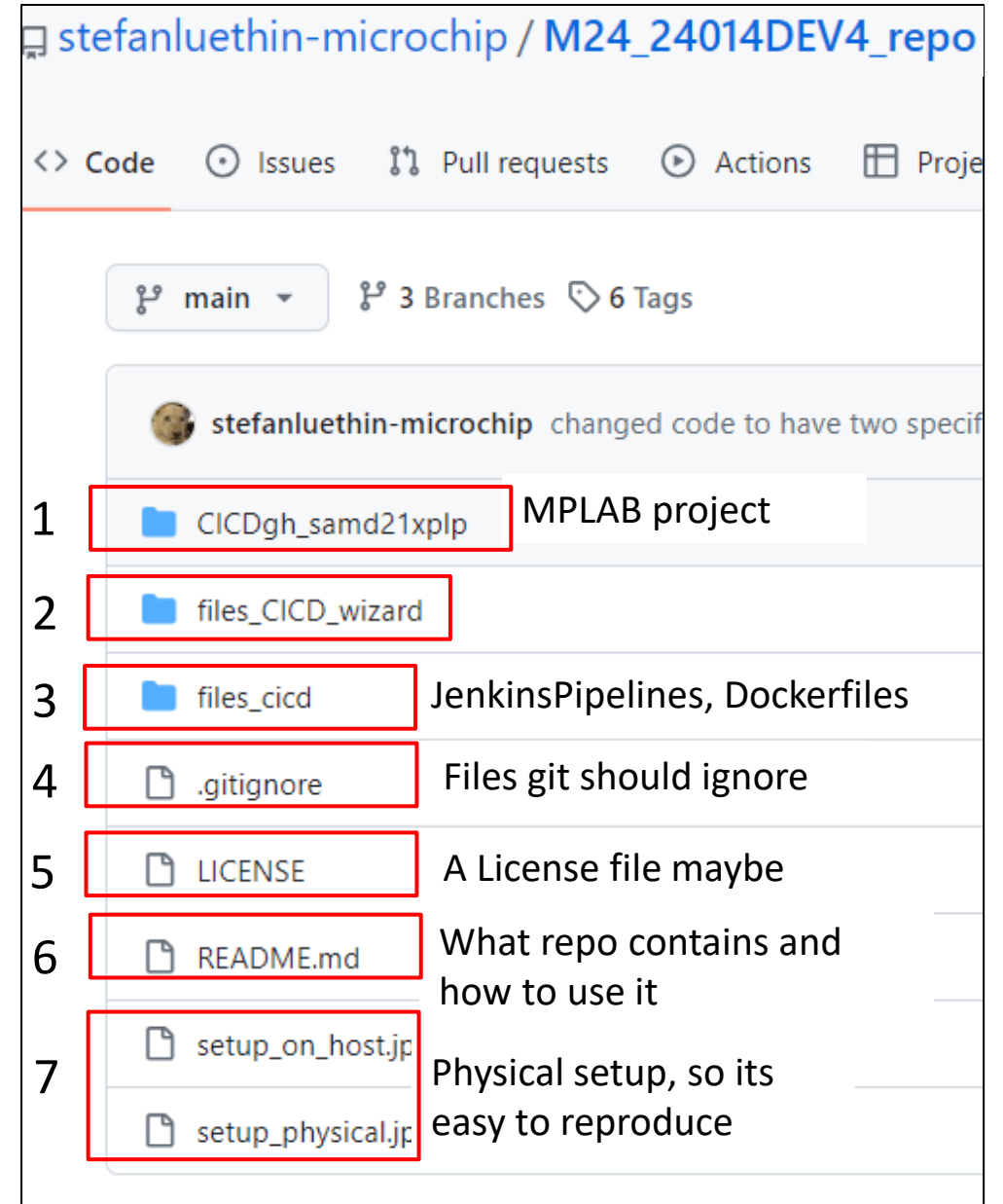
Demos on github (*just reminder*)

All labs/files are on the following github-repo:

www.github.com/stefanluethin-microchip/M24_24014DEV4_repo

<git_repo: M24_24014DEV4_repo>/

- 1 | |- CICDgh_samd21xplp/
 - 1 | |-firmware/ = MPLAB-project
 - 1 | |-mdb/ = MDBCore-scripting
- 2 | |- files_cicd_wizard = Output of MPLAB®X CICD-wizard
- 3 | |- files_cicd = JenkinsPipelines, Dockerfiles of this class
- 4 | |- .gitignore = git ignore file
- 5 | |- LICENSE = a license file
- 6 | |- README.md = Readme for repo
- 7 | |- setup_*.jpg = pictures of setup



Demo3: HW-in-the-loop

- Demo0 contains two parts

Part1 = compile: preparation -> create Makefiles -> compile

```
()> C:\CICDgh_samd21xplp_test1a_Setup_env.bat && C:\CICDgh_samd21xplp_test2a_comp.bat
```

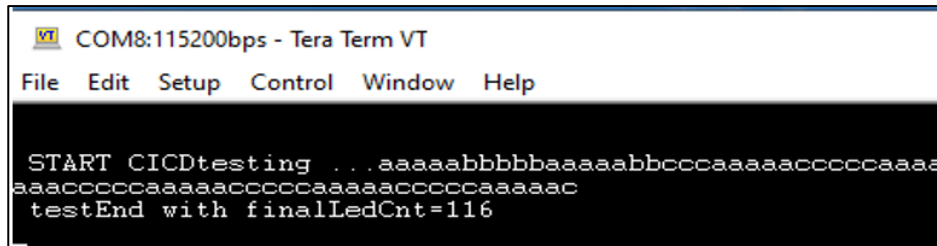
Part2 = Run-on-HW: connect-2-debugger -> download *.elf -> run

```
()> ...\\CICDgh_samd21xplp_test3a_mdbbat-run.bat
```

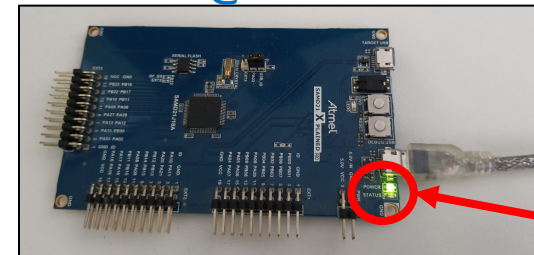
- Demos to come:

- Demo1 will re-do Demo0/part1=compile but from *Jenkins-pipeline*
- Demo2 will just show typical errors&solutions

- Demo3 will re-do Demo0/part1&part2=compile & HW-run but from *Jenkins-pipeline*
printf()/vCOM/console -> msg on UART



LEDblink @ 1Hz



- Demo4 will re-do Demo0/part1=compile but from *Jenkins-pipeline inside a DockerContainer*

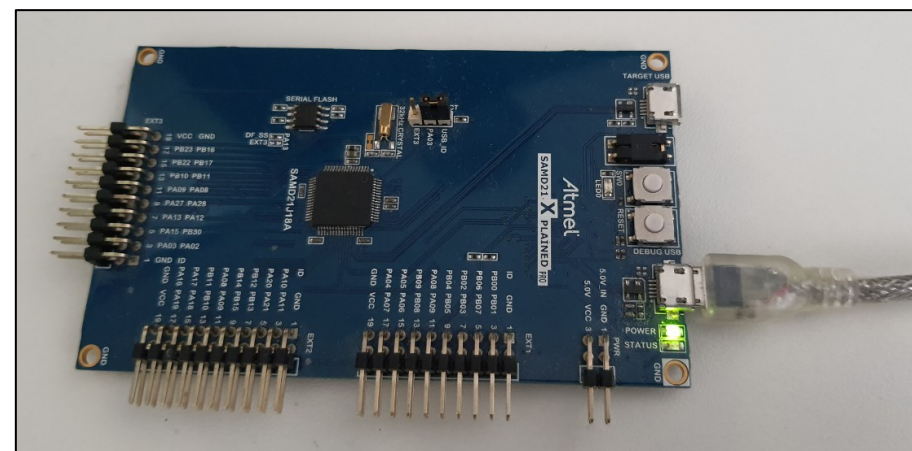
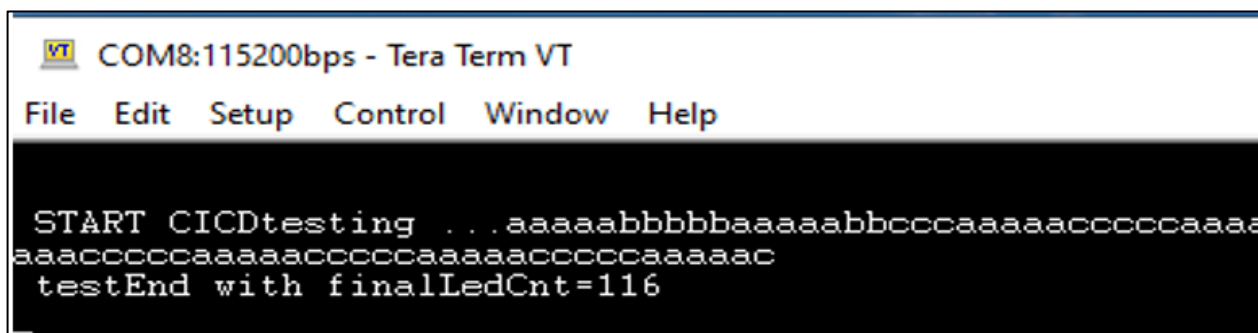
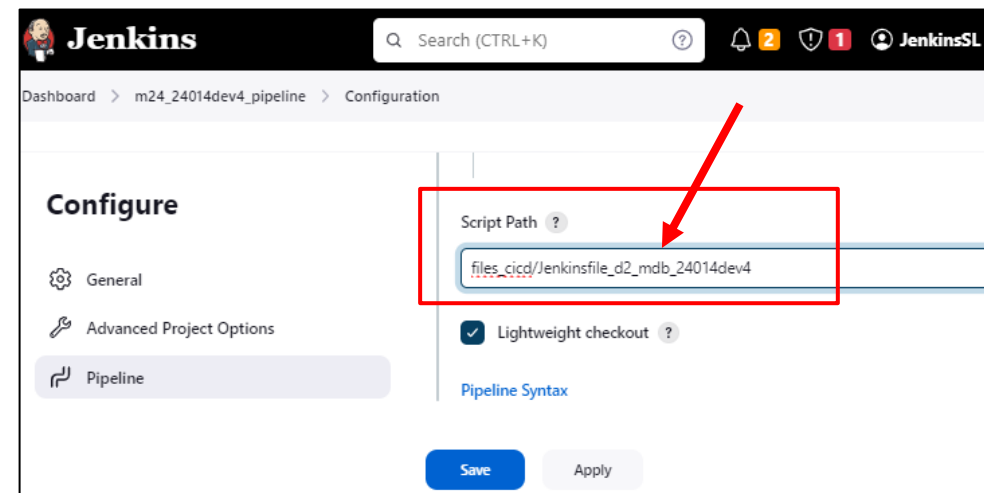
skip-4-time-just-mentiond, but don't run

Demo3: HW-in-the-loop

Demo3: ==Demo1 + *mdb*/hw-in-the-loop

- Use pipeline *m24_24014dev4_pipeline*
- pipeline-script *files_cicd\Jenkinsfile_d2_mdb_24014dev4*
- This Pipeline-script has 2 stages 'Build' and 'Hwrun'
-> so this runs both parts of Demo0 compile+hwrun
- SAMD21_xpl-HW: *attached-2-J_agents !!*
- trigger execution: *manually*

->run demo3 now



Demo4: Docker

- Demo0 contains two parts

Part1 = compile: preparation -> create Makefiles -> compile

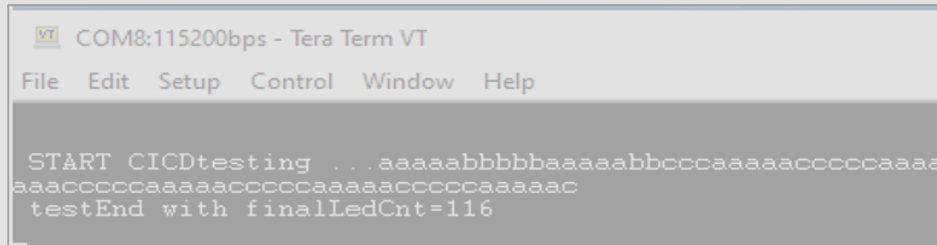
```
()> C1CDgh_samd21xplp_test1a_Setup_env.bat && C1CDgh_samd21xplp_test2a_comp.bat
```

Part2 = Run-on-HW: connect-2-debugger -> download *.elf -> run

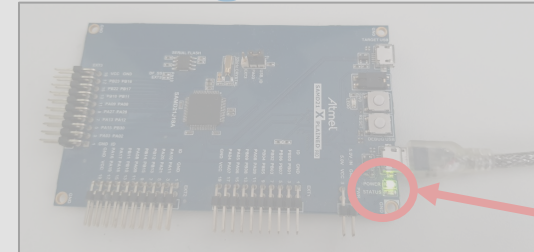
```
()> ...\C1CDgh_samd21xplp_test3a_mdbbat-run.bat
```

- Demos to come:

- Demo1 will re-do Demo0/part1=compile but from *Jenkins-pipeline*
- Demo2 will just show typical errors&solutions
- Demo3 will re-do Demo0/ part1&part2=compile & HW-run but from *Jenkins-pipeline*
printf()/vCOM/console -> msg on UART



LEDblink @ 1Hz



- Demo4 will re-do Demo0/part1=compile but from *Jenkins-pipeline inside a DockerContainer*

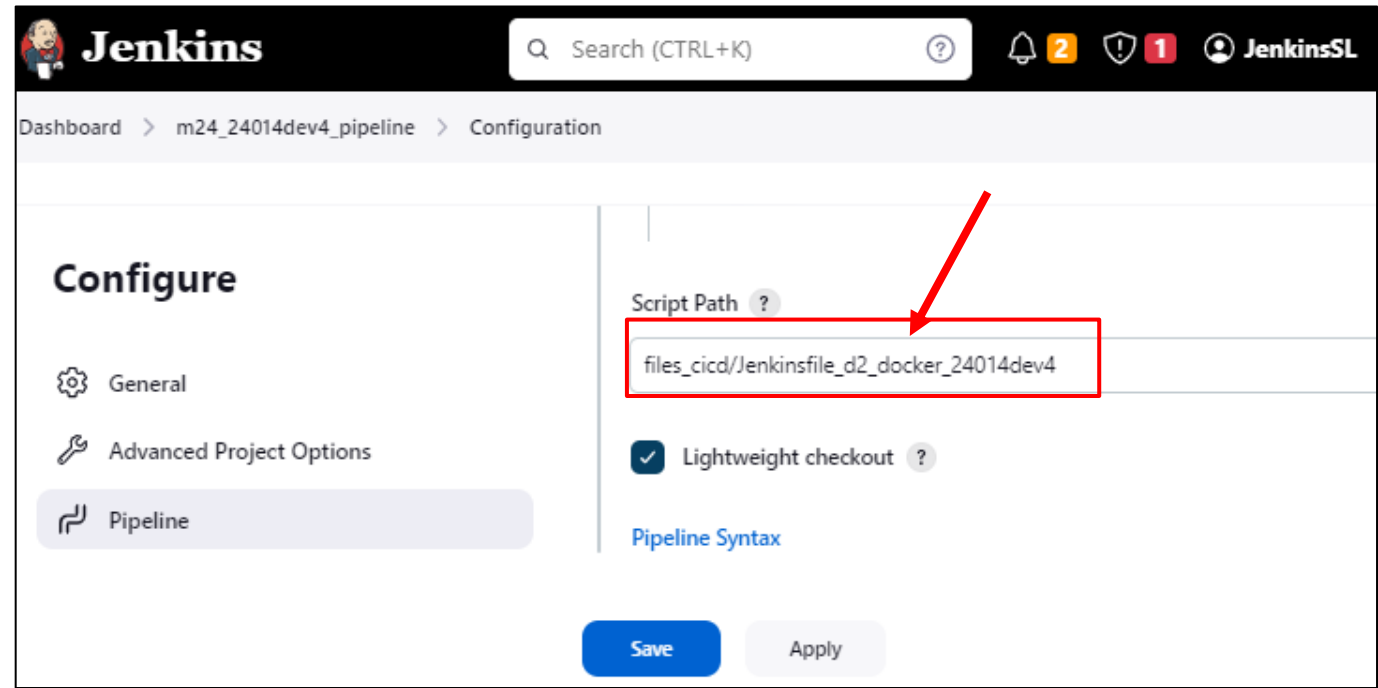
Demo4: Docker

Demo4: ==Demo1 + **Docker**

Task: re-do demo1 but this time

- Use pipeline [m24_24014dev4_pipeline](#)
- pipeline-script `files_cicd\Jenkinsfile_d2_docker_24014dev4`
- Pipeline-script has only one stage 'Build'
-> runs again only part0 of Demo0 compile
- No SAMD21_xpl-HW needed
- trigger execution: **manually**

->run [demo4 now](#)



Summary of all demos

- Demo0 contains two parts

Part1 = compile: preparation -> create Makefiles -> compile

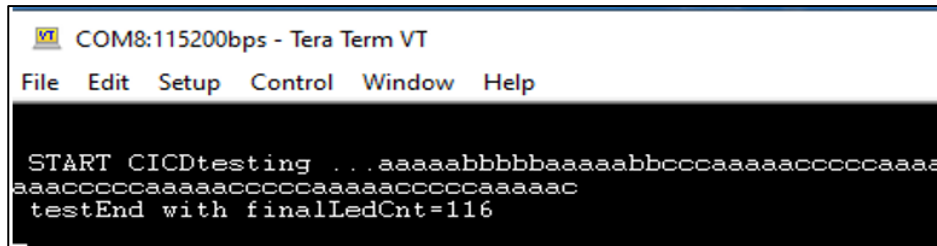
```
()> C:\CICDgh_samd21xplp_test1a_Setup_env.bat && C:\CICDgh_samd21xplp_test2a_comp.bat
```

Part2 = Run-on-HW: connect-2-debugger -> download *.elf -> run

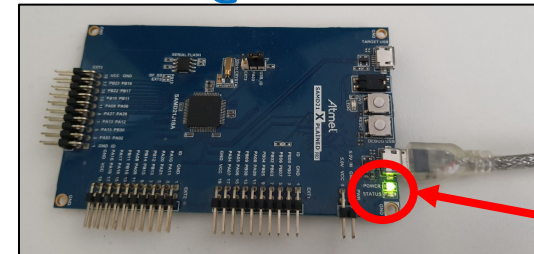
```
()> ...\\CICDgh_samd21xplp_test3a_mdbbat-run.bat
```

- Demos seen:

- Demo1 will re-do Demo0/part1=compile but from **Jenkins-pipeline**
- Demo2 will just show typical errors&solutions
- Demo3 will re-do Demo0/part1&part2=compile & HW-run but from **Jenkins-pipeline**
printf()/vCOM/console -> msg on UART



LEDblink @ 1Hz



- Demo4 will re-do Demo0/part1=compile but from **Jenkins-pipeline inside a DockerContainer**

Still following?

Seen so far:

- 1. Intro, what is CI/CD, structure, ...*
- 2. MPLAB-analysis & -testing capabilities*
- 3. Used FW, how to build (part1) & run + expected results (part2)*
- 4. CI/CD-pillar1: SCM/git*
- 5. CI/CD-pillar2: Jenkins-Master & -Node/Agent & -Pipeline and last looked at the Pipeline-script (=the bread&butter)*
- 6. CI/CD-pillar3: runtime environment (phyPC or Docker)*
- 7. Demos: compile-in-J-pipeline only 1xStage and no Docker & error+olutions*
- 8. review CI/CD-wizard, with better understanding now*
- 9. Demos: HWrun (part2 of first demo) -> run in DockerContainer -> wrapUp*

Next

- wrapUp, Summary, Outlook*

Agenda

- Introduction to CICD
- MPLAB®X ecosystem testing features and analysis capabilities
- Demos (part1): used FW and expected result
- Three pillars of a CICD system:
 - SourceControlManagement=SCM/git
 - Automation/Jenkins: Jenkins-Master -> -Agent -> Pipeline -> Demo (bash-in-Pipe)
 - RuntimeEnv/PC-Docker
- Demos (part2): simple-pipeline, Jenkins-Trigger, errors+solutions
- MPLAB®X CICD wizard
- Demos (part3): HW-in-the-loop testing, DockerContainer
- **Summary and Outlook**

Summary: setup a CI/CD system

Setting up a CI/CD system

- pillar1/source-code-management:
 - select a source-code-management=**SCM**-tool eg: git
- pillar2/automation:
 - automation with eg: **Jenkins**
 - Jenkins-Master (Linux, Window -> selected Linux as easier to handle and license-free)
 - Install Jenkins-SW with

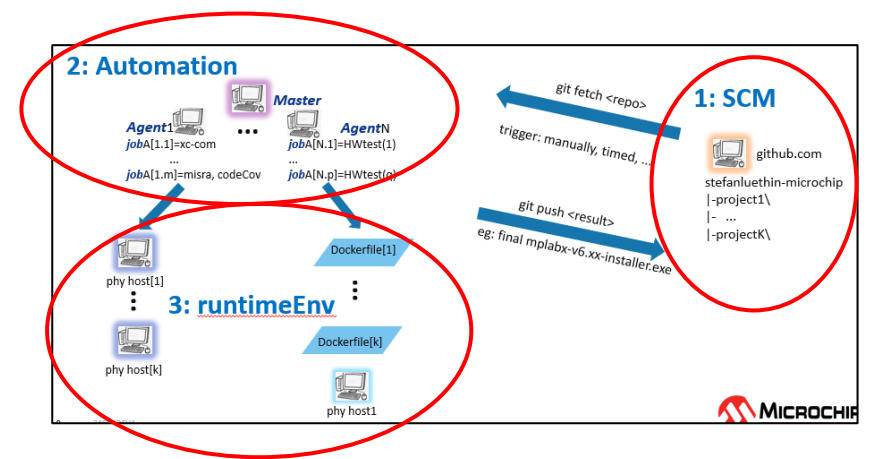
```
(lnx)$ sudo apt-get install openjdk-8-jdk && sudo apt-get install jenkins
```
 - Start Jenkins-master (=Linux-service) or check if up with
 - start:

```
(lnx)$ systemctl start jenkins
```
 - Jenkins-Master is up with:

```
(lnx)$ service --status-all | grep jenkins
```

```
[ + ] jenkins
```
 - Jenkins-Agent(s) -> log into webIF (URL: <IP-of-jenkinsMaster>:8080)
In webIF (->newItem) create new agent, num executors, root/working-directory... and start agent with

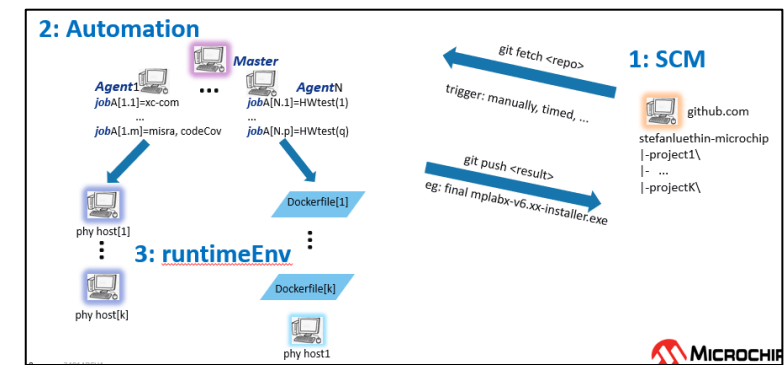
```
(lnx)$ java -jar agent.jar -jnlpUrl"http://..." -secret @secret-file -workDir ...
```
 - Create Jenkins Pipeline = 'sequence of tasks to do'
fetch+compile -> +analysis (misraCheck, codeCoverage) -> +testing (simulation, hw-in-the-loop)
- pillar3/**runtime-environment**:
 - PhysicalPC vs Docker -> usb-dongle-lic vs network-lic ...



Summary: start with CICD

Where, how to start with CICD?

- Setup CICD-system = last slide
- Select Testing-features to be used
 - SW: unitTesting , Simulation -> DockerContainer
 - HW-in-the-loop testing (-> physical PC): type? mdb, mdbcs, Java -> MU-class
- Create test-framework
 - 'a' test needs: testID, testName, testEndCriteria, testPassFailCriteria, testTimeout -> #ifdef TEST
 - testRegression-suite
 - Vmodel: unitTests->subsystemTests->systemTests->finalTest to prove all requirements met
 - Use CICD-system to continuously run regression (start with 1xtest->add more over time to full suite)
- Select Analysis-capabilities to be used
 - Checkout revision and compile+link -> obvious, but maybe different compiler options and version?
 - Setup Misra-checker -> which rules 'mandatory, required...' in which testrun? Finally need to pass all!
 - Setup codeCoverage -> store <date>_result_codeCoverage.html centrally in pipeline step 'publish'
- What you gain - just some thoughts:
 - Project planning - how far are we today and what's left? -> easy with full regression-suite and time/Test
 - Predictive development, reduce late failures,...



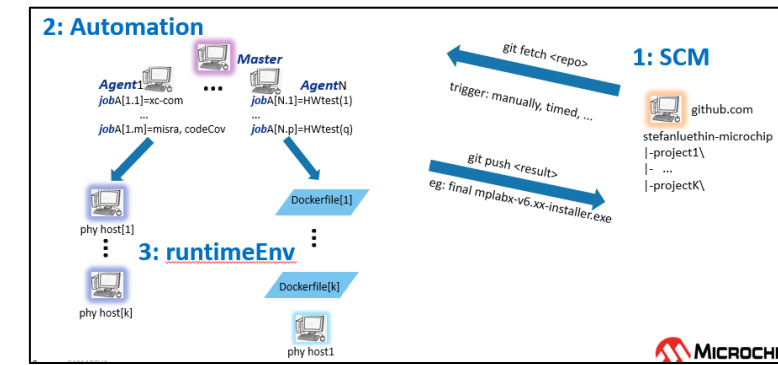
Agenda

- Introduction to CICD
- MPLAB®X ecosystem testing features and analysis capabilities
- Demos (part1): used FW and expected result
- Three pillars of a CICD system:
 - SourceControlManagement=SCM/git
 - Automation/Jenkins: Jenkins-Master -> -Agent -> Pipeline -> Demo (bash-in-Pipe)
 - RuntimeEnv/PC-Docker
- Demos (part2): simple-pipeline, Jenkins-Trigger, errors+solutions
- MPLAB®X CICD wizard
- Demos (part3): HW-in-the-loop testing, DockerContainer
- **Summary and Outlook**

Outlook: advanced topics

Some advanced topics

1. You can share data between J-Agents - eg:
 - J-Agent1 with a lot of RAM
 - J-Agent2 littleRAM+old, but HWrev1 attached
 - So, you can use J-Agent1 to continuously build and on certain triggers you can transfer the created *.elf to J-Agent2 to run it on HW (next slides)
2. You can collect various docs, like codeCoverage_report.xml, misra_report.txt, ... from various runs and store central, so you can easily view them in a meeting (next slides)
3. You can use webHooks (git-server must see J-Master => www.dyndns.org but might be blocked by IT) to monitor only 'release-branch' and start a certain pipeline (eg: pipe_release with 'compile->misra->codeCov ->run-on-releaseHWonly') (next slides)
4. You can setup mail-system, so you receive an email from certain pipeline-publish-states..
5. You can create very sophisticated simulations (remember why Simulation?)
-> use CI/CD-wizard output to study how to do this
6. ...



Outlook: advanced topics

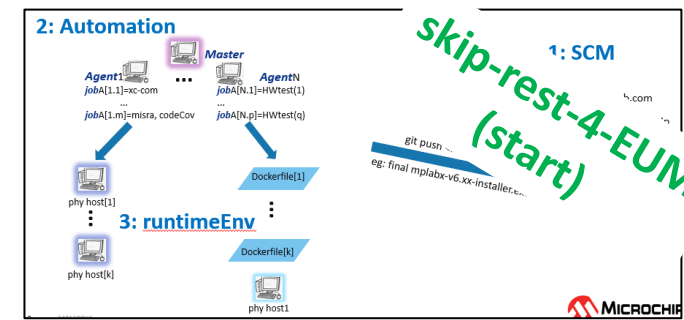
Some advanced topics (EMEA-Masters new)

1. How to store result of a stage?

- Let's see if the CI/CD-wizard provides hints? -> actually yes it does:
 - CI/CD-wizard created 'Jenkinsfile' contains a section *stash* at end of each stage
 - Search web for 'Jenkins pipeline meaning of stash' to find eg:
<https://www.jenkins.io/doc/pipeline/steps/workflow-basic-steps/#stash-stash-some-files-to-be-used-later-in-the-build>
-> so with *stash* you can store results created in this stage like logfiles centrally in
`$JENKINS_HOME/jobs/MY_JOB/builds/MY_BUILD/stashes/`
 - Further CI/CD-wizard *Jenkinsfile* has a `stage('Publish')` which shows how to handle *stashes*
- Although these type of *stash* is only valid for the current pipeline-run -> if you want to keep OR even share between Nodes/Agents, you need to setup a separate service like *an Artifactory* ...

2. How to connect a real HW to a DockerContainer?

-> See demo-video that explains this topic: <https://www.youtube.com/watch?v=ULemsUTU5v4>



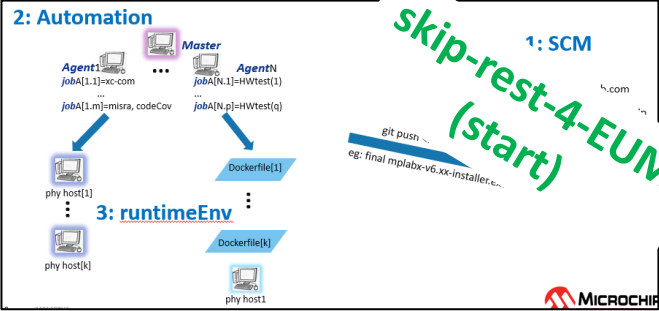
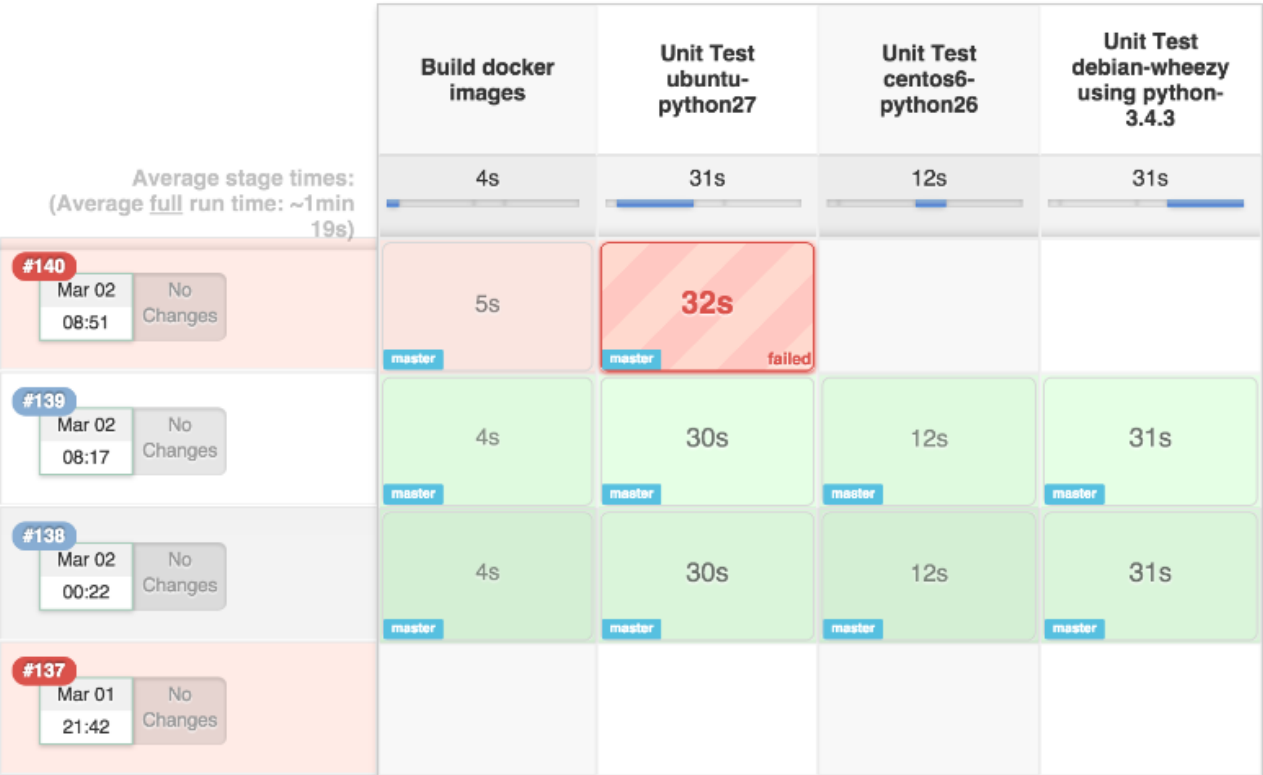
Outlook: advanced topics

Some interesting Jenkins-plugins

- 1. For easier view of build-stages

Stage View:

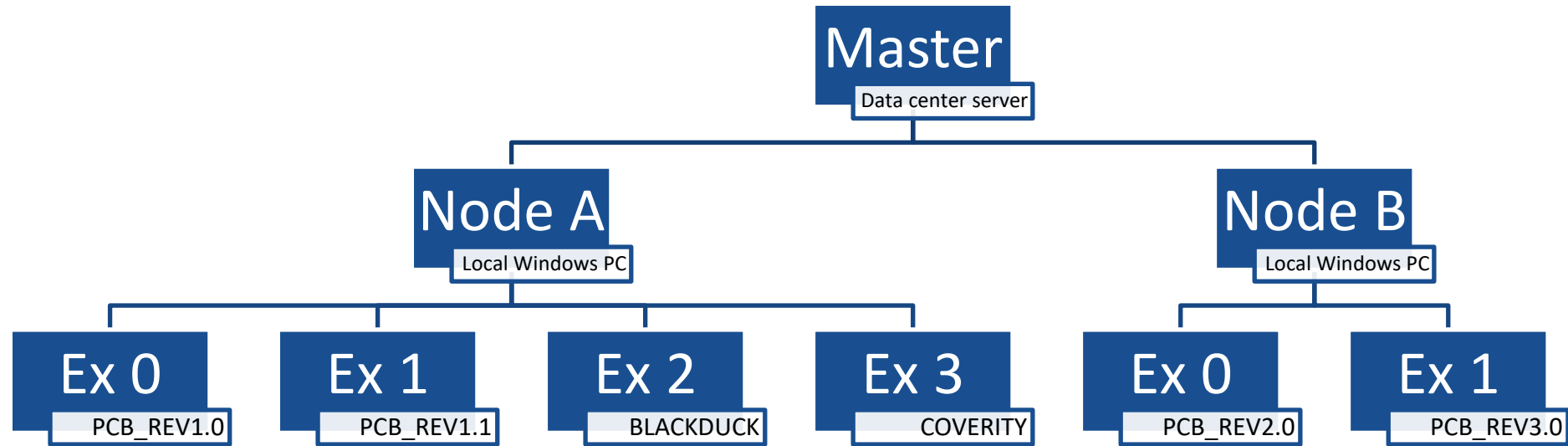
Stage View



Outlook: Best practices for Embedded SW Build & Test

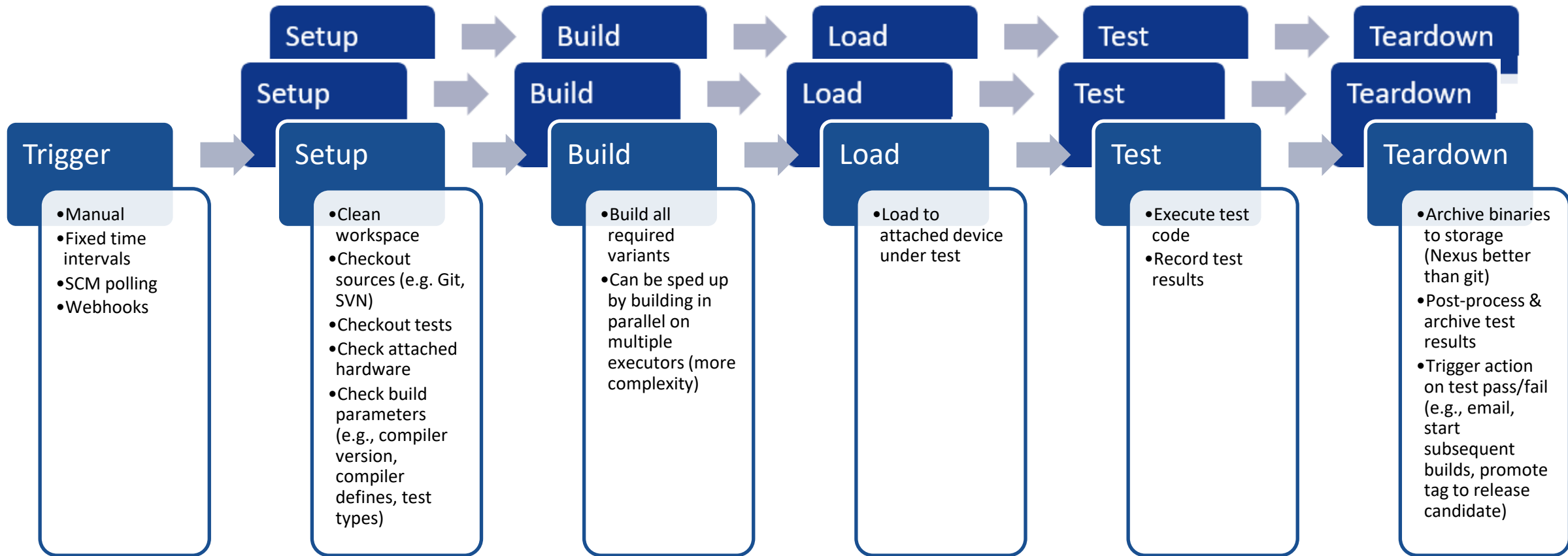
- **Unify test entry / exit & pass/fail/error definitions**
 - Use a common strategy to read test result from embedded device, e.g., by hitting a exit breakpoint and reading signatures from registers, or by sending test results through UART/SPI
 - Use a common pass/fail/error definition & try to always generate same total amount of tests
- **Don't go overboard because you can**
 - Log enough information to locate errors, but don't log everything
- **Use sandbox environments & backups**
 - Jenkins plugins can be installed with mouse-click, but can bring down entire system & are difficult to remove
- **Think about security**
 - Use https instead of http
 - Use credentials providers (e.g., git) and matrix-based user permissions
 - Jobs can have access to restricted source code or IP – and so have its users

Outlook: Jenkins Embedded SW Build & Test Node Setup



- Jobs are controlled by a master. Every run of a job is a build. Builds run on nodes. One node can have multiple executors
- Jobs can require specific HW or SW capabilities, usually defined by labels („PCB_REV1.0“)
- Above setup could be used to perform development and release testing for a project where multiple hardware revisions need to be supported, and BlackDuck open source compliance and Coverity MISRA/static analysis is required

Outlook: a potential complete Jenkins-setup



Outlook: Jenkins Pipeline types

- Jenkins pipelines
 - can be written in different languages
 - can have different structures

all intelligence in called python-script -> simple pipeline

```
pipeline {
  agent any

  stages {
    stage('Build') {
      steps {
        sh('''
          export PATH=$PATH:/opt/microchip/mplabx/v5.50/mplab_platform/bin/
          echo "Check license"
          xclm -status
          ''')
        sh('''
          export PATH=$PATH:/opt/microchip/mplabx/v5.50/mplab_platform/bin/
          python3 scripts/python/mplabxprojects.py cleanBuild BootloaderEmmcXip
          ''')
        sh('''
          export PATH=$PATH:/opt/microchip/mplabx/v5.50/mplab_platform/bin/
          python3 scripts/python/mplabxprojects.py cleanBuild MsdEmmc
          ''')
      }
    }
  }
}
```

all intelligence in pipeline -> complex pipeline

```
1  #!/usr/bin/env groovy
2
3  // This file was generated by the CI/CD Wizard version 1.0.375.
4  // See the user guide for information on how to customize and use this file.
5  //-SL: Jenkins Pipeline syntax type = Declarative
6
7  pipeline {
8    environment {
9      DEMO_V = "demo_simple_d1"
10     JPIPELINE = "Jenkinsfile_d2"
11     //-SL: root path to tools and IDE
12     MPLABX_ROOT = "/opt/microchip/mplabx"
13     ...
14
15     ELF_TEST_SCR = "." + "result_check.sh"
16   } //- end environment
17
18   //-SL: instead of 'any agent' use a DockerContainer described in 'Dockerfile_4_J_d2'
19   agent any
20   stages {
21     stage('Build') {
22       agent {
23         dockerfile {
24           label 'DEM-LT-M16422u_localLnxPC'
25           filename 'Dockerfile_4_J_d2'
26           registryUrl 'https://hub.docker.com/'
27         }
28       } //- run 'build' stage in DockerContainer
29       steps {
30         sh(
31           label: 'Generate build makefiles',
32           script: """
33             echo "###-SL: re-creating makefiles for ${env.PRJ_MK_STR}"
34             #-SL: need to provide path to prjMakefilesGenerator, whereas below for make you don't need to provide
35             #-SL: a path, as it is extracted from the configuration.xml file
36             #-SL: "${MPLABX_ROOT}" + "/" + "${MPLABX_V_2_USE}" + "/mplab_platform/bin/" + prjMakefilesGenerator.sh
37             #-SL /opt/microchip/mplabx/v6.05/mplab_platform/bin/prjMakefilesGenerator.sh -v -f ./CICDgh_samd21xplp
38             prjMakefilesGenerator.sh -v -f ./CICDgh_samd21xplp/firmware/CICDgit_samd21xplp.X@samd21xplp
39             """
40         ) //-SL: end 'step: update/build Makefiles'
41         sh(
42           label: 'compile = running Makefile',
43           script: """
44             #-SL: above for prjMakefilesGenerator need to provide path to it, whereas for make
45             #-SL: you don't need to provide a path, as it is extracted from the configuration.xml file
46             cd "${env.PRJ_WS_REL_F}"
47             """
48         )
49       }
50     }
51   }
52 }
```

- Jenkins plugins to visualize results - eg: BlueOcean to see your Stages in a graphical way

Summary: CICD

The CICD-wizard and the goal of this class

- MPLAB®X CICD-wizard is a Q&A-flow that results in 2x files: JenkinsPipeline, DockerFile
- Without further knowledge and many hours studying what CICD, Jenkins, Docker and all these tools and concept do -> this where this class should help you to get started
- After this class the MPLAB®X CICD-wizard is very helpful because
 - It creates full and complete Jenkins-Pipeline with all calls, like how to start codeCoverage
 - So, you can just extract what you need and quickly get the syntax
 - It shows you what's possible, eg the ideal Simulation-setup, but way too complex for beginning

!! Good luck !!

Course Survey

- We appreciate your feedback
- [24014 DEV4 Class Survey](#)



END

Thank you

LEGAL NOTICE

MASTERS 2024

SOFTWARE:

Microchip software provided to you during the MASTERS 2024 conference is provided “AS IS”, without warranty. Microchip assumes no liability or responsibility for your use of any Microchip software. You must use Microchip software exclusively with Microchip products. Further, use of Microchip software is subject to copyright and the disclaimers and license terms accompanying such software, whether set forth at the install of each program or posted in a header or text file.

Notwithstanding the above, certain components of software offered by Microchip and 3rd parties may be covered by “open source” software licenses, including licenses that require that the distributor of such software make the software available in source code format. To the extent required by any such open source software license, the terms of such license govern.

Any transfer of Microchip product, technology, or software may be subject to export controls. Microchip encourages any entity transferring product, technology, or software to seek appropriate legal advice and/or consult the applicable governmental agencies prior to transfer. The information provided herein is subject to change at any time and without notice.

NOTICE & DISCLAIMER:

Materials, software and accompanying information (including, for example, any references to 3rd party companies and 3rd party websites) are for informational purposes only and are provided “AS IS.” Microchip assumes no responsibility for statements made by 3rd party companies, or materials or information that such 3rd parties may provide.

MICROCHIP DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, OR STATUTORY, INCLUDING ANY IMPLIED WARRANTIES OF NONINFRINGEMENT, MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY DIRECT OR INDIRECT, SPECIAL, PUNITIVE, INCIDENTAL, OR CONSEQUENTIAL LOSS, DAMAGE, COST, OR EXPENSE OF ANY KIND RELATED TO THESE MATERIALS OR ACCOMPANYING INFORMATION PROVIDED TO YOU BY MICROCHIP OR OTHER THIRD PARTIES, EVEN IF MICROCHIP HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES OR THE DAMAGES ARE FORESEEABLE. PLEASE BE AWARE THAT THE IMPLEMENTATION OF INTELLECTUAL PROPERTY PRESENTED IN MASTERS 2024 MAY REQUIRE A LICENSE FROM MICROCHIP OR THIRD PARTIES.

TRADEMARKS:

The Microchip name and logo and the Microchip logo are registered trademarks of Microchip Technology Incorporated in the U.S.A. and other countries. Information on and a listing of Microchip trademarks is available at <https://www.microchip.com/en-us/about/legal-information/microchip-trademarks>.

All other trademarks mentioned herein are property of their respective companies.

COPYRIGHT:

© 2024 Microchip Technology Incorporated, All Rights Reserved.